راهنمای مهندسی SwitchLoc — روش‌ها، محدودیت‌ها و نمایه کد

ویرایش ۱.۲۲ — راهنمای کاربرد، مفروضات، روابط محاسبات و نمایه خودکار منبع. داده و خروجی هر پروژه نیاز به تأیید مهندس دارد.

فهرست فصل‌ها

۱. گردش کار و پروژه
۲. شبکه و توپولوژی
۳. ورود و بازبینی خاموشی
۴. طبقه‌بندی و نرخ خرابی
۵. شاخص‌های پیوستگی
۶. مدل بار و تنظیمات مهندسی
۷. جایابی و N-1
۸. جایابی کلید و EENS
۹. نقشه و سبک
۱۰. داشبورد و خروجی
۱۱. محدودیت‌ها و رفع اشکال
۱۲. قرارداد واحد و تعاریف
۱۳. ساختار پروژه و ماندگاری داده
۱۴. ورود GIS، CRS و کنترل کیفیت
۱۵. ردیابی، زون و مسیر انتقال
۱۶. ورود فایل خاموشی و شناسایی فیلدها
۱۷. جزئیات قواعد متنی طبقه‌بندی
۱۸. پوشش زمانی و r/ با مثال عددی
۱۹. شاخص‌های پیوستگی با مثال عددی
۲۰. بار، ظرفیت و عدم قطعیت
۲۱. رتبه‌بندی فیدر و ارزش اقتصادی
۲۲. فرمول EENS و راهبرد جست‌وجو
۲۳. مکان خاموشی، UTM و نقشه
۲۴. خروجی، آزمون و پذیرش مهندسی
۲۵. راهنمای توسعه و دامنه مرجع کد
۲۶. روابط کلیدی در قالب چپ به راست

راهنمای کاربری و محاسبات SwitchLoc

این راهنما برای بازبینی مهندسی است و جایگزین کنترل داده، نقشه، تنظیمات حفاظتی یا تصمیم بهره‌برداری نیست. مقادیر فرضی را پیش از استناد رسمی تأیید کنید.

۱. گردش کار و پروژه

از Data Import شبکه GIS را وارد یا پروژه SQLite را باز کنید. تطبیق لایه‌ها، سیستم مختصات و هشدارهای کیفیت را بررسی کنید. پروژه را با Save یا Save As ذخیره کنید؛ فایل SQLite مستقل را با Open باز کنید. خاموشی‌ها پس از پردازش در پروژه ذخیره می‌شوند، نه صرفا با انتخاب نام فایل. پس از بازکردن پروژه توپولوژی را اجرا کنید تا بازبینی و نمایش مکان‌های خاموشی فعال شود. از داده اصلی نسخه پشتیبان بگیرید.

۲. شبکه و توپولوژی

فیدرها از نام کامل شبکه شناخته می‌شوند. فیدرهای با شماره مشترک ولی نام متفاوت را ادغام نکنید. توپولوژی شبکه را ردیابی می‌کند، زون‌ها و مرزها را می‌سازد و نقاط اتصال باز و مسائل مدل را گزارش می‌دهد. Tieهای کاندید را با نقشه و وضعیت واقعی تجهیزات تأیید کنید؛ پیشنهاد توپولوژی مجوز مانور نیست.

۳. ورود و بازبینی خاموشی

فایل جمعی یا فایل هر فیدر را انتخاب و پردازش کنید. پیش‌فرض واحد انرژی MWh است؛ قبل از محاسبه آن را با منبع کنترل کنید، چون مقدار پیش‌فرض مدرک واحد فایل نیست. برای هر فیدر فایل صحیح را با نام کامل مشخص کنید. پنجره زمانی و ماه‌های بدون رکورد را بررسی کنید. بازبینی مکان‌ها پس از اجرای توپولوژی: فیلتر و مرتب‌سازی ستون، انتخاب مکان/زون، تأیید یا رد و ذخیره تصمیم. پیشنهاد بانک مکان‌ها الزاما صحیح نیست. جدول بزرگ تدریجی بارگذاری می‌شود؛ تا تکمیل، ذخیره و فیلتر غیرفعال‌اند.

۴. طبقه‌بندی و نرخ خرابی

قاعده متنی به‌ترتیب داده ناکافی، برنامه‌ای، علت ناشناخته، خارج از دامنه، تعمیر، حفاظت و قابل کاهش با کلیدگذاری را بررسی می‌کند. کات‌اوت سرخط ممکن است نیازمند بررسی تلقی شود. کلمات به تنهایی قطعیت ماهیت رخداد نیستند. داده ناکافی و خارج از دامنه از شمارش حذف می‌شوند.

T = روزهای ماه‌های مؤثر / 365 سال؛ r = U = r / 60 × ساعت در سال. ماه بدون رکورد خودکار از میانگین مدت معتبر رخدادهای تصادفی به دقیقه، شامل صفر؛ T / تعداد تصادفی = T / قابل کاهش = تعداد طبقه قابل کاهش ؛ T . r / قابل کاهش مخرج حذف می‌شود مگر صریحا دارای پوشش اعلام گردد. صورت کسر در نسخه فعلی با ماه‌های مستثنا لزوما فیلتر نمی‌شود. داده صفر را با داده ناموجود اشتباه نگیرید.

۵. شاخص‌های پیوستگی

اگر تعداد مشترکان متأثر موجود نباشد برنامه از سهم بار تخمینی استفاده می‌کند: P(kW) / E(kWh) = f) × مدت(h)] و آن را حداکثر ۱ می‌کند. بدون انرژی یا بار، بریکر/ریکلوزر/دیژنکتور سهم ۱ می‌گیرند و رخدادهای جزئی بدون سهم کنار گذاشته می‌شوند. SAIFI = f / T ؛ SAIDI = f × مدت دقیقه) / T ؛ CAIDI = SAIDI / SAIFI ؛ ASAI = max(0, 1 – اینها تخمین سهم مشترکان با فرض سهم بار هستند و شاخص دقیق مشترک‌محور نیستند. جمع .(SAIDI/525600 سامانه وزنی با بار یا میانگین ساده فیدرهای قابل محاسبه است. واحد نامعلوم انرژی، بار فرضی، داده ناقص و مدت صفر روی نتیجه مؤثرند.

۶. مدل بار و تنظیمات مهندسی

در تنظیمات مهندسی بار P/Q، ظرفیت، ضریب بار، زمان‌ها، هزینه‌ها، اولویت بار و وزن معیارها را کنترل کنید. پیش‌فرض‌ها داده واقعی نیستند. انتخاب و کپی/پیست تنها در سلول‌های مجاز انجام شود. بار تخمینی از ظرفیت ترانس و ضریب بارگذاری ساخته می‌شود؛ برای نتیجه رسمی بار اندازه‌گیری‌شده وارد و منشأ ثبت شود.

۷. جایابی و N-1

تحلیل N-1 بر اساس زون، بار و مسیر انتقال قابل استفاده است. EP = 100 × بیشینه بار خاموش‌شده / بار کل؛ TL = 100 × بیشینه مستقل بار قابل انتقال / بار کل. رتبه فیدرها از نرمال‌سازی و وزن‌دهی تعداد اتصال، EP، معکوس TL، اولویت بار، و r ساخته می‌شود. خروجی به داده و وزن‌ها حساس است. در جایابی دستی نقطه را روی نقشه یا با UTM انتخاب، طرح را ارزیابی و تأیید کنید.

۸. جایابی کلید و EENS

نوع کلید، روش، r ،، سقف تعداد، زمان بازیابی و ظرفیت پشتیبان را کنترل کنید. روش یکنواخت و جهت‌دار r/ خاموشی را در صورت وجود می‌خوانند؛ در نبود داده از پیش‌فرض تنظیمات با هشدار استفاده می‌شود. انتخاب دستی صریح اولویت دارد. قطعه = فیدر × طول قطعه / طول کل. در مدل یکنواخت برای هر قطعه: EENS = قطعه / 60 × [بار قطعه معیوب × زمان تعمیر + بار سالم بازیابی‌شده × زمان بازیابی + بار بازیابی‌نشده × زمان تعمیر]. EENS کل جمع سهم قطعات به kWh/سال است. روش جهت‌دار به بار پایین‌دست و ساختار قطعه توجه می‌کند. زمان بازیابی حداکثر زمان تعمیر در نظر گرفته می‌شود. خطای گذرا برای ریکلوزر جداگانه مدل می‌شود. انتخاب حریصانه است نه اثبات بهینه سراسری. درصد بهبود = 100 × (EENS پایه - EENS طرح) / EENS پایه. دو روش می‌توانند مکان یکسان یا متفاوت انتخاب کنند. پاک‌کردن مطالعه فقط نتیجه و نماد پیشنهادی EENS را با تأیید حذف می‌کند، نه داده شبکه و خاموشی.

۹. نقشه و سبک

از فیلتر فیدرها، کنترل نمایش تک‌کلید و دسته کلید، زوم و انتخاب زون استفاده کنید. پنهان‌کردن تجهیزات فقط نمایش را تغییر می‌دهد. در Map Style رنگ، ضخامت و نماد فیدر، کلید، خاموشی و مختصات UTM را تنظیم کنید. خط خاموشی به مرکز زون به‌طور پیش‌فرض خاموش است. نمایش توزیع رنگی به رخدادهای دارای مختصات و فیدر انتخابی وابسته است؛ شعاع و آستانه‌ها نمایش تحلیلی‌اند و اثبات مکان خطا نیستند. پس‌زمینه OSM نیازمند دسترسی شبکه است.

۱۰. داشبورد و خروجی

داشبورد شاخص‌ها، بار، طول، وضعیت و پیشنهادها را نمایش می‌دهد. خروجی اکسل شاخص فیدرها و گزارش‌های مطالعه را ذخیره و منشأ اعداد، واحدها، هشدارها و فرض‌ها را بررسی کنید. علامت — به معنی ناموجود بودن داده است، نه صفر. ذخیره تصویر نقشه از منوی راست‌کلیک ممکن است.

۱۱. محدودیت‌ها و رفع اشکال

در صورت خالی بودن تب خاموشی پس از بازگشایی، وجود اجرای پردازش‌شده را بررسی و توپولوژی را مجددا اجرا کنید. فایل خاموشی انتخاب‌شده بدون پردازش، نتیجه ذخیره‌شده نیست. دوره پوشش، واحد انرژی و نام کامل فیدر را بررسی کنید. خاموشی برنامه‌ای از متن با قواعد محدود شناسایی می‌شود؛ علت نامعلوم نباید حدس زده شود. نتایج نقشه و EENS بدون تأیید میدانی و مهندسی قطعی نیستند.

مرجع مهندسی تفصیلی

۱۲. قرارداد واحد و تعاریف

توان اکتیو P بر حسب kW، توان راکتیو Q بر حسب kvar، توان ظاهری S بر حسب kVA، زمان خاموشی و بازیابی به دقیقه، طول شبکه به متر، نرخ به رخداد در سال و انرژی مدل بر حسب kWh/سال هستند. برای تبدیل MWh به kWh مقدار در ۱۰۰۰ ضرب می‌شود. واحد پیش‌فرض MWh در ورودی نرم‌افزار به‌هیچ‌وجه مستندکننده واحد واقعی ستون فایل منبع نیست. مقدار ناموجود با صفر اندازه‌گیری‌شده تفاوت دارد. نتیجه یک مدل با داده فرضی نباید به عنوان اندازه‌گیری عملیاتی عرضه شود.

۱۳. ساختار پروژه و ماندگاری داده

پروژه SQLite، داده‌های شبکه، منشأ لایه‌ها، رخدادهای پردازش‌شده، ماتریس r/، توپولوژی، ظرفیت، تصمیم‌های بازبینی، تنظیمات و خروجی‌های ذخیره‌شده را نگهداری می‌کند. از این نسخه، انتخاب فایل اکسل برای هر فیدر bytes فایل را نیز داخل جدول outage_workbook ذخیره می‌کند. فایل‌های xlsx خود یک ظرف ZIP هستند، بنابراین فشرده‌سازی مضاعف به‌طور معمول اثر چشمگیری ندارد. SHA-256 پیش از استفاده کنترل می‌شود و فایل معیوب بی‌صدا پردازش نمی‌شود. فایل موقت برای پردازش پس از پایان کار حذف می‌شود.

انتخاب فایل و پردازش دو عمل متفاوت‌اند: انتخاب موفق فایل به پروژه متصل است، اما تا Process per-feeder اجرا نشود رکورد، طبقه و ماتریس جدیدی تولید نشده است. پس از باز کردن پروژه، نام فایل‌های ذخیره‌شده کنار فیدر ظاهر می‌شود. دکمه مشاهده سوابق، رخدادهای آخرین تحلیل پردازش‌شده را بدون نیاز به توپولوژی نمایش می‌دهد؛ بازبینی زون به اجرای توپولوژی نیاز دارد.

نسخه مستقلی شامل فایل‌ها و داده‌های ثبت‌شده می‌سازد. روی پروژه جدید کنترل تعداد SQLite backup با Save As فایل‌ها و رخدادها را انجام دهید. پروژه‌های قدیمی که فقط مسیر در حافظه داشته‌اند بدون منبع اکسل قابل بازیابی نیستند؛ فایل اصلی را دوباره انتخاب کنید. کاربر مسئول پشتیبان‌گیری است.

۱۴. ورود GIS، CRS و کنترل کیفیت

ورود شبکه شامل شناسایی لایه‌ها، نگاشت فیلدها، تشخیص مختصات و تبدیل به دستگاه تحلیل پروژه است. داده جغرافیایی با سیستم مختصات ناشناخته یا ناسازگار می‌تواند فاصله، شعاع و اتصال را تغییر دهد. وضعیت CRS و واحد طول را کنترل کنید. شناسه فیدر از نام کامل نرمال‌شده استخراج می‌شود؛ مقایسه نام کوتاه و کامل تنها هنگام تطبیق یکتا امن است. F2 دو پست مستقل نباید به هم ادغام شوند.

کنترل کیفیت خطا، هشدار و اطلاعات را همراه با منبع، شناسه رکورد و اثر بالقوه گزارش می‌کند. هشدار عدم اتصال، مختصات، داده ناقص یا لایه نامطابق را قبل از انتشار نتیجه بررسی کنید. گراف شامل گره‌ها، یال‌ها، کلیدها، بارها، منابع و انتساب ویژگی‌های GIS است. فاصله snap یک فرض هندسی است، نه اثبات اتصال برقی.

۱۵. ردیابی، زون و مسیر انتقال

ردیابی از سر فیدر در وضعیت عادی کلیدها انجام می‌شود. نقاط باز و مرز تجهیز می‌توانند شبکه را به زون تقسیم کنند. نام یا جهت یال به‌تنهایی اتصال قابل اتکا نیست. کاندید Tie اتصال بالقوه است؛ تصمیم مهندس و ظرفیت، وضعیت تجهیز و امکان مانور را باید جداگانه کنترل کرد. داده حلقوی با ساختار درخت پوشا برای مدل شعاعی ساده می‌شود و یال‌های اضافی باید در هشدار گزارش بررسی شوند.

برای هر سناریوی N-1 خطا در یک زون، بار قطع‌شده و قابل انتقال با گراف، کلیدها، Tieهای تأییدشده و ظرفیت مجاز محاسبه می‌شوند. TL = 100 × max(restorable_kw / total_load_kw) / (EP = 100 × max(shed_kw ؛ دو بیشینه ممکن است به زون‌های مختلف متعلق باشند. بار صفر یا توپولوژی ناقص، درصد معتبرtotal_load_kw ایجاد نمی‌کند.

۱۶. ورود فایل خاموشی و شناسایی فیلدها

موتور فایل xlsx یا xlsm را با openpyxl در حالت مقادیر خوانده، سطر خالی را کنار می‌گذارد و ستون‌های متناظر نام فیدر، تاریخ، مدت، علت، قطع‌کننده، انرژی و توضیحات را نگه می‌دارد. در حالت per-feeder تمام سطرهای هر فایل با فیدری که کاربر به آن منتسب کرده برچسب می‌خورند. این انتساب باید خود کاربر کنترل شود. در حالت فایل جمعی، فیدر از داده ورودی و مدل تطبیق می‌یابد. رکورد خارج از مدل در آمار خام ممکن است بماند ولی در ماتریس r/ مدل وارد نمی‌شود.

بهتر است پس از پردازش تعداد رخداد، نام فیدر، تاریخ حداقل/حداکثر، واحد انرژی و تعداد موارد داده ناکافی را با فایل منبع مقایسه کنید. قالب مدت و تاریخ نامعتبر می‌تواند باعث حذف از بعضی محاسبات شود. شناسه، شماره ردیف و

متن خام برای ردیابی ثبت می‌شوند. فقط انتخاب فایل کافی برای محاسبه نیست. بدون مقدار معتبر، نتیجه را صفر واقعی تفسیر نکنید.

۱۷. جزئیات قواعد متنی طبقه‌بندی

اولویت قواعد ثابت است و اولین تطبیق پیروز می‌شود. ابتدا نبود علت، فیدر، تاریخ یا مدت داده ناکافی است. سپس وجود عبارت‌های صریح برنامه‌ای در علت یا توضیحات طبقه برنامه‌ای می‌دهد؛ کد فعلی «قطع اضطراری» را هم در همین گروه قرار می‌دهد و این قرارداد نیازمند تأیید است. علت نامطابق با الگوهای شناخته‌شده به داده ناکافی و هشدار نگاشت‌نشده می‌رود.

بعد، سرقت و موارد شبکه داخلی مشترکان، اندازه‌گیری و فشار ضعیف خارج از دامنه تلقی می‌شوند. نام تجهیزهایی مانند ترانس، مقره، کابل، برقگیر و پایه در علت به نیازمند تعمیر یا تعویض منجر می‌شود. وجود فیوز یا کات‌اوت در علت/قطع‌کننده در حالت معمول به هماهنگی حفاظتی منجر می‌شود؛ اگر هم‌زمان نشانه سرخط، ابتدای فیدر یا کلید اصلی فیدر وجود داشته باشد، داده ناکافی برای بررسی دستی است. قاعده تعمیر قبل از فیوز بررسی می‌شود. سایر علل شناخته‌شده و قطع‌کننده‌های بالادست طبق قاعده نهایی قابل کاهش با کلیدگذاری تلقی می‌شوند.

طبقه‌بند تطبیق زیررشته انجام می‌دهد؛ نفی جمله، زمینه عملیاتی و درست‌بودن علت واقعی را نمی‌فهمد. شناسه‌های مبنای R-SWITCH-01 و R-COORD-01، R-REP-01، R-DOM-01، R-UNK-99، R-PLAN-01، R-NAK-01 حسابرسی‌اند. R-SWITCH-01 اثبات حذف‌پذیری خاموشی توسط کلید نیست. گروه علت گزارش، مستقل از طبقه شش‌گانه است.

۱۸. پوشش زمانی و r/ با مثال عددی

پنجره به طور پیش‌فرض از اولین تا آخرین ماه تاریخ‌دار انتخاب می‌شود. پوشش هر فیدر مستقل محاسبه می‌شود؛ وجود هر رکورد در ماه، آن ماه را دارای رکورد می‌کند، حتی اگر در طبقه نباشد. ماه‌های بدون رکورد در پنجره پیش‌فرض حذف می‌شوند مگر مهندس آنها را صریحا دارای پوشش اعلام کند. ماه مستثنا از مخرج کم می‌شود. طول ماه شمسی کامل محاسبه و بر ۳۶۵ تقسیم می‌شود؛ اولین و آخرین ماه بخشی به صورت ماه کامل لحاظ می‌شوند.

مثال: ۱۲ رخداد تصادفی در ۱۸۶ روز پوشش یعنی T=186/365=0.509589 سال و 12/T=23.5484= رخداد/سال. اگر مدت‌ها جمعا ۷۲۰ دقیقه باشند، r=720/12=60 دقیقه و U=23.5484 ساعت/سال است. اگر همان داده در یک سال واقعی کامل ثبت شده و ماه‌های بدون رخداد واقعا تحت پوشش باشند، نرخ ۱۲ است نه ۲۳٫۵۴۸۴. مهندس باید ماه‌های بدون رخداد را از ماه فاقد داده تفکیک کند.

محدودیت فعلی کد: صورت کسر از تمام رخدادهای فیدر تشکیل می‌شود و لزوما با پنجره صریح و ماه‌های مستثنا فیلتر نمی‌شود، در حالی که مخرج با آن تغییر می‌کند. قبل از استفاده رسمی فایل را برای همان پنجره تصفیه و سازگاری مخرج و صورت را کنترل کنید. نبود پوشش معتبر ممکن است به fallback تعداد سال‌های رخداد یا صفر داخلی منجر شود؛ صفر داده ناکافی به معنی نرخ واقعی صفر نیست.

۱۹. شاخص‌های پیوستگی با مثال عددی

تعاریف: SAIDI = System Average Interruption Duration Index؛ SAIFI = System Average Interruption Frequency Index؛ CAIDI = Customer Average Interruption Duration Index؛ ASAI = Average Service Availability Index. مدل فعلی بدون تعداد مشتریان متأثر از سهم بار به‌عنوان تقریب استفاده می‌کند. برای هر رخداد با مدت مثبت، انرژی مثبت و بار معتبر، f=min(1,E_kWh/(P_kW × duration_hours)). اگر شرط انرژی/بار برقرار نباشد برای نام بریکر، ریکلوزر یا دیژنکتور f=1 فرض می‌شود؛ فیوز یا کات‌اوت جزئی بدون انرژی و رکورد بدون سهم کنار گذاشته می‌شوند. وقوع ریکلوزر الزاما اثبات قطع کل فیدر نیست.

دقیقه؛ CAIDI=SAIDI/SAIFI دقیقه/مشترک/سال؛ SAIDI=(f×duration_min)/T قطع/مشترک/سال؛ SAIFI=f/T وجود دارد اما در محاسبه سهم شاخص خدمت پذیرفته r مدت صفر در میانگین. ASAI=max(0,1-SAIDI/525600)

نمی‌شود. در SAIFI/SAIDI اصلی، رخدادهای با سهم قابل محاسبه فارغ از طبقه می‌توانند وارد شوند؛ نسخه تصادفی جداگانه وجود دارد. سامانه بر اساس بار فیدرها یا میانگین ساده فیدرهای محاسبه‌پذیر جمع می‌شود، نه لزوما وزن واقعی تعداد مشترک.

مثال یک ساله: دو رخداد به ترتیب با سهم‌های ۰٫۲ و ۰٫۵ و مدت‌های ۶۰ و ۱۲۰ دقیقه؛ SAIFI=0.7، SAIDI=0.2×60+0.5×120=72، CAIDI=72/0.7=102.86 دقیقه و ASAI≈99.9863 درصد. این مثال عدد ساختگی است و شاخص فیدر واقعی نیست. وقتی SAIFI صفر باشد کد نسبت CAIDI را صفر می‌نویسد، اما از نظر ریاضی نسبت تعریف‌نشده است.

۲۰. بار، ظرفیت و عدم قطعیت

چهار سطح مدل بار عبارت‌اند از داده واقعی precise، تخمینی estimated، جایگزین substitute و مقیاس‌شده scaled. در scaled با ورودی P و cos=P/S، S=sqrt(P²+Q²)، Q و K=S/sum(Srated)؛ سهم توان ترانس i برابر فیدر می‌رسد. ترانس یا فیدر بدون ورودی لازم در این سطح P است و اگر داده کامل باشد مجموع به K×Srated_i×cos ممکن است صفر داخلی و هشدار بگیرد، نه بار مشاهده‌شده صفر.

ظرفیت نامی پیش‌فرض ۷۰۰۰ kW و ضریب بار پیش‌فرض ۱ فرض نرم‌افزارند؛ این اعداد نیازمند تأیید مهندس هستند و جای ظرفیت ثبت‌شده یا بار اندازه‌گیری‌شده را نمی‌گیرند. حاشیه ظرفیت، ظرفیت اضطراری، ضریب توان و حد انتقال واقعی در تنظیمات کنترل شوند. منشأ بار هر گره را در گزارش مدل بار بررسی کنید.

۲۱. رتبه‌بندی فیدر و ارزش اقتصادی

شش معیار رتبه شامل تعداد TL، EP، Tie، اهمیت/اولویت بار، و r است. برای معیار مستقیم norm_i=x_i/sum(x). در معیار معکوس ابتدا y_i=1/x_i و سپس norm_i=y_i/sum(y). سیاست فعلی برای معکوس صفر مقدار ۱۰ را جایگزین می‌کند. امتیاز فیدر مجموع وزن ضربدر معیار نرمال است؛ وزن پیش‌فرض شش معیار ۱٫۱،۱٫۳،۲٫۲،۱٫۱ است و باید مطابق سیاست مطالعه تأیید شود. نبود r/ می‌تواند در این مسیر به صفر داخلی تبدیل شود و روی رتبه اثر بگذارد.

برای قاعده زونی، U_i=_i×r_i/60 و ENS_i=U_i×P_i با P بر حسب MW، انرژی به MWh/سال می‌دهد. انحراف نرمال برابری U×P برابر ریشه میانگین مربع اختلاف مقادیر از میانگین تقسیم بر میانگین است. این معیار به تنهایی صحت طرح را تضمین نمی‌کند.

در تابع اقتصادی، CE=ضریب ریالی × (E_before-E_after=هزینه کلید جدید و جابه‌جایی، CN)، و CE/CN نسبت مزیت به هزینه است. ضریب پیش‌فرض ۱۰۰۰ ریال/kWh و هزینه‌ها باید با زمان و سیاست شرکت کنترل شوند. هزینه صفر در کد نسبت معتبر معمولی تولید نمی‌کند. شاخص اقتصادی یک مدل مقایسه‌ای است؛ نرخ تنزیل، دوره عمر و هزینه‌های بهره‌برداری باید مطابق دامنه مطالعه به طور مستقل ارزیابی شوند.

۲۲. فرمول EENS و راهبرد جست‌وجو

در درخت شعاعی، مرزهای موجود و کاندیدها قطعاتی با طول L_i و بار P_i می‌سازند. _i=_f×L_i/L_total، بنابراین با طول کل مثبت مجموع نرخ قطعات برابر فیدر است. این فرض یکنواختی خرابی بر واحد طول است و تفاوت کابل، تجهیز، فرسودگی و شرایط جوی را جداگانه مدل نمی‌کند.

مدل uniform برای هر قطعه: EENS_i=_i/60 × [Trep×Pfault+Trest×Prestored+Trep×Punrestored].

بار سالم بازیابی‌شده با ظرفیت پشتیبان محدود است؛ بدون مقدار پشتیبان، در این .(Trep ،زمان بازیابی)Trest=min مدل نامحدود فرض می‌شود. در مدل directional به جای کل بار سالم، پایین‌دست قطعه مبنای بازیابی است و بالادست مطابق پیاده‌سازی آن سناریو محاسبه می‌شود. حاصل جمع قطعه‌ها kWh/سال است. مدل خطای گذرا ریکلوزر به نرخ گذرا، زمان وصل، احتمال موفقیت و توپولوژی ریکلوزر وابسته و جداگانه افزوده می‌شود.

کاندیدها با قیود عمق، نسبت بار پایین‌دست، وضعیت تجهیز موجود و سقف فهرست فیلتر می‌شوند. جست‌وجوی حریصانه در هر گام کاندیدی را که کمترین EENS می‌دهد انتخاب می‌کند و در نبود بهبود یا سقف تعداد متوقف می‌شود؛

بهینه کلی همه ترکیبات را تضمین نمی‌کند. EENS پایه با کلیدهای موجود مدل می‌شود. درصد بهبود=100×(base-best)/base برای پایه مثبت. یکسان بودن نقطه خروجی uniform و directional از نظر ریاضی ممکن است؛ مقایسه همه تنظیمات، بار، r/، کاندیدها و گام‌های حل لازم است.

۲۳. مکان خاموشی، UTM و نقشه

بانک مکان نام، alias، مختصات و source دارد. امتیاز تطبیق متنی فقط پیشنهاد است؛ نام‌های مشابه، مخفف، املای متفاوت و چند فیدر می‌توانند ابهام ایجاد کنند. مرکز محدوده چندضلعی جای دقیق تجهیز نیست. تصمیم انتساب زون باید به تأیید مهندس برسد. تبدیل مختصات ورودی UTM به CRS پروژه به زون و سیستم مختصات صحیح وابسته است؛ طول و شعاع در CRS نامناسب می‌تواند خطا ایجاد کند.

لایه توزیع رنگی تعداد نقاط در شعاع هر یال را محاسبه می‌کند؛ یک رخداد می‌تواند چند یال را تحت تأثیر قرار دهد. این لایه مدل مکان دقیق خرابی یا نرخ خرابی مستقل هر یال نیست. فیلتر فیدر نمایش را محدود می‌کند و فیدر نامشخص به طور خودکار به فیدر انتخاب‌شده تخصیص نمی‌یابد. خاموشی نماد، تغییر رنگ و پنهان‌کردن کلید صرفا نمایشی‌اند و توپولوژی یا نتایج را تغییر نمی‌دهند.

۲۴. خروجی، آزمون و پذیرش مهندسی

خروجی Excel شاخص‌ها و گزارش‌های Word/PDF از مدل نتایج ساخته می‌شوند؛ بررسی شماره نسخه، تاریخ، نام پروژه، فایل‌ها، شناسه اجرا، واحد، هشدارها، بار، پنجره و تصمیم‌ها ضروری است. ردیف دارای داده ناکافی را صفر نخوانید. برای نمونه‌برداری مستقل، یک فیدر و چند رخداد آن را دستی با r ، و سهم قطع بازحساب و با جدول خروجی مقایسه کنید. نقشه را با داده اصلی و محل واقعی تجهیزات مقایسه کنید.

قبل از تحویل رسمی، واحد انرژی، تعداد مشترکان متأثر، منشأ بار، وضعیت حفاظتی و عملیاتی Tieها، زمان تعمیر/مانور، کیفیت GIS، نرخ خرابی و مقیاس زمانی باید توسط متولی داده تأیید شوند. آزمون نرم‌افزاری اثبات صحت فیزیکی داده و قابلیت اجرای مانور نیست.

۲۵. راهنمای توسعه و دامنه مرجع کد

معماری لایه‌ای شامل domain (مدل و شناسه)، gis (ورود و topology ،(CRS (گراف، ردیابی، زون و reliability ،(Tie (خاموشی، طبقه‌بندی، پوشش، بار، شاخص و level1 ،(EENS و placement (سناریو، رتبه و طرح)، application (هماهنگی سرویس‌ها)، reporting ،persistence (SQLite) (گزارش)، ui (نقشه و Qt) و cli است. متن زیر به طور خودکار از کد تولید می‌شود: مسیر هر ماژول، نام کلاس/تابع، امضا، خط منبع و خلاصه docstring. برای جزئیات تمام شاخه‌های اجرایی و نسخه دقیق رفتار، فایل منبع و آزمون متناظر ملاک‌اند؛ این نمایه جانشین کد نیست.

۲۶. روابط کلیدی در قالب چپ به راست

رابطه‌های زیر برای جلوگیری از ابهام چیدمان دوطرفه به صورت مستقل ارائه شده‌اند. واحد هر ورودی در فصل‌های قبل تعریف شده است.

```
T_year = exposure_days / 365
lambda_total = N_random / T_year
lambda_reducible = N_reducible / T_year
r_min = sum(duration_min for valid random events) / N_valid
U_h_per_year = lambda_total * r_min / 60
f_i = min(1, E_i_kWh / (P_f_kW * duration_i_hours))
SAIFI = sum(f_i) / T_year
SAIDI = sum(f_i * duration_i_minutes) / T_year
CAIDI = SAIDI / SAIFI
ASAI = max(0, 1 - SAIDI / 525600)
S_load = sqrt(P_load**2 + Q_load**2)
```

cos_phi = P_load / S_load

K = S_load / sum(S_rated_i)

P_transformer_i = K * S_rated_i * cos_phi

EP = 100 * max(zone_shed_kW) / feeder_total_kW

TL = 100 * max(zone_restorable_kW) / feeder_total_kW

normalized_i = transformed_i / sum(transformed_i)

score_i = sum(weight_j * normalized_i_j)

lambda_section_i = lambda_feeder * length_i / total_length

EENS_i = lambda_i / 60 * (P_fault * T_repair + P_restored * T_restore + P_unrestored * T_repair)

EENS_total = sum(EENS_i) + optional_transient_EENS

improvement_percent = 100 * (EENS_before - EENS_after) / EENS_before

CE = rial_per_kWh * (energy_before_kWh - energy_after_kWh)

benefit_cost_ratio = CE / switch_cost_rials

پیوست: نمایه ماژول‌ها، کلاس‌ها و توابع کد

این پیوست از درخت کد همین نسخه با AST تولید شده است. خط منبع، امضای تابع و مستندات موجود در کد را نشان می‌دهد. نبود docstring به معنی نبود رفتار نیست. برای منطق کامل، بدنه فایل و آزمون مربوط ملاک است.

src/switchloc/__init__.py

SwitchLoc — مطالعات جایابی کلید و برنامه‌ریزی اتوماسیون شبکه توزیع برق. Level-1 switch placement & automation planning for MV distribution networks, per Tavanir directive TAV122-02/00. Phase-5 scope (this drop — «گزارش‌ها و خروجی GIS»): - reporting : StudyModel + neutral content tree (single source of truth), RTL Excel workbook (openpyxl), Persian RTL Word report (python-docx, w:bidi/w:bidiVisual), offline Persian PDF (ReportLab + arabic_reshaper + python-bidi + bundled Vazirmatn, optional extras), GeoJSON×3 + CSV crew sheet in WGS84 (RFC 7946) - service : ReportService — in-memory re-run + cross-check vs the persisted placement run (mismatch

src/switchloc/__main__.py

src/switchloc/application/__init__.py

Application services (use-case orchestration layer).

src/switchloc/application/import_service.py

ImportService — end-to-end import orchestration (Phase 1). Flow (docs/ARCHITECTURE.md §۲): hash inputs discover (multi-format) suggest mapping *explicit confirmation* read CRS resolve/transform canonical assembly QC persist SQLite re-hash inputs (GG-02 guarantee). The confirmation step is deliberate: per main prompt §۵-۳ no automatic mapping is ever final. The caller supplies `confirm_fn`; without it the service stops before reading and returns the suggestions (the UI wizard of Phase 2 will plug in here — same contract).

sha256_of(path: Path) [line 34]
class ImportSettings [line 43]
class ImportResult [line 53]
ImportResult.accepted(self) [line 65]
class ImportService [line 69]
ImportService.__init__(self, settings: ImportSettings | None=None) [line 70]
ImportService.run(self, source, confirm_fn=None) [line 74]

src/switchloc/application/outage_review.py

Human-in-the-loop outage place/zone review with auditable SQLite storage. (درخواست کاربر نیاز این نسخه (سه
---------- ۱. **انتخاب خودکار زون هر ردیف** بر اساس فیدر رخداد و مختصات مکان پیشنهادی: مرز و هندسه هر زون از همان گراف/زون‌بندی توپولوژی می‌آید (بدون تولید هندسه حدسی). اگر مختصات در محدوده تحمل نبود، قاعده «تک‌زون‌بودن فیدر» و سپس «نامشخص» اعمال می‌شود — هیچ زونی حدس زده نمی‌شود و مبنای انتخاب در ستون جدا ثبت می‌گردد (بند ۱۹). ۲. **مکان پیشنهادی در ابتدای فهرست** و انتخاب‌شده به‌صورت پیش‌فرض، هرگاه بانک آفلاین مکان برای «نام پست» تطبیقی پیدا کرده باشد. ۳. **وضعیت پیش‌فرض «تأیید مهندسی»** برای همه ردیف‌ها، به‌ج

default_gazetteer_path() [line 50]

مسیر بانک آفلاین مکان‌های اردبیل را بدون ایجاد هیچ فایلی پیدا می‌کند. ترتیب (مسیرها محدود و از پیش‌تعیین‌شده‌اند — هیچ گردش نامحدودی تا ریشه فایل‌سیستم انجام نمی‌شود و هیچ فایلی ساخته/تغییر داده نمی‌شود): 1. ``SWITCHLOC_GAZETTEER$`` — اگر تنظیم شده باشد، مسیر همان است؛ اگر نبود، ``None`` برگردانده می‌شود (مسیر غلط سکوت نمی‌شود). 2. کنار فایل اجرایی: ``<dir(exe)>/data/geography/ardabil_places.sqlite`` (چیدمان نسخه exe که BUILD.bat کنار خروجی کپی می‌کند). 3. داخل بسته PyInstaller: ``sys._MEIPAS

class OutagePlaceSuggestion [line 86]
suggest_event_places(records: Iterable, gazetteer: PlaceGazetteer, limit: int=5) [line 92]

Suggest places from free-text descriptions; suggestions are never assignments.

class ZoneGeometry [line 126]

هندسه سبک یک زون: نقاط گره‌های آن در CRS پروژه.

ZoneGeometry.label(self) [line 135]
ZoneGeometry.centroid(self) [line 139]
ZoneGeometry.bbox(self) [line 146]
ZoneGeometry.distance_to(self, point: tuple[float, float]) [line 153]

کمترین فاصله اقلیدسی نقطه تا گره‌های زون (متر).

ZoneGeometry.as_dict(self) [line 163]
build_zone_geometries(graph, zones) [line 169]

.(گراف هندسه هر زون (نقاط گره‌های موجود + ``ZoneResult``

class ZonePick [line 194]

نتیجه انتساب خودکار: زون + مبنای انتخاب (قابل حسابرسی).

ZonePick.basis_fa(self) [line 204]
ZonePick.as_dict(self) [line 207]
_bbox_lower_bound(bbox, point) [line 217]

کمترین فاصله ممکن نقطه تا هر جایی داخل bbox (۰ = داخل).

_feeder_matches(label: str, feeder: str | None) [line 228]

همان فیدر؟ (نگارش کوتاه «F5» با «F5 - KOSAR» یکی است)

nearest_zone(geometries: dict[str, ZoneGeometry], point, feeder: str | None=None, feeder_only: bool=True) [line 236]

نزدیک‌ترین زون به یک نقطه (با هرس bbox برای کارایی).

pick_zone(geometries: dict[str, ZoneGeometry], *, feeder: str='', point: tuple[float, float] | None=None, max_distance_m: float=DEFAULT_MAX_ZONE_DISTANCE_M, feeder_only: bool=True, reason: str='') [line 259]

زون متناظر یک رخداد را از فیدر و مختصات تعیین می‌کند. قواعد (به ترتیب، هرگز حدس مطلق): 1. اگر مختصات موجود باشد و نزدیک‌ترین هندسه زون همان فیدر در همان زون، مبنا «مختصات پیشنهادی». 2. اگر فیدر فقط **یک** زون داشته باشد همان زون (انتساب قطعی)، مبنا «تک‌زون فیدر». 3 ``max_distance_m`` در غیر این صورت هیچ زونی انتخاب نمی‌شود و نزدیک‌ترین فاصله برای راهنمای مهندس گزارش می‌گردد.

place_options(suggestion: OutagePlaceSuggestion | None, saved_place_id: str='') [line 308]

گزینه‌های «مکان پیشنهادی» — بهترین پیشنهاد **اول** فهرست. هرگاه بانک آفلاین مکان تطبیقی پیدا کرده باشد، بهترین تطبیق اولین گزینه و انتخاب پیش‌فرض است (درخواست کاربر)؛ «بدون مکان تأییدشده» آخرین گزینه می‌ماند تا اشتباهی انتخاب نشود.

zone_options(geometries: dict[str, ZoneGeometry], feeder: str, point=None, feeder_only: bool=True) [line 345]

گزینه‌های زون همان فیدر، مرتب بر اساس فاصله تا نقطه (نزدیک‌تر اول).

class ReviewRowPlan [line 364]

برنامه یک ردیف بازبینی — ورودی دیالوگ و خروجی آزمون‌ها.

ReviewRowPlan.zone_basis_fa(self) [line 381]
ReviewRowPlan.needs_review(self) [line 385]
ReviewRowPlan.as_dict(self) [line 388]
plan_review_rows(records: Iterable, *, suggestions: dict | None=None, saved: dict | None=None, geometries: dict[str, ZoneGeometry] | None=None, project_crs: str='', max_zone_distance_m: float=DEFAULT_MAX_ZONE_DISTANCE_M, feeder_only: bool=True, transform=None) [line 398]

برنامه کامل بازبینی: مکان پیشنهادی، زون خودکار و وضعیت پیش‌فرض. ``transform``: تابع ``(lat, lon) -> (x, y)`` در CRS پروژه؛ پیش‌فرض ``gis.crs.transform_wgs84_to_project``. اگر CRS پروژه نامشخص باشد هیچ مختصاتی حدس زده نمی‌شود و مبنا «پروژه نامشخص» ثبت می‌گردد.

outage_map_points(records: Iterable, *, geometries: dict | None=None, saved: dict | None=None, suggestions: dict | None=None, transform=None, max_distance_m: float=2500.0) [line 516]

رخدادهای دارای مختصات را به نقاط آماده نمایش روی نقشه تبدیل می‌کند. منبع مختصات، به ترتیب: مختصات ذخیره‌شده بازبینی (CRS پروژه) مکان پیشنهادی بانک آفلاین تبدیل‌شده با ``transform``. رخدادی که هیچ مختصاتی ندارد وارد لایه نمی‌شود (شمارش آن در پنل نقشه گزارش می‌گردد). زون هر نقطه از بازبینی ذخیره‌شده یا انتساب خودکار ``pick_zone`` می‌آید و مرکز آن زون برای رسم خط اتصال برگردانده می‌شود. ``label`` / ``place`` نام مکان است (بازبینی ذخیره‌شده > بانک مکان‌ها > نام پست > شماره رخداد) تا برچسب نقشه بت

_clean_place_name(value) [line 602]

نام مکان قابل نمایش روی نقشه. برچسب‌های بی‌معنی فایل خاموشی (مثل «Sheet1» که نام برگه اکسل است) یا خطتیره‌ها حذف می‌شوند تا برچسب نقشه نشود «Sheet1 (4)».

_cluster_points(points: list, distance_m: float) [line 617]

خوشه‌بندی تک‌پیوندی نقاط (نقاط نزدیک‌تر از آستانه یک برچسب می‌گیرند).

outage_label_groups(points: Iterable[dict], *, cluster_distance_m: float=2000.0) [line 634]

نقاط لایه خاموشی برچسب‌های «نام (تعداد)». چند رخداد در یک مکان یک برچسب می‌گیرند و تعداد خطاها داخل پرانتز می‌آید (درخواست کاربر: «کرندق (۴)»، «نوده (۶)») — به‌جای چند برچسب تکراری روی هم. اگر همان نام در نقاط دور از هم (بیش از ``cluster_distance_m``) ثبت شده باشد به برچسب‌های جدا شکسته می‌شود تا شمارش گمراه‌کننده نشود. مختصات هر برچسب = میانگین نقاط همان خوشه.

_attr_or_dict(rec, name: str) [line 668]

رخداد ممکن است دیتاکلاس یا dict باشد (هر دو پشتیبانی می‌شوند).

_feeder_of(rec, candidates=()) [line 675]

شناسه فیدر رخداد؛ در صورت امکان با نگاشت به شناسه فیدر مدل. ``candidates`` (فیدرهای زون‌ها/شبکه) برای یکی‌دانستن نگارش کوتاه و آراسته است: «F5» فایل خاموشی «F5 - KOSAR». در ابهام (دو فیدر F2) هیچ نگاشتی انجام نمی‌شود.

class OutageReviewStore [line 697]

Persist engineer-confirmed eventplace/zone decisions in project DB.

OutageReviewStore.__init__(self, db_path: str | Path) [line 712]
OutageReviewStore.save(self, event_id: str, zone_id: str, *, place_id: str='', place_name: str='', reviewer: str='', note: str='', status: str='confirmed', coord_x: float | None=None, coord_y: float | None=None, zone_basis: str='') [line 737]
OutageReviewStore.load(self) [line 765]
OutageReviewStore.confirmed_event_to_zone(self) [line 771]
_point_segment_distance(px: float, py: float, ax: float, ay: float, bx: float, by: float) [line 778]

کمترین فاصله نقطه تا پاره‌خط (واحد: متر).

outage_heat_counts(points: Iterable[dict], graph, *, radius_m: float=300.0, edge_ids=None) [line 790]

تعداد خاموشی‌های نزدیک هر یال (متناسب با تکرار در طول فیدر). هر نقطه خاموشی به **همه** یال‌هایی که در شعاع ``radius_m`` آن هستند نسبت داده می‌شود (یک خاموشی می‌تواند روی چند یال مجاور اثر بگذارد); خروجی: ``{edge_id: count}``. یال‌های بدون مختصات نادیده گرفته می‌شوند.

assign_heat_level(count: int, levels: Iterable[dict]) [line 831]

سطح رنگی متناظر با تعداد خاموشی (None = بدون رنگ).

outage_place_ranking(points: Iterable[dict], *, limit: int | None=None) [line 853]

رتبه‌بندی مکان‌های خاموشی (درخواست کاربر: جدول رتبه‌بندی). هر ردیف: رتبه، مکان، فیدر، تعداد خاموشی، سهم درصدی، توزیع وضعیت (تأیید/در انتظار/رد)، مرکز مختصات و شماره زون غالب.

src/switchloc/application/outage_service.py

OutageService — orchestration of the Phase-3 engine (ARCHITECTURE §۵). Pipeline (گام ۲ / بند ۷-۷-۳ + شکل ۵ + بند ۱۰ پرامپت مادر): workbook OutageRecord list (A-08 cleaning) six-class classifier unmapped-cause Errors /r matrix (model feeders) four official review-level tables (شکل ۵) QC verdict persistence + Persian report. The service never invents data: no planned/unplanned guess (OUT-POL-02), no category guess for loads, no feeder guess for out-of-model events.

class OutageSettings [line 29]

class OutageResult [line 65]

OutageResult.verdict(self) [line 79]

filter_records_by_scope(records: list, scope_feeder: str | None=None, scope_feeders=()) [line 83]

Keep records in the requested feeder set; return excluded count.

class OutageService [line 98]

OutageService.__init__(self, settings: OutageSettings | None=None) [line 99]

OutageService.run(self, workbook: str | None=None, out_db: str | None=None, report_dir: str | None=None, project_name: str='تحلیل خاموشی\u200cها') [line 103]

OutageService._settings_snapshot(st: OutageSettings) [line 308]

OutageService.write_report(self, res: OutageResult, report_dir: str) [line 327]

src/switchloc/application/placement_service.py

PlacementService — end-to-end orchestration of Phase 4 (گام‌های ۴-۸). Loads the project network from SQLite, re-runs the topology in-memory (persisted for traceability), takes the /r matrix (from the optional outage workbook, else the latest persisted outage run), executes the PlacementEngine and persists the tables + writes the MD/JSON report. Nothing is recomputed twice: the engine is pure; persistence/reporting live here (application layer).

class PlacementAppSettings [line 29]

class PlacementRunResult [line 46]

PlacementRunResult.verdict(self) [line 57]

_lambda_from_workbook(path: str) [line 66]

_lambda_from_store(store: ProjectStore) [line 77]

class PlacementService [line 86]

PlacementService.__init__(self, settings: PlacementAppSettings | None=None) [line 87]

PlacementService.run(self, db_path: str) [line 91]

PlacementService._fmt_switches(row: dict) [line 161]

PlacementService.write_report(res: PlacementResult, report_dir: str, run_id: str | None=None) [line 166]

Persian MD + JSON report (same convention as phases 1-3).

src/switchloc/application/sectionalizer_service.py

نام قدیمی سرویس — به :mod:`switchloc.application.switch_placement_service` منتقل شد.

src/switchloc/application/study_preflight.py

study_preflight.py — پنجره ری‌چک کامل EENS قبل از اجرای جایابی بهینه» :(v1.15) درخواست کاربر .EENS بازبینی کامل ورودی‌ها **قبل از** اجرای جایابی
ورودی‌ها باز شود؛ در حالت فعلی با داده بار پیش‌فرض کار می‌کند.» این ماژول Qt-free است: از ورودی‌های مؤثر همان اجرا یک «چک‌لیست» می‌سازد و برای هر ردیف یکی از سه وضعیت را می‌دهد: * ``ok`` — مقدار صریح و کامل است؛ * ``warn`` — مقدار پیش‌فرض/تخمینی است؛ اجرا مجاز است ولی نتیجه باید با همین برچسب خوانده شود؛ * ``error`` — ورودی ناقص/ناسازگار است؛ اجرا نباید شروع شود. هیچ عددی در این ماژول ساخته نمی‌شود؛ فقط وضعیت ورودی‌های واقعی گزارش می‌شود تا کاربر بداند با چه داده‌ای اجرا می‌کند.

class PreflightRow [line 40]

class PreflightReport [line 49]

PreflightReport.errors(self) [line 55]

PreflightReport.warnings(self) [line 59]

PreflightReport.blocking(self) [line 63]

آیا اجرا باید متوقف شود؟ (فقط خطاها مانع‌اند)

PreflightReport.counts(self) [line 68]

PreflightReport.summary_fa(self) [line 72]

_fmt(v, nd=2) [line 78]

build_preflight(*, method: str, switch_type: str, engine: dict | None=None, load_model: dict | None=None, load_total_kw: float | None=None, feeders: list | None=None, all_feeders: list | None=None, outage_rates: dict | None=None, repair_times: dict | None=None, explicit_rates: dict | None=None,

backup_kw: dict | None=None, s_rated_kva: dict | None=None, outage_available: bool=True) [line 85]

چک‌لیست ورودی‌های اجرای جایابی EENS (بدون Qt).

preflight_from_settings(*, settings: dict, load_model: dict | None, load_total_kw: float | None, feeders: list | None, all_feeders: list | None, persisted_outages: dict | None=None, backup_kw: dict | None=None, s_rated_kva: dict | None=None) [line 278]

پل میان تنظیمات تب و :func:`build_preflight` (شکل داده یکسان).

src/switchloc/application/switch_placement_service.py

به همین ترتیب در رابط کاربری): 1) EENS اجرای موتور «روش جایابی بهینه کلید» روی یک پروژه. سه روش محاسبه — SwitchPlacementService. واقعی خاموشی** + مدل** r و — ``uniform`` .واقعی خاموشی** + مدل شعاعی/جهت‌دار (فقط بار پایین‌دست قطع می‌شود). 2** r و — ``directional`` یکنواخت (هم‌ارز کد مرجع). 3. ``manual`` — ورودی‌های دستی (r ،، زمان تعمیر/بازیابی و پارامترهای موتور). در دو روش اول و r از ماتریس r/ پروژه به‌صورت خودکار خوانده می‌شوند و ورودی‌های دستی نادیده گرفته می‌شوند (با هشدار صریح). این سرویس همان الگوی بقیه سرویس‌های SwitchLoc را دارد ورودی‌ها را از پایگاه پروژه می‌خواند، موتور را با منشأ صری :(Import/Topology/ Outage/Placement)

class SwitchPlacementAppSettings [line 80]

تنظیمات سطح کاربر (تب «روش جایابی بهینه کلید و کاهش EENS»).

class SwitchPlacementAppResult [line 107]
SwitchPlacementAppResult.markdown(self) [line 117]
SwitchPlacementAppResult.as_dict(self) [line 120]
class SwitchPlacementService [line 132]

ورودی پروژه موتور نتیجه + گزارش + ذخیره‌سازی.

SwitchPlacementService.__init__(self, settings: SwitchPlacementAppSettings | None=None) [line 135]
SwitchPlacementService.run(self, db_path: str, *, net=None, topo=None, outage_lambda: dict | None=None) [line 142]
SwitchPlacementService._settings_snapshot(self) [line 193]
SwitchPlacementService._build_study(self, net, topo, store: ProjectStore | None, outage_lambda: dict | None, warnings: list) [line 212]
SwitchPlacementService._feeder_keys(net, topo) [line 427]
SwitchPlacementService.write_report(self, res: SectionalizerAppResult, report_dir: str) [line 432]
_network_summary(net) [line 444]
_branch_labels(net) [line 451]

برچسب branch_id GIS (اگر منشأ رکورد موجود باشد).

load_persisted_study(db_path: str) [line 464]

نتیجه ذخیره‌شده در ``project_kv`` را برمی‌گرداند (یا None).

clear_persisted_study(db_path: str) [line 473]

Remove only the saved EENS study; leave inputs and other runs intact.

src/switchloc/application/topology_service.py

TopologyService — orchestration of the Phase-2 engine (ARCHITECTURE §۵). Pipeline: CanonicalNetwork (in-memory or loaded from SQLite) NetworkGraph normal-state tracing tie/NO candidates (A-09) زون‌بندی base scenario + connectivity QC issues persistence + Persian report. The service never mutates the canonical model; the only auditable mutations are the engineer's tie confirmations (switch.normally_state).

class TopologySettings [line 33]
class TopologyResult [line 47]
TopologyResult.verdict(self) [line 61]
class TopologyService [line 65]
TopologyService.__init__(self, settings: TopologySettings | None=None) [line 66]
TopologyService.run(self, net: CanonicalNetwork, issues: IssueLog | None=None, source_run_id: str='', out_db: str | None=None, zone_of_edge_hint: dict | None=None) [line 70]
TopologyService._settings_snapshot(st: TopologySettings) [line 164]

src/switchloc/cli.py

CLI — python -m switchloc (Phase 1: import / inspect). The PySide6 wizard (Phase 2) reuses exactly these services; the CLI is the UI-agnostic contract.

_cmd_import(args) [line 20]
_cmd_inspect(args) [line 48]
_cmd_topology(args) [line 67]

Phase 2 — graph, feeder tracing, tie candidates and zones.

_cmd_report(args) [line 121]

Phase 5 — Excel / Word / PDF / GIS exports of a completed study.

_cmd_placement(args) [line 143]

Phase 4 — n-1, feeder ranking (formula 1 / table 16), schemes (table 17), CE/CN comparison (table 18).

_cmd_sectionalizer(args) [line 195]

روش جایابی بهینه کلید و کاهش EENS (ریکلوزر/سکشنالایزر/سکسیونر).

_cmd_outages(args) [line 266]

Phase 3 — outage ingestion/classification + /r matrix (step 2 / fig. 5).

_cmd_places(args) [line 285]

پرس‌وجو/آمار بانک آفلاین مکان‌ها (غنی‌سازی نسخه ۱.۱۳).

_cmd_outage_ranking(args) [line 317]

جدول رتبه‌بندی مکان‌های خاموشی (درخواست کاربر — بند ۷). رخدادهای دارای مختصات (از بازبینی ذخیره‌شده در همان دیتابیس پروژه) را بر اساس تعداد تکرار خاموشی گروه‌بندی و رتبه‌بندی می‌کند. هیچ محاسبه شبکه‌ای انجام نمی‌شود؛ فقط داده موجود خوانده و مرتب می‌شود.

main(argv=None) [line 411]

src/switchloc/domain/__init__.py

Canonical domain model of SwitchLoc (docs/DATA_MODEL.md).

src/switchloc/domain/entities.py

Canonical domain entities (Phase 1 — docs/DATA_MODEL.md §۱ و §۲). Design rules (docs/DATA_MODEL.md «قواعد داده»): * every entity keeps its SourceRecord chain — traceable to (file, layer, record); * `coord_original` is preserved forever; `coord_snapped` is derived; * nothing is deleted without a recorded issue (`excluded_with_issue`); * missing value ≠ zero — missing stays None and surfaces as an issue; * quantitative fields carry their unit in the attribute name.

class SourceRecord [line 26]

Traceability anchor: every entity points at one of these.

SourceRecord.as_row(self, entity_id: str) [line 34]

class LayerMapping [line 40]

One confirmed slotlayer binding (never automatic-final, بند ۵-۳).

class CRSDefinition [line 52]

Per-layer CRS bookkeeping — prevents double transforms (D-01).

class StudyRunMeta [line 63]

Snapshot of one study execution (reproducibility, بند ۱۹).

class Feeder [line 78]

Feeder.__post_init__(self) [line 87]

class Node [line 93]

Node.__post_init__(self) [line 103]

class LineSegment [line 111]

One 2-point piece of a split polyline (GG-03).

class Branch [line 121]

Branch.__post_init__(self) [line 136]

class SwitchDev [line 146]

SwitchDev.__post_init__(self) [line 160]

class Transformer [line 168]

Transformer.__post_init__(self) [line 178]

class LoadPt [line 184]

نقاط بار = نقاط ترانسفورماتورهای توزیع (بند ۷-۶ متن رسمی).

LoadPt.__post_init__(self) [line 196]

class Substation [line 203]

Substation.__post_init__(self) [line 208]

class TieConnection [line 214]

اتصال پشتیبان فیدرها — ثبت فقط با کلید اتوماسیون (جدول ۱۳ متن رسمی).

TieConnection.__post_init__(self) [line 223]

class CanonicalNetwork [line 232]

In-memory canonical model produced by the assembler (Phase 1).

CanonicalNetwork.summary(self) [line 248]

src/switchloc/domain/enums.py

Shared constants and enumerations (TAV122-02/00 definitions, بند ۵). Every value here is traceable to the directive or the audited field catalog — nothing is invented.

class NodeKind [line 47]

class BranchKind [line 53]

class SwitchFamily [line 60]

class SwitchState [line 69]

وضعیت سرویس از فیلد T__outserv (تعریف ۵-۷ نقطه کار شبکه).

class YesNoUnknown [line 76]

src/switchloc/domain/ids.py

Deterministic, reproducible entity ids (main prompt §19 reproducibility). All internal ids are UUID-5 derived from stable, human-auditable parts (layer, record id, slot, coordinates rounded) so that two identical study runs produce byte-identical databases.

stable_id(*parts: object) [line 16]

UUID-5 string from the given parts (None '').

src/switchloc/geography/__init__.py

Geographic gazetteer and outage-place matching for Iranian provinces.

src/switchloc/geography/places.py

Offline gazetteer for Persian place names. The database is deliberately source-attributed. Import GeoJSON point/polygon features or CSV records; no geocoder/network call is made at analysis time.

normalize_place_name(value: str | None) [line 30]

_haversine_m(lat1: float, lon1: float, lat2: float, lon2: float) [line 49]

فاصله دو نقطه روی کره (متر) — برای نزدیک‌ترین مکان.

compact_place_name(value: str | None) [line 59]

کلید «بدون فاصله» برای مقایسه نام‌ها. «فیروزآباد»، «فیروز آباد» و «فیروزاباد» یک مکان‌اند ولی در متن خاموشی‌ها به هر سه شکل نوشته می‌شوند؛ مقایسه فشرده این تفاوت را از بین می‌برد.

_compact_of(normalized: str) [line 69]

class Place [line 81]

class PlaceMatch [line 95]

class PlaceGazetteer [line 102]

SQLite-backed offline place index; names and aliases are searchable.

PlaceGazetteer.__init__(self, path: str | Path) [line 104]

PlaceGazetteer._migrate(self, db) [line 130]

افزودن ستون‌های نسخه ۱.۱۳ به بانک‌های قدیمی (بدون حذف داده).

PlaceGazetteer._connect(self) [line 173]

PlaceGazetteer.count(self) [line 179]

PlaceGazetteer.upsert(self, places: Iterable[Place]) [line 183]

PlaceGazetteer.import_csv(self, path: str | Path, source: str='user-import') [line 211]

CSV columns: id,name,lat,lon[,type,county,district,aliases].

PlaceGazetteer.import_geojson(self, path: str | Path, source: str='OpenStreetMap', boundary_geojson: str | Path | None=None) [line 225]

Import named Point features; optionally clip by an admin boundary GeoJSON.

PlaceGazetteer.search(self, text: str, limit: int=8) [line 275]

جست‌وجوی نام مکان: دقیق، «فشرده» (فاصله‌ناپذیر) و واژه‌ای. قاعده‌ها (برای بازبینی عملیاتی امن): * تطبیق **دقیق** نام یا نام جایگزین اطمینان «زیاد». * تطبیق **فشرده** (با/بی‌فاصله و آ/ا) همان مکان، «زیاد». * تطبیق **واژه‌ای**: نام باید به‌صورت واژه کامل (با فاصله در دو طرف) در متن بیاید؛ زیررشته یک واژه دیگر پذیرفته نمی‌شود (باگ قبلی: «سالن ورزشی زرج‌آباد» به مکان «رز» تطبیق می‌شد). * تطبیق فشرده‌ی جزئی فقط برای نام‌های ≤۴ حرف و با اطمینان «متوسط».

PlaceGazetteer.add_aliases(self, place_id: str, aliases: Iterable[str]) [line 352]

افزودن نام‌های جایگزین به یک مکان (بدون حذف نام‌های موجود).

PlaceGazetteer.find_near(self, lat: float, lon: float, max_distance_m: float=2000.0, name: str | None=None) [line 372]

مکان‌های نزدیک یک نقطه (مرتب بر اساس فاصله) — برای یکسان‌سازی.

PlaceGazetteer.nearby_county(self, lat: float, lon: float, max_distance_m: float=40000.0) [line 389]

نام شهرستان از نزدیک‌ترین مکان دارای شهرستان (برای داده‌های تازه).

PlaceGazetteer.set_metadata(self, values: dict) [line 401]

PlaceGazetteer.metadata(self) [line 408]

PlaceGazetteer.stats(self) [line 413]

خلاصه بانک برای گزارش/رابط (تعداد بر نوع، منبع و شهرستان).

src/switchloc/gis/__init__.py

gis — GIS ingestion layer (readers, CRS, snap, assembly, mapping).

src/switchloc/gis/assemble.py

CanonicalAssembler — Feature stream CanonicalNetwork (Phase 1). Pipeline (docs/ARCHITECTURE.md §۲, decisions 1 مصوب فاز صفر): CRS resolve/analysis-selection (metric, D-01) + one-time transforms; 2. single-linkage metric snap of ALL line endpoints across layers (replaces GeoGrid decimal rounding — GG-06); 3. polyline segmentation with registered-length distribution (GG-03); 4. device splicing onto snapped nodes — devices on a mid-vertex of a line connect because that vertex became a segment endpoint (GG-05); ambiguous devices (2+ candidates in tolerance) are flagged, never guessed; orphans get their own node + distance-annotated issue; 5. e

normalize_feeder(name: str) [line 66]

Normalize a feeder label (yeh/kaf unification, spacing, case).

_to_float(v) [line 73]
_state_enum(s: str) [line 82]
_yesno_enum(s: str) [line 87]
class AssembleContext [line 93]
class CanonicalAssembler [line 98]

Builds the Phase-1 canonical model from confirmed mappings.

CanonicalAssembler.__init__(self, issues: IssueLog, crs: CrsService | None=None, ctx: AssembleContext | None=None) [line 101]
CanonicalAssembler.assemble(self, layers: list[LayerInfo], feats_by_layer: dict[str, list[Feature]], mapping: LayerMappingService) [line 109]
CanonicalAssembler._ep_index(li_name, fid, vi, ep_owner) [line 354]
CanonicalAssembler._source(self, layer: str, f: Feature) [line 357]
CanonicalAssembler._role(self, mapping: LayerMappingService, layer: str, role: str) [line 366]
CanonicalAssembler._device_node(self, net: CanonicalNetwork, kind: NodeKind, coord: tuple | None, feeder: str) [line 372]
CanonicalAssembler._make_device_entity(self, net: CanonicalNetwork, li_name: str, slot: str, f: Feature, node_id: str, mapping: LayerMappingService, src: SourceRecord) [line 379]
CanonicalAssembler._dup_ids(self, feats_by_layer: dict[str, list[Feature]]) [line 429]
CanonicalAssembler._dup_lines(self, net: CanonicalNetwork) [line 448]
CanonicalAssembler._collect_feeders(self, net: CanonicalNetwork, feats_by_layer) [line 468]

src/switchloc/gis/crs.py

CRS management (main prompt §۵-۴ / D-01 of ARCHITECTURE.md). Rules implemented: * resolve each layer's CRS: explicit EPSG parameter > .prj WKT > GeoJSON default (OGC:CRS84) — the same precedence documented for GeoGrid geo.py (audit §۶) but re-implemented independently with pyproj; * project-level analysis CRS = a *metric* CRS (UTM here); layers whose CRS differs are transformed **once**, recorded in CRSDefinition, and repeated transforms are blocked (GIS-DOUBLE-TRANSFORM guard); * degree-based (geographic) layers are never silently mixed into metric math — they are transformed or the study stops with an issue.

class CrsInfo [line 33]
CrsInfo.parse(cls, text: str) [line 40]

Parse a WKT1 string or 'EPSG:xxxx'.

transform_utm_to_project(easting: float, northing: float, zone: int, target_crs: str | int) [line 65]

Convert a northern-hemisphere UTM coordinate in zone 38/39 to the project's metric CRS. Coordinates are never silently treated as project coordinates when the selected UTM zone differs.

_wgs84_transformer(target_crs) [line 95]
transform_wgs84_to_project(lat: float, lon: float, target_crs: str | int) [line 102]

منتقل ValueError به‌صورت CRS/متری لازم است. خطای ورودی CRS مختصات متری پروژه. برای جای‌گذاری نقاط گازیتیر (درجه) روی شبکه با WGS84 (lat, lon) می‌شود تا هیچ مختصاتی حدس زده نشود (بند ۱۹).

class CrsService [line 121]

Per-project CRS bookkeeping and one-time transformation.

CrsService.__init__(self, analysis_epsg: int | None=None, analysis_wkt: str='') [line 124]
CrsService.analysis_crs(self) [line 136]
CrsService.set_analysis_from_majority(self, infos: list[CrsInfo], issues: IssueLog) [line 139]

Choose the most frequent projected (metric) CRS as analysis CRS.

CrsService.resolve(self, layer_name: str, layer_crs_wkt: str, issues: IssueLog, explicit_epsg: int | None=None) [line 156]

Resolve one layer's CRS; emits issues when missing/geographic.

CrsService.to_analysis(self, layer_name: str, x: float, y: float, source: CrsInfo, issues: IssueLog) [line 189]

Transform one point to the analysis CRS — once per layer.

CrsService.guard_no_double_transform(self, layer_name: str) [line 221]

Explicit guard: transforming a layer twice is a bug raise.

src/switchloc/gis/mapping/__init__.py

mapping — slot catalog, field catalog, layer/field mapping service.

src/switchloc/gis/mapping/field_catalog.py

GIS field catalog — real field names of the utility's shapefile exports. Provenance (recorded per main-prompt §5 legal rules): * observed directly in the Khalkhal sample export (Export_1404_08_17_12_34_09), * cross-checked against the *documented knowledge* of GeoGrid 4.2.0's config.json (reuse audit doc §10 "config.json"). No GeoGrid code is imported or copied; only the field-name knowledge is re-used as data.

s(v) [line 37]

Coerce a DBF attribute to a stripped string ('' for None).

first_value(rec: dict, fields, default=UNKNOWN) [line 42]

First non-empty value among `fields`, else `default`.

record_id(rec: dict) [line 51]
feeder_name(rec: dict) [line 55]

Feeder name via the documented fallback chain.

service_state(rec: dict, field: str=SERVICE_FIELD) [line 60]

Map T__outserv to 'in' | 'out' | 'unknown'. Provenance: GeoGrid 4.2.0 `Converter._switch_on` treats {'1','true','خارج از مدار'} as off; empty is treated as on there, but SwitchLoc keeps 'unknown' for empty to honour main-prompt rule «وضعیت نامشخص کلید» (QC item §7) — an empty value becomes an Information issue at reader level, not a silent assumption.

remote_controlled(rec: dict, field: str=REMOTE_CTRL_FIELD) [line 79]

Map T_ctrl_otm to 'yes' | 'no' | 'unknown'.

w_prefix_alias(name: str) [line 91]

ELWxxx <-> ELxxx layer-name alias. Provenance: this utility's exports are seen with and without the 'W' after 'EL' (GeoGrid 4.2.0 `shapefile_io._w_prefix_alias` behaviour, re-implemented independently). Returns None if the name doesn't start with 'EL'.

src/switchloc/gis/mapping/layer_mapping.py

Layer/field mapping service (main prompt §۵-۳, ARCHITECTURE.md §۲). Cycle: suggest() (user review) confirm() persistable JSON. No automatic mapping is ever final: a suggestion is only a *proposal* with a confidence and a reason; `confirm()` records who confirmed and when. Field maps are validated against the layer's real fields.

class MappingSuggestion [line 23]
MappingSuggestion.as_dict(self) [line 31]
default_field_map(fields: list[str]) [line 56]

Propose field_map: role real field name (first present).

class LayerMappingService [line 72]

Suggest confirm workflow, with JSON persistence for the project.

LayerMappingService.__init__(self) [line 75]
LayerMappingService.suggest(self, layers: list[LayerInfo]) [line 82]
LayerMappingService._suggest_slot(self, li: LayerInfo) [line 93]
LayerMappingService.confirm(self, layer: LayerInfo, slot_key: str, confirmed_by: str, field_map: Optional[dict]=None, enabled: bool=True) [line 112]

Record the user's final decision; validates the field map.

LayerMappingService.load(self, data: list[dict]) [line 132]

Restore previously confirmed mappings from project JSON.

LayerMappingService.dump(self) [line 138]
LayerMappingService.save_json(self, path) [line 149]
LayerMappingService.load_json(path) [line 154]

src/switchloc/gis/mapping/slots.py

Equipment slot catalog (docs/DATA_MODEL.md §۵ + GeoGrid audit §۱۰). Provenance: slot *pattern* follows GeoGrid 4.2.0 `EquipmentSlot` (audit doc §۳-۲); the slot set is extended per main prompt §۳-۲ (۱۶+ device types). Layer aliases come from the Khalkhal real export + documented GeoGrid config knowledge — data, not copied code. A slot is a *semantic* role a GIS layer can play. The mapping service suggests slots for discovered layers, but nothing is final until the user confirms (main prompt §۵-۳: «هیچ نگاشت خودکاری قطعی نیست»).

class EquipmentSlot [line 19]
slot_for_layer_name(name: str) [line 75]

Resolve a slot from the layer base name (ELWxxx ELxxx aliasing).

slot_for_unknown_layer(name: str) [line 81]

src/switchloc/gis/readers/__init__.py

readers — multi-format ingestion behind one GisReader contract.

src/switchloc/gis/readers/base.py

Reader contract (docs/ARCHITECTURE.md §۲ — رابط GIS Adapter). Every input format must satisfy `GisReader`: discover() returns layer metadata, read() returns traceable Features and surfaces every anomaly as a ValidationIssue — never a silent drop, never an input mutation.

class LayerInfo [line 17]

Metadata of one discovered layer.

LayerInfo.to_dict(self) [line 27]
class Feature [line 38]

One traceable feature (record) of a layer.

class GisReader [line 52]

Interface — subclass per format (shapefile/geojson/gpkg/tabular).

GisReader.discover(self, source: str | os.PathLike) [line 56]
GisReader.read(self, layer: LayerInfo, issues: IssueLog | None=None) [line 59]

src/switchloc/gis/readers/geojson_reader.py

GeoJSON GisReader (Phase 1). RFC 7946 declares WGS84 (OGC:CRS84) unless a legacy "crs" member exists; a degree-based layer entering a metric analysis pipeline is flagged so the CRS service transforms it once, explicitly.

_geom_points(geom: dict) [line 22]

Return (points_of_first_part, parts, geometry_type).

class GeoJSONReader [line 44]
GeoJSONReader.discover(self, source) [line 47]
GeoJSONReader.read(self, layer: LayerInfo, issues: IssueLog | None=None) [line 77]
_qc(li: LayerInfo, rec: dict, pts: list, idx: int) [line 109]

src/switchloc/gis/readers/gpkg_reader.py

GeoPackage GisReader (Phase 1). Implemented over pyogrio (GDAL binding) when available — the offline Windows bundle may omit it, in which case the reader reports a clear, actionable error instead of failing silently.

class GeoPackageReader [line 26]
GeoPackageReader.discover(self, source) [line 29]
GeoPackageReader.read(self, layer: LayerInfo, issues: IssueLog | None=None) [line 64]

src/switchloc/gis/readers/registry.py

Reader registry — format detection and fan-out discovery (Phase 1).

readers_for_source(source) [line 27]

Which reader classes can handle this path/folder?

discover_all(source, issues: IssueLog | None=None) [line 40]

Discover layers across every supported format under `source`. Returns (layers, reader_by_layer_name). Name collisions between formats are disambiguated as '<name>@<format>' and an Information issue records the rename — nothing is silently merged.

src/switchloc/gis/readers/shapefile_reader.py

Production Shapefile GisReader (Phase 1). Contract (docs/ARCHITECTURE.md §۲, audit tests GG-01/02/09): * discover layers recursively in a folder (with ELW/EL aliasing handled by the mapping catalog, not by renaming), * read features + geometry via pyshp (light path, like the audited GeoGrid pipeline) with a geopandas fallback, * keep every record traceable to (file, layer, record index), * surface every anomaly as a ValidationIssue instead of silently dropping anything, * NEVER modify the input files (GG-02 — hashes taken by ImportService). Re-used documented knowledge (no code import): pyshp-first reading, UTF-8/CPG encoding handling, per-fe

_read_prj(shp_path: Path) [line 49]
_read_cpg(shp_path: Path) [line 59]
class ShapefileReader [line 69]

GisReader implementation for ESRI shapefiles.

ShapefileReader.discover(self, folder: str | Path) [line 75]

Recursively find *.shp under `folder`; unique by base name.

ShapefileReader.read(self, layer: LayerInfo, issues: IssueLog | None=None) [line 103]

Read all features of one layer; log issues, never drop silently.

ShapefileReader._read_pyshp(self, li: LayerInfo, issues: IssueLog) [line 113]
ShapefileReader._read_geopandas(self, li: LayerInfo, issues: IssueLog) [line 147]
ShapefileReader._quality_issues(self, li: LayerInfo, rec: dict, pts: list, idx: int, parts: int=1) [line 176]

src/switchloc/gis/readers/tabular_reader.py

Tabular reader for Excel (.xlsx) and CSV — attribute-only layers. Use cases: لیست خاموشی‌ها، جداول اولویت بار، کتابخانه نوع تجهیزات. Geometry-less features are legal; they simply cannot enter topology and any attempt is flagged downstream by the assembler/QC.

_cell(v) [line 23]
class TabularReader [line 29]

Excel/CSV reader; each sheet (or file) becomes one layer.

TabularReader.discover(self, source) [line 33]
TabularReader._discover_csv(self, f: Path) [line 48]
TabularReader._discover_xlsx(self, f: Path) [line 61]
TabularReader.read(self, layer: LayerInfo, issues: IssueLog | None=None) [line 80]

TabularReader._read_csv(p: Path) [line 131]
TabularReader._read_xlsx(p: Path, sheet: str) [line 138]

src/switchloc/gis/segmentation.py

Polyline segmentation with registered-length distribution (test GG-03). Provenance: the *behaviour* is documented from GeoGrid 4.2.0 `Converter._split_line_into_segments` (audit §۳-۴) — split a polyline into 2-point pieces and distribute the registered length (`Shape_STLe`/`T_FeatureL`) proportionally to the Euclidean vertex distances. Re-implemented independently; degenerate segments (both ends the same vertex) are flagged, not silently dropped.

class SegmentRow [line 18]
split_polyline(points: list[tuple[float, float]], registered_length_m: float | None) [line 26]
Split a polyline into 2-point segments. Returns (segments, geometric_length). If `registered_length_m` is None/0 the geometric length is used for distribution (recorded by the assembler as an Information issue — an assumption, never silent).

src/switchloc/gis/snap.py

Metric-tolerance snapping (main prompt §۵-۵; replaces GeoGrid's decimal `round(x,3)` snap — audit §۷ decision, test GG-06). Mechanism: single-linkage clustering of points on a uniform grid of cell = tolerance, with neighbour-cell merging so border effects (the documented weakness of decimal rounding) cannot split one cluster. Outputs: * cluster key per input point, * cluster centroid (`coord_snapped`), * ambiguity list (a device with 2+ cluster candidates inside tolerance).

class SnapResult [line 21]
_cell(x: float, y: float, tol: float) [line 27]
snap_points(points: list[tuple[float, float]], tol_m: float=0.5) [line 31]
Single-linkage cluster of points within `tol_m` (metres). Pure Python (no scipy) so the offline desktop bundle stays light. Complexity O(n) over grid cells with 9-neighbour union passes.
ambiguous_device_candidates(point: tuple[float, float], cluster_centroids: dict[str, tuple], tol_m: float) [line 93]
All cluster centroids within tolerance of a device point (GG-05/06).
class CentroidIndex [line 104]
Exact radius queries over fixed centroids, preserving insertion order.
CentroidIndex.__init__(self, centroids: dict[str, tuple], tol_m: float) [line 107]
CentroidIndex.candidates(self, point: tuple[float, float]) [line 115]

src/switchloc/level1/__init__.py

src/switchloc/level1/formulas.py

Audited Level-1 formula kernels of TAV122-02/00 (Phase 01). Every function below carries its directive anchor (بند/جدول/رابطه) and is covered by unit tests anchored to the directive's own numerical examples (tests/unit/test_normalization.py, test_ep_tl_up.py, test_official_anchors.py). Phase 1 (official PDF verification): the zero-inversion policy of ۳۳ (گام ششم، صفحه ۵-۷-۷ (بند) is now CONFIRMED verbatim by the official TAV122-02/00 PDF: «معکوس مقادیر صفر برابر با مقدار 10 در نظر گرفته شود». The default policy is therefore 'confirmed'; the 'pending' lock remains available as an explicit argument for future re-verification rounds (main prompt
normalize(values, inverse: bool=False, inverse_zero_policy: str='confirmed', inverse_zero_value: float=10.0) [line 64]
Norm(Xi) = Xi / Xi (TAV122-02/00 ۵-۷-۷ بند ،۱ فرمول). Parameters ---------- values : list[float] Raw criterion values per feeder. inverse : bool Use 1/Xi before normalising (criteria 1 and 3 of ۱۶ جدول). inverse_zero_policy : 'confirmed' | 'pending' * 'confirmed' (default since Phase 1): zero inverts to `inverse_zero_value` (10) — verified verbatim in the official PDF: «معکوس مقادیر صفر برابر با مقدار 10 در نظر گرفته شود" (بند ۵-۷-۷، صفحه ۳۳). The value stays configurable so a company policy ch
score_feeders(raw_values, weights=None) [line 110]
Weighted ranking per ۱۶ جدول. raw_values: list of 6-tuples (criterion 1..6 raw values per feeder). Returns per-feeder dict(raw, weighted_score) — normalisation of each criterion is the caller's responsibility (so raw/normalised values can both be displayed per main prompt §12-1).
class ZoneFaultRow [line 134]
One row of ۱۵ جدول (زون خطادیده، بار قطع‌شده، بار قابل بازیابی kW.
class EPTLResult [line 142]
compute_ep_tl(zone_rows: list[ZoneFaultRow], total_load_kw: float) [line 149]
EP = max_zone(shed) / P ; TL = max_zone(restorable) / P. Provenance: ۱۴ جدول definitions + the directive's own worked example (۶-۷ شکل): shed 1800 kW of 2500 kW EP = 72٪; restorable 700 kW TL = 28٪. Only automation-capable ties count as transfer paths — the caller must already have filtered TieConnections by automation_capable.
ce_cn(e1_kwh: float, e2_kwh: float, cost_rials: float, rial_per_kwh: float=DEFAULT_RIAL_PER_KWH) [line 178]
Benefit/cost pair of ۱۸ جدول per the OFFICIAL text (page 35). Official definition (verbatim, verified Phase 1): * N = تعداد کلیدهای مورد نیاز E1 و E2 * **به ترتیب** = انرژی توزیع‌نشده **قبل و بعد** از کلیدگذاری [kWh] قاعده «به ترتیب»: حرف رسمی E2 = قبل، حرف رسمی E1 = بعد * (E2 - E1) × (CE = 1000 [ریال] = 1000 × (قبل - بعد) ارزش ریالی انرژی صرفه‌جویی‌شده (مزیت؛ برای طرح بهبوددهنده مثبت) * CN = هزینه خرید کلیدهای جدید و جابجایی کلیدهای موجود [ریال] (هزینه) * انتخاب طرح: بیشینه‌سازی CE/CN («طرح

class ZoneUP [line 220]
ZoneUP.u_hours(self) [line 227]

U = r / 60 × [ساعت/سال].

ZoneUP.ens_mwh(self) [line 232]

ENS = U × P [MWh] — ۸ رابطه) term / ۲۶ جدول sample.

up_rule_deviation(zones: list[ZoneUP]) [line 237]

Normalised RMS deviation of U_i×P_i from its mean (رابطه ۱۰). Ui×Pi = Uj×Pj is the directive's golden placement rule for zones of a feeder (پیوست ب، رابطه ۱۰). Placement algorithms minimise this deviation; zero means the ideal equal-U×P split is achieved.

ens_total(zones: list[ZoneUP]) [line 254]

ENS = Ui×Pi [MWh] — ۸ رابطه) aggregated over zones.

src/switchloc/level1/n_minus_1.py

n-1 engine of ۵-۴ گام‌های (TAV122-02/00، ۱۳/۱۴/۱۵ جداول) — Phase 4. Computes, on the REAL traced network of Phase 2: * جدول ۱۳ — feeder connections (tie pairs); per the official note only automation-capable connections count for transfer («تنها اتصالات از طریق کلید اتوماسیون محاسبه می‌شوند»). * جدول ۱۵ — the zone-by-zone n-1 loop: for a fault in every zone the sustained-shed and restorable load follow the algebra of ۸ رابطه): a fault in zone i hits the zones *downstream* of i (subtree of the radial zone tree, including zone i itself); upstream zones are restored by reclosing and are not counted (same pattern as U1(P1+P2+P3) in جد * رابطه ۸).

class NMinusOneSettings [line 55]
class FeederConnection [line 65]

One row of ۱۳ جدول (feeder-to-feeder connection via a tie).

class FeederEPTL [line 79]

row: EP/TL of one feeder + its worst fault zone ۱۴ جدول.

class NMinusOneResult [line 88]
class ZoneTree [line 103]

Radial zone tree of one feeder (normal state).

ZoneTree.subtree_zones(self, zone_id: str) [line 114]

Zone + all its descendants (fault-in-this-zone outage set).

ZoneTree.subtree_load_kw(self, zone_id: str) [line 123]
ZoneTree.zone_depth(self, zone_id: str) [line 127]

Depth in the zone tree (head zone = 0).

build_zone_tree(graph: NetworkGraph, trace: FeederTrace, zones_of_feeder: list[SwitchZone], load_kw_of_transformer: dict[str, float], issues: list) [line 136]

Zone adjacency tree of one feeder from its OWNED zones. Ownership follows the zone partition handed in (Phase 4 default: zones rebuilt on the strict core, so they are disjoint per feeder). Zones connect where they share a boundary (cut) node; the head zone is the one containing the feeder source node. A cycle in the zone graph is registered and resolved by min-zone_no parent choice.

transfer_capacity_kw(tree: ZoneTree, tie, spare_kw_of_feeder: dict[str, float], settings: NMinusOneSettings) [line 227]

kW this tie can back-feed into ``tree``'s feeder (0 = unusable). Only the backup feeder's capacity limits the transfer. Non-automation ties are ignored when ``automation_only_transfer`` (the official ۱۳ جدول note). When the backup rated capacity is unknown (absent from ``spare_kw_of_feeder``) the result is 0.0 — the conservative v1.9.1 policy; the caller surfaces the data gap as an issue instead of assuming an infinite/unlimited transfer.

spare_capacity_map(trees: dict[str, 'ZoneTree'], rated_kw: dict[str, float] | None) [line 250]

feeder key spare kW = rated - own load; {} when rated unknown.

zone_fault_rows(tree: ZoneTree, tie_transfer_kw: dict[str, float], barrier_nodes: set[str] | None=None) [line 262]

One ۱۵ جدول row per zone fault of the feeder. tie_transfer_kw: zone id kW restorable downstream of that zone through an automation tie (already capacity-checked). For a fault in zone i every downstream zone connected to a backup tie without crossing the isolated fault zone can be back-fed: the islands are the connected components of the downstream zone-adjacency graph (zones sharing a boundary node). Each island is restored up to min(island load, Tie capacities in that island) — see ``level1

evaluate_feeder_n1(tree: ZoneTree, tie_transfer_kw: dict[str, float], barrier_nodes: set[str] | None=None) [line 293]

جدول ۱۵ rows + جدول ۱۴ kernel V-02) anchor) EP/TL of one feeder).

run_n_minus_one(graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], zones_by_feeder: dict[str, list[SwitchZone]], ties: list, settings: NMinusOneSettings | None=None, transformer_load_kw: dict[str, float] | None=None) [line 305]

Full ۱۳/۱۴/۱۵ جداول pass over all feeders of the traced network.

src/switchloc/level1/ranking.py

Feeder ranking of ۶ گام (TAV122-02/00، ۱۶ جدول (۱)، فرمول ۷-۵-۷، بند). Six official criteria per feeder (weights ۱،۲،۲،۳،۱،۱ — PDF-verified): (1) معکوس تعداد ارتباطات نرمال‌شده — tie count of ۱۳ جدول EP, (V-01); (2) نرمال‌شده inverted with the official zero10 rule — from the n-1 engine (transferability inverted: the less a feeder can be backed up, the higher its priority; (3) معکوس TL نرمال‌شده — 4) مجموع اولویت (جدول ۱۴); (4) بارهای فیدر — load kW × priority)) روی ترانس‌های زون‌های مطالعه (یک مبنای واحد)؛ the default categories are unassigned in the sample GIS

priority is applied and REGISTERED (open decision 3, never guessed per load); (5)

class RankingSettings [line 37]
class FeederRankRow [line 50]
class RankingResult [line 68]
_safe_normalize(values, inverse: bool, st: RankingSettings) [line 74]

normalize() with the all-zero guard (all-zero column zeros).

load_priority_by_transformer(net: CanonicalNetwork, st: RankingSettings) [line 83]

transformer id جدول ۱۲ priority (assigned loads win, else default). Load categories are unassigned in the sample GIS (open decision 3); every transformer that falls back to the default is counted so the report can show the coverage — nothing is silently guessed (بند ۱۹).

feeder_criterion4(net: CanonicalNetwork, n1: NMinusOneResult, feeder_key: str, prio_of_tr: dict[str, float], st: RankingSettings,
load_kw_of_transformer: dict[str, float] | None=None) [line 98]

Criterion 4 = ۱۱/۱۲ (مجموع اولویت بارهای فیدر (جدول * ``load_weighted`` (default, A-14): priority × بار همان ترانس در مدل بار واحد** (kW) — یک مبنای واحد با زون‌ها/ENS. اگر نگاشت بار داده نشود، به‌صورت سابق capacity_kva (شاخص ظرفیت) استفاده می‌شود. * ``priority_sum``: the literal priority (count of load points).

rank_feeders(n1: NMinusOneResult, net: CanonicalNetwork, lam_matrix: list[FeederLambdaR] | None=None, settings: RankingSettings |
None=None, scope_feeder: str | None=None, scope_feeders=None, transformer_load_kw: dict[str, float] | None=None) [line 129]

Assemble the 6 criteria; single-feeder studies normalize only that feeder.

_settings_snapshot(st: RankingSettings) [line 224]

src/switchloc/level1/transfer.py

انتقال بار با Tieهای اتوماسیون — مدل مشترک جداول ۱۴/۱۵ و رابطه (۸)/(۱۱). معناشناسی (سطح اول، بدون پخش بار — 1 :(10-A. خطای زون/قطعه i مجموعه خروجی = زیردرخت همان گره در درخت رادیال است (بالادست با وصل مجدد بریکر برمی‌گردد — الگوی رابطه (۸)). 2. عضو معیوب جداسازی می‌شود: یال‌های آن از مدار خارج می‌شوند و عبور توان از آن ممنوع است. 3. جزیره‌های بازمانده = مؤلفه‌های همبندی اعضای پایین‌دست (بدون عضو معیوب): دو عضو متصل‌اند اگر نقطه جداسازی مشترکی داشته باشند و حداقل یک تجهیز رسانا روی آن نقطه توان را رد کند (گره "مانع" طبق ``barrier_cut_nodes`` جزیره‌ها را جدا می‌کند). برای زون‌ها، «گره مشترک رسانا» ملاک همبندی است؛ برای درخت قطعه‌ها، هر کلید

components(ids, links) [line 33]

مؤلفه‌های همبندی روی ``ids`` با یال‌های ``links`` (union-find فشرده، ریشه قطعی به ترتیب الفبایی شناسه).

shared_node_links(id_to_nodes: dict[str, set], barrier_nodes: set[str] | None=None) [line 59]

جفت‌هایی از شناسه‌ها که حداقل یک گره رسانای مشترک دارند. ``barrier_nodes``: گره‌های مرزی که هیچ تجهیزی در آن‌ها توان را رد نمی‌کند (همه تجهیزها NO غیراتوماسیون یا از سرویس خارج‌اند) — این گره‌ها جزیره‌ها را از هم جدا می‌کنند.

cut_node_links(upper_of: dict[str, str], lower_of: dict[str, str], barrier_nodes: set[str] | None=None) [line 84]

پیوند قطعه بالا قطعه پایین در هر کلید مرزی رسانا (درخت قطعه‌ها).

barrier_cut_nodes(graph, net, ties) [line 100]

گره‌های مرزی که توان را عبور نمی‌دهند. تجهیز روی گره X هنگام جداسازی خطا توان را عبور می‌دهد اگر: * مسیر وصل‌شونده نرمال آن NC یا وضعیت نامعلوم باشد و در سرویس باشد؛ * یا یک اتصال اتوماسیون‌پذیر روی همین گره باشد (طرح پس از خطا آن را می‌بندد — همان سازوکار بازپل). گره‌ای که *هیچ‌کدام* از تجهیزهایش این شروط را نداشته باشد (همه NO غیراتوماسیون یا OUT_OF_SERVICE) مانع است و جزیره‌ها را جدا می‌کند؛ بستن دستی این کلیدها طبق جدول ۱۳ خارج از بازپل است.

islands_with_ties(member_ids, member_load: dict[str, float], tie_cap_by_id: dict[str, float], links) [line 134]

بار کل قابل بازپل (kW) برای یک خطای مشخص. ``member_ids``: شناسه‌های اعضای پایین‌دست عضو معیوب (بدون خودش). ``member_load``: kW هر عضو. ``links``: یال‌های همبندی بین اعضا. (ندارد Tie هر عضو (۰ یعنی Tie ظرفیت :``tie_cap_by_id``.

src/switchloc/logging_setup.py

Central logging — one rotating log file in a per-user writable folder. Why: the old design appended raw tracebacks to ``switchloc_run_log.txt`` in the current directory (often read-only or inside the shipped folder, and it ended up inside the release archive with developer paths). * ``SWITCHLOC_LOG_DIR`` env var overrides the location. * Windows: ``%LOCALAPPDATA%/SwitchLoc/logs``; other OS: ``~/.local/state/switchloc``; last resort: the current directory. * ``setup_logging()`` is idempotent; every module uses ``logging.getLogger("switchloc.<area>")``.

_candidate_dirs() [line 27]
setup_logging(level: int=logging.INFO) [line 42]

Attach the rotating file handler once; return the log path.

log_path() [line 66]

Path of the active log file (``None`` if none is writable).

log_path_text() [line 71]
get_logger(area: str='app') [line 76]
format_exc(exc: BaseException) [line 81]
log_exception(exc: BaseException, area: str='app', context: str='') [line 86]

Log an exception with traceback (never raises).

src/switchloc/manual_placement.py

Manual switch placement & evaluation (روش جایابی دستی — درخواست کاربر). Workflow: 1. the engineer picks the location: a click on the map OR explicit UTM coordinates — `nearest_node()` snaps the point to the nearest graph node (a switch sits ON the node — same convention as ZoneBuilder / the automatic scheme switches); 2. `evaluate_manual_placement()` inserts virtual NC switches at the chosen node(s) and re-runs the SAME level-1 accounting the automatic schemes use (segment tree + رابطه ۸ with tie transfer) for the BEFORE and AFTER configurations of that feeder; 3. the returned indicators — ENS [kWh/yr], EP [%], TL [%], segment (zone) count an

class ManualEvaluation [line 45]

Result of one manual evaluation — before/after level-1 indicators.

ManualEvaluation.as_dict(self) [line 58]

JSON-safe snapshot (GUI table / report / copy-paste).

nearest_node(graph: NetworkGraph, x: float, y: float, restrict_to: set | None=None, max_dist_m: float | None=None) [line 72]

UTM point (nearest node id, distance [m]). ``restrict_to`` limits candidates to the given node set (e.g. one feeder's traced nodes). ``max_dist_m`` refuses far-away clicks (None = unlimited). Returns ``(None, inf)`` when nothing qualifies.

_ep_tl_of_segments(segs: SegTree, total_load_kw: float, transfer_cap_by_seg: dict[str, float], barrier_nodes: set[str] | None=None) [line 100]

(EP %, TL %, worst segment) — worst single fault in the tree. A fault in segment i sheds its subtree (minus what the ties restore) shed_i / P = EP contribution of that fault; the worst fault over all segments defines EP (جدول ۱۴). TL is the maximum over ALL single faults of the restorable share (the same independent maximum the official kernel ``compute_ep_tl`` uses). The islands restored by automation ties are the connected components of the remaining segment tree (cut nodes link the segment

_zones_for_feeder(zones: ZoneResult, feeder: str) [line 150]

rebuild_zones_with_manual_switches(graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], switch_nodes_by_feeder: dict[str, list[str]], policy: ZonePolicy | None=None) [line 168]

Rebuild an isolated after-state zone model with virtual NC switches. The canonical project and its graph are never mutated. The virtual devices are study overlays only; they are not persisted as installed assets until the engineer explicitly approves/exports them.

evaluate_manual_placement(graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], zones: ZoneResult, ties, feeder_key: str, switch_nodes: list[str], lam_matrix: list[FeederLambdaR] | None=None, settings: EngineSettings | None=None, cap_map: dict | None=None, switch_cost_rials: float | None=None, allow_existing_cut: bool=False, zone_policy: ZonePolicy | None=None) [line 214]

Before/after level-1 indicators for manual switch node(s). ``switch_nodes``: graph node ids (from `nearest_node` or typed UTM coordinates snapped by the caller). Every node must belong to the feeder's own traced tree — a foreign node is refused with an explicit issue, never silently moved.

evaluate_manual_interfeeder_tie(graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], zones: ZoneResult, ties, feeder_a: str, node_a: str, feeder_b: str, node_b: str, lam_matrix=None, settings=None, cap_map=None, automation_capable=False, zone_policy: ZonePolicy | None=None) [line 432]

Evaluate a proposed two-point NO tie between two adjacent feeders. The tie is represented as a normally-open transfer opportunity at each selected feeder endpoint. Restoration is credited only when the user explicitly marks the link automation-capable AND usable reserve data for both feeders is supplied. This is a Level-1 screening model; it does not replace load-flow, protection coordination, voltage-drop or route design. Returns (evaluations_by_feeder, issues).

src/switchloc/persistence/__init__.py

SQLite project persistence (docs/ARCHITECTURE.md D-03).

src/switchloc/persistence/outage_workbooks.py

Portable, project-local source workbooks for outage processing. XLSX is already a ZIP archive. Store its bytes intact, with a digest, so Save As uses SQLite's normal backup and no external path is required.

save_workbook(db_path: str, feeder_key: str, source: str | Path) [line 19]

list_workbooks(db_path: str) [line 32]

export_workbook(db_path: str, feeder_key: str, target: str | Path) [line 41]

src/switchloc/persistence/schema.py

SQLite DDL — tables generated 1:1 from docs/DATA_MODEL.md (Phase 1). Schema versioning (``PRAGMA user_version``) ---------- * ``SCHEMA_VERSION`` is the structure version this build understands. * Version 1 is the baseline: every project created by v1.0–v1.8 (they carry ``user_version = 0`` and are identical in structure because new tables were always added with ``CREATE TABLE IF NOT EXISTS``). * To change an existing table in a future release: bump ``SCHEMA_VERSION`` and add ``MIGRATIONS[<new version>] = "<SQL>"`` (the SQL upgrades version N-1 N). ``ProjectStore`` backs the file up, applies the steps in one

src/switchloc/persistence/store.py

SQLite project store (main prompt §۱۶ — ذخیره/بازگشایی + بازتولیدپذیری). Save/load is deterministic: entities are written in sorted-id order, so two identical imports produce byte-identical databases (regression test test_persistence.py).

class SchemaTooNewError [line 26]

The project file was written by a newer SwitchLoc build.

_num(v) [line 30]

class ProjectStore [line 34]

One SQLite file per project (D-03).

ProjectStore.__init__(self, path: str | Path) [line 37]

ProjectStore.schema_version(self) [line 48]

ProjectStore._ensure_schema(self) [line 51]

ProjectStore._backup(self, tag: str) [line 85]

Copy the project next to itself before a destructive upgrade.

ProjectStore.close(self) [line 97]

ProjectStore.save_network(self, net: CanonicalNetwork, issues: IssueLog, run: StudyRunMeta) [line 102]

ProjectStore.load_network(self, run_id: str | None=None) [line 203]

Rebuild a CanonicalNetwork + issues from the newest/selected run.

ProjectStore.runs(self) [line 317]

ProjectStore.save_topology(self, source_run_id: str, zones: list, zone_of_edge: dict, zone_of_node: dict, boundaries: list, traces: list, ties: list, settings: dict, user: str, algo_version: str, summary: dict, verdict: str, scenarios: list | None=None) [line 329]

Persist one topology execution (zones, boundaries, ties, traces). Deterministic id: identical inputs/settings identical topo_run_id identical database bytes (regression-test guarantee).

ProjectStore.list_topo_runs(self) [line 389]

ProjectStore.save_outages(self, records: list, lam: list, settings: dict, user: str, algo_version: str, summary: dict, verdict: str, input_file: str='', input_sha256: str='') [line 401]

Persist one outage-analysis execution (events + /r matrix). Deterministic id: identical inputs/settings identical outage_run_id (same reproducibility guarantee as save_topology).

ProjectStore.list_outage_runs(self) [line 450]

ProjectStore.load_outages(self, outage_run_id: str | None=None) [line 459]

Reload a persisted outage execution as plain dicts (UI/report).

ProjectStore.load_topology(self, topo_run_id: str | None=None) [line 510]

Reload a persisted topology execution as plain dicts (UI/report).

ProjectStore.save_placement(self, snapshot: dict, settings: dict, user: str, algo_version: str, source_run_id: str | None=None, topo_run_id: str | None=None, outage_run_id: str | None=None) [line 563]

Persist one placement execution (فاز ۴) — deterministic id. ``snapshot`` is the JSON-safe dict of ``PlacementResult.to_dict()``; identical inputs identical placement_run_id identical database bytes.

ProjectStore.list_placement_runs(self) [line 629]

ProjectStore.load_placement(self, placement_run_id: str | None=None) [line 641]

Reload a persisted placement execution as plain dicts (UI).

ProjectStore._now(self) [line 715]

ProjectStore.log_setting_change(self, run_id: str, key: str, old, new, user: str, at: str) [line 719]

ProjectStore.save_feeder_capacity(self, rows: list[dict], user: str='ui-user', at: str | None=None) [line 730]

Upsert ظرفیت فیدرها؛ خروجی FeederCapacity.as_dict() = ``rows``.

ProjectStore.load_feeder_capacity(self) [line 763]

feeder_key FeederCapacity (از جدول پروژه؛ خالی = وارد نشده).

ProjectStore.save_tie_review(self, rows: list[dict], user: str='ui-user', at: str | None=None) [line 785]

ProjectStore.load_tie_reviews(self) [line 808]

ProjectStore.save_plan_review(self, rows: list[dict], user: str='ui-user', at: str | None=None) [line 818]

ثبت گردش کار تأیید مهندسی (پس از کنترل ظرفیت — نیاز ۹/۱۰).

ProjectStore.load_plan_reviews(self) [line 845]

ProjectStore.set_kv(self, key: str, value, user: str='ui-user') [line 865]

Upsert one JSON-safe value under a stable key.

ProjectStore.get_kv(self, key: str, default=None) [line 879]

Read one JSON value; missing/corrupt ``default``.

ProjectStore.delete_kv(self, key: str) [line 895]

src/switchloc/placement/__init__.py

Placement package — Phase 4 (گام‌های ۴-۸ of the first-level method). golden.py — قاعده طلایی U×P (روابط ۹/۱۰) trunk sectioning schemes.py — جدول ۱۷ scheme families (الف/ب/ج) + پیوست ب economics.py — ENS accounting (رابطه ۸ + (transfer + جدول ۱۸ service.py — CE/CN PlacementEngine orchestration

src/switchloc/placement/acceptance.py

Scheme acceptance / plan selection (۱. کاهش ENS نیاز ۱۰ — معیارهای انتخاب طرح). انتخاب طرح حداقل با این پنج معیار انجام می‌شود: مثبت و معنادار باشد (ENS = E2 - E1 > 0 و ≥ آستانه) ۲. منفعت اقتصادی مثبت باشد (CE - CN > 0 (۳. ظرفیت انتقال و محدودیت‌های فنی رعایت شده باشند (ظرفیت کنترل‌شده، رابطه ۱۱ برقرار — نتیجه ۴ (check_scheme_capacity. تعداد کلیدها و بودجه از سقف تعیین‌شده تجاوز نکنند ۵. در صورت نبود طرح قابل قبول، نتیجه صریحا «عدم توجیه نصب کلید» باشد قاعده طلایی این نیاز: نرم‌افزار **به اجبار برای هر فیدر طرح معرفی نمی‌کند**. خروجی recommendation به‌ازای «هر» فیدر یک سطر دارد، اما وضعیت آن سطر می‌تواند باشد: * ``accepted`` — طرح اصلی (ب

class AcceptanceCriteria [line 48]

آستانه‌های پذیرش طرح — همه قابل تنظیم از GUI (نیاز ۱۲/تنظیمات).

AcceptanceCriteria.as_dict(self) [line 63]
class SchemeDecision [line 75]

نتیجه ارزیابی یک طرح (یک سطر جدول ۱۸) علیه معیارهای پذیرش.

SchemeDecision.acceptable(self) [line 83]
SchemeDecision.pending_capacity(self) [line 87]
evaluate_scheme(comp, capacity_check, criteria: AcceptanceCriteria) [line 91]

ارزیابی یک مقایسه جدول ۱۸ علیه معیارهای ۱ تا ۵ (نیاز ۱۰). ``comp: SchemeComparison`` یا dict هم‌ساختار؛ ``capacity_check: CapacityCheck`` خروجی check_scheme_capacity.

select_plans(comparisons, decisions: dict[str, SchemeDecision], ranking_rows: list, schemes: list) [line 189]

طرح اصلی هر فیدر — «بدون اجبار» (قلب نیاز ۱۰). برای هر فیدر (به ترتیب رتبه): * میان طرح‌های پذیرفته‌شده آن فیدر، بیشینه ``accepted`` ``CE/CN`` * وگرنه میان طرح‌های «نیازمند تأیید ظرفیت»، بیشینه ``needs_capacity_confirmation`` ``CE/CN`` (منتظر ورود ظرفیت/تأیید مهندس) * وگرنه ``no_justification`` با پیام صریح «عدم توجیه نصب کلید»

class PlanApprovalError [line 274]

تأیید نهایی مهندسی برای طرح کنترل‌ظرفیت‌نشده مجاز نیست (نیاز ۹).

approve_plan(plan_row: dict, capacity_map: dict | None=None) [line 278]

گردش کار تأیید مهندسی — دروازه سخت (نیاز ۹ + ۱۰). قاعده: طرحی که ``status != accepted`` یا ``capacity_status`` کنترل نشده دارد هرگز تأیید نهایی نمی‌گیرد؛ تلاش برای تأیید، خطای صریح ``PlanApprovalError`` می‌دهد (نه سکوت و نه تأیید خودکار).

src/switchloc/placement/capacity.py

Feeder capacity model ظرفیت نامی فیدر (ورودی — ``nominal_kw`` * :نیاز ۹ — ظرفیت نامی / پیک / رزرو / اضطراری). چهار کمیت به‌ازای هر فیدر کاربر/پرونده شبکه) * ``peak_kw`` — بار پیک موجود فیدر (از «مدل بار» تب Load Model می‌آید — یک ورودی واحد برای بار؛ این تب فقط نامی/اضطراری می‌گیرد و پیک از تنظیمات مدل بار همگام می‌شود) * ``reserve_kw`` — ظرفیت رزرو = نامی - پیک (محاسبه‌شده؛ می‌تواند منفی باشد = اضافه‌بارداری، که صراحتا گزارش می‌شود و پنهان نمی‌شود) * ``emergency_kw`` — ظرفیت اضطراری (کوتاه‌مدت، مبنای رابطه ۱۱ پیوست ب) — از ظرفیت عادی «تفکیک‌شده» است؛ حدس زده نمی‌شود (بند ۱۹). وضعیت کنترل ظرفیت هر فیدر و هر طرح: * ``ok`` — داده کافی و قیود

class FeederCapacity [line 43]

ظرفیت‌های یک فیدر + منشأ هر مقدار (ردیابی‌پذیر).

FeederCapacity.reserve_kw(self) [line 56]

ظرفیت رزرو = نامی - پیک (خام؛ منفی = اضافه‌بارداری).

FeederCapacity.usable_reserve_kw(self) [line 63]

ظرفیت رزرو قابل استفاده برای انتقال (منفی صفر).

FeederCapacity.overloaded(self) [line 69]
FeederCapacity.has_basic_data(self) [line 74]

نامی و پیک هر دو مشخص — و نامی پیش‌فرض نرم‌افزاری نباشد. مقدار پیش‌فرض ۷۰۰۰ kW فرم (منشأ «default») فرض اولیه UI است، نه اندازه‌گیری؛ برای تأیید مهندسی به‌حساب نمی‌آید (نیازمند تأیید ظرفیت می‌ماند).

FeederCapacity.status(self) [line 85]

وضعیت فیدر: ok یا needs_capacity_confirmation.

FeederCapacity.emergency_or_none(self) [line 89]

ظرفیت اضطراری — بدون حدس؛ None یعنی کنترل رابطه ۱۱ ممکن نیست.

FeederCapacity.as_dict(self) [line 93]
FeederCapacity.from_row(row: dict) [line 102]
class CapacityCheck [line 125]

نتیجه کنترل ظرفیت یک طرح (قابل درج در جدول ۱۸ و گزارش).

CapacityCheck.is_blocking(self) [line 132]

کنترل‌نشده یا نقض‌شده = مجوز تأیید نهایی مهندسی ندارد.

CapacityCheck.as_dict(self) [line 136]
check_scheme_capacity(scheme, cap_of: dict[str, FeederCapacity], load_kw_of: dict[str, float] | None=None) [line 141]
کنترل ظرفیت انتقال یک طرح کلیدگذاری. قواعد (پیوست ب + نیاز ۹): * طرح فقط-NC (قطعه‌بندی داخل فیدر، بدون انتقال) ظرفیت انتقال لازم ندارد؛ اما اگر پیک > نامی باشد «نیازمند تأیید ظرفیت» می‌شود (اضافه‌بارداری موجود باید صریح دیده شود). * طرح دارای NO (انتقال/پشتیبانی دوسویه) ظرفیت اضطراری هر دو فیدر باید مشخص باشد؛ نامشخص = needs_capacity_confirmation (بدون حدس). * بار پیک هر دو فیدر طرح ۱٫۵ کلیدی باید ≥ ظرفیت اضطراری باشد (رابطه ۱۱)؛ نقض = violated.

src/switchloc/placement/economics.py

ENS accounting + ۱۸ جدول (CE/CN) scheme comparison — Phase 4. Energy model (level-1, ۸ رابطه algebra + the transfer extension of رابطه ۱۱): a fault in segment i hits the segments *downstream* of i (subtree of the radial segment tree, i included) — upstream segments are restored by reclosing and never appear in the outage terms, exactly the ``U1(P1+P2+P3)`` pattern of ۸ رابطه. An automation tie at segment k back-feeds the whole branch of the faulted subtree in which k sits — the child of the faulted segment containing k, i.e. everything between the isolation boundary of the faulted segment and the tie, plus every branch hanging off that path —

class CostModel [line 42]
class SchemeComparison [line 54]
One row of ۱۸ جدول.
class EconomicsResult [line 69]
class SegTree [line 81]
build_seg_tree(graph: NetworkGraph, trace: FeederTrace, tree: _Tree, cut_nodes: set[str], load_kw_at: dict[str, float], edge_set: set) [line 91]
Edge components of the feeder tree when traversal is blocked at ``cut_nodes`` (same convention as ZoneBuilder: a boundary switch node separates two segments). The edge above a cut belongs to the upper segment; the cut node's own load is counted on the lower (below) side. Segment parent relations are produced directly by the BFS — no second pass.
_edge_between(graph: NetworkGraph, edge_set: set, a: str, b: str) [line 144]
ens_of_configuration(segs: SegTree, lam_per_year: float, r_minutes: float, switching_minutes: float, transfer_cap_by_seg: dict[str, float], total_length_m: float | None=None, barrier_nodes: set[str] | None=None) [line 155]
ENS [kWh/yr] of the feeder for one (cuts, ties) configuration. ``transfer_cap_by_seg``: segment id kW restorable downstream of that segment through an automation tie (inf when the backup capacity is explicitly declared unlimited — flagged by the caller). For a fault in segment i the remaining segments form islands (connected components through cut nodes that can conduct); each island is restored up to min(island load, Tie capacities in that island). Returns (ens, per-fault rows). ``barrier_n
_subtree_segs(segs: SegTree, sid: str) [line 199]
_subtree_load(segs: SegTree, sid: str) [line 208]
existing_cut_nodes(zone_list) [line 213]
Nodes where two zones of THIS feeder meet (= current zone cuts). ``zone_list`` = the feeder's own zones (Phase-4 default: strict-core zones, disjoint per feeder).
scheme_cost(scheme: SwitchScheme, cm: CostModel) [line 229]
compare_schemes(schemes: list[SwitchScheme], ens_before: dict[str, float], ens_after_of: dict[str, dict[str, float]], cm: CostModel) [line 235]
جدول ۱۸ rows; ``selected`` = max CE/CN per involved feeder («طرح‌هایی که مقدار CE/CN بیشتری دارند به‌عنوان طرح اصلی برای هر اولویت انتخاب می‌شوند»).

src/switchloc/placement/golden.py

Golden-rule sectioning (پیوست ب، روابط ۹ و ۱۰) — Phase 4. «قاعده طلایی» (۱۰): ``U_i × P_i = U_j × P_j`` — the directive's ideal for the zones of one feeder. With the level-1 uniform fault-rate assumption (_i length, ۹ رابطه; A-15) and a common r, minimising the U×P deviation reduces to choosing trunk section points that equalise ``length_i × load_i`` of the resulting segments. * trunk = longest path of the radial feeder tree (by length); * segment load = transformer-capacity index hanging below the segment (the documented open-decision-4 proxy); * segment length includes ALL branches («رابطه ۹: «با احتساب تمام شاخه‌ها»); * exact sea
class GoldenSettings [line 27]
class GoldenSectioning [line 38]
class _Tree [line 52]
build_feeder_tree(graph: NetworkGraph, trace: FeederTrace, edge_set: set, load_kw_at: dict[str, float]) [line 61]
Radial node tree of one feeder over ``edge_set`` (the feeder's OWNED edges — Phase 4 default: the strict-core zones' edges). Root = source.
_edge_of(graph: NetworkGraph, edge_set: set, child: str, parent: str | None) [line 97]
The feeder edge between a child and its tree parent.
feeder_trunk(graph: NetworkGraph, trace: FeederTrace, edge_set: set) [line 106]
Longest root path of the radial feeder tree (by length).
owned_edges(zones) [line 134]
Union of the edges of one feeder's own zones.
_root_of(t: _Tree) [line 142]
segment_stats(t: _Tree, cuts: list[str]) [line 146]
Per-segment (length, load) between consecutive trunk cut points. Segment i spans from ``starts[i]`` (root for i=0, else cuts[i-1]) down to

(excluding) ``cuts[i]``; its load/length = subtree(start) - subtree(end) — off-trunk branches hang below their trunk node and are therefore counted exactly once (رابطه ۹ branch inclusion).

up_deviation(segments: list[dict]) [line 168]

Normalised RMS deviation of U_i×P_i (رابطه ۱۰) for the segments. With _i length_i and a common r, U_i×P_i length_i × load_i, so the constant factors cancel inside the relative deviation.

golden_section(graph: NetworkGraph, trace: FeederTrace, edge_set: set, load_kw_at: dict[str, float], lam_per_year: float, r_minutes: float, existing_cut_nodes: set[str], switch_nodes: set[str] | None=None, settings: GoldenSettings | None=None, excluded_nodes: set[str] | None=None) [line 182]

Choose ``target_zones - 1`` trunk section points minimising the U×P deviation (exact search). Returns None for degenerate trunks.

src/switchloc/placement/schemes.py

Switching-scheme generation of پیوست ب + جدول ۱۷ (گام ۷) — Phase 4. Three scheme families of جدول ۱۷ (A-12: the official table is an image; the text + پیوست ب relations define them): (الف) ``sequential_nc`` — کلیدگذاری متوالی NC (پیوست ب-۱): golden-rule section points along the feeder trunk (رابطه ۱۰); (ب) ``one_half_switch`` — طرح ۱٫۵ کلیدی (پیوست ب-۲، روابط ۱۱-۱۳): one end-NC per feeder + one NO between two feeders; validity rule of رابطه ۱۱: both feeders loaded ≤ 2/3 of the emergency capacity (checked when rated capacities are known — A-10); (ج) ``head_tie`` — سایر طرح‌های مانوری: tie candidates in the head zones (ارتباط زون‌های ابتدایی).

class PlacementSettings [line 33]

class ProposedSwitch [line 53]

class SwitchScheme [line 61]

generate_schemes(graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], trees: dict, ranking_rows: list, ties: list, zone_no_of: dict, load_kw_at: dict[str, float], lam_of: dict, r_of: dict, cut_nodes_of: dict[str, set], edge_set_of: dict[str, set], settings: PlacementSettings | None=None) [line 71]

Generate the جدول ۱۷ families; returns (schemes, golden_by_feeder, issues). Deterministic throughout (stable ids, sorted iteration).

src/switchloc/placement/service.py

PlacementEngine — orchestration of the Phase-4 pipeline (گام‌های ۴-۸). Pipeline (pure computation, no I/O — the application layer persists): n-1 engine (جداول ۱۳/۱۴/۱۵) feeder ranking (فرمول ۱ + جدول ۱۶) scheme generation (جدول ۱۷ + پیوست ب) ENS before/after (رابطه ۸ + transfer of رابطه ۱۱) جدول ۱۸ CE/CN recommendation (طرح اصلی برای هر اولویت = ``max CE/CN``). Area model (default ``strict``): the n-1/zoning analysis runs on each feeder's GUARANTEED core (Phase-2 ``traces_strict`` — unknown-state switches treated as open, so feeders are disjoint). The flooded normal-state trace covers the whole interconnection island for every feeder (A

class EngineSettings [line 46]

class PlacementResult [line 58]

PlacementResult.to_dict(self) [line 72]

JSON-safe snapshot for reports and persistence.

node_load_map(graph: NetworkGraph, net: CanonicalNetwork) [line 147]

node transformer capacity_kva (legacy proxy — kept for the report path; the engine itself uses the three-level load model of نیاز ۱۱ through ``build_node_loads``).

_spare_map(result_n1: NMinusOneResult, rated_kw: dict | None, capacity_map: dict[str, FeederCapacity] | None=None, peak_kw_of_feeder: dict[str, float] | None=None) [line 158]

ظرفیت پشتیبان هر فیدر برای انتقال (kW). اولویت: رزرو قابل استفاده مدل ظرفیت — **نامی - بار پیک**، که بار پیک آن از *همان ورودی واحد مدل بار* (نیاز ۱۱؛ P هر فیدر) گرفته می‌شود، نه یک عدد جدا. نامی پیش‌فرض نرم‌افزاری (منشأ «default») به‌عنوان تأییدنشده تلقی و رد می‌شود. در نبود آن، ظرفیت نامی قدیمی (A-10) با بار پیک همان مدل بار (در نبودش، بار تخصیص‌شده زون‌ها). اگر هیچ‌کدام موجود نباشد، ظرفیت انتقال صفر منظور می‌شود (سیاست محافظه‌کارانه v1.9.1 — انتقال تأییدنشده هرگز فرض بی‌نهایت نمی‌شود) و ه

class PlacementEngine [line 202]

Runs گام‌های ۴-۸ of the first-level method on a traced network.

PlacementEngine.__init__(self, settings: EngineSettings | None=None) [line 205]

PlacementEngine.run(self, graph: NetworkGraph, net: CanonicalNetwork, traces, zones, ties, lam_matrix: list[FeederLambdaR] | None=None, traces_strict=None, feeder_capacity: dict[str, FeederCapacity] | None=None, scope_feeder: str | None=None, scope_feeders=None) [line 208]

اجرای گام‌های ۴-۸. ``feeder_capacity``: مدل ظرفیت فیدرها (نیاز ۹) — رزرو/اضطراری. ``scope_feeder``: مطالعه تک‌فیدر (نیاز ۱۲) — رتبه‌بندی روی کل شبکه عیب‌یج می‌شود (نرمال‌سازی ثابت می‌ماند) و سپس فقط طرح‌ها/نتیجه‌های همان فیدر گزارش می‌شوند.

PlacementEngine._economics(self, graph, net, traces, n1: NMinusOneResult, ties, schemes: list[SwitchScheme], node_load, lam_of: dict, r_of: dict, cut_nodes_of: dict, edge_set_of: dict, cap_map: dict[str, FeederCapacity] | None=None, peak_kw_of_feeder: dict[str, float] | None=None) [line 528]

src/switchloc/reliability/__init__.py

فاز ۳ (بار و خاموشی). گام ۲ روش سطح اول (بند ۷-۷-۳): ورود لیست خاموشی‌ها، طبقه‌بندی شش‌طبقه بند ۱۰ پرامپت مادر بر محور چهار سطح — reliability رسمی شکل ۵ (A-11)، مدل اولویت بار جدول ۱۲ و ماتریس r/ فیدرها برای معیارهای ۵ و ۶ جدول ۱۶. Modules: outages — خواندن لیست خاموشی Excel/CSV OutageRecord classify — طبقه‌بند قاعده‌محور شش‌طبقه + گروه نوع اشکال (سطح دوم) — lambdar — تجمیع فیدر + گزارش‌های چهار r/ سطح شکل ۵ loads — اولویت بار جدول ۱۲ (قابل تغییر توسط شرکت — ابهام A-03)

src/switchloc/reliability/classify.py

Six-class outage classifier (بند ۱۰ پرامپت مادر) on the four official review levels of ۵ شکل (A-11 — resolved in Phase 1). Six mandatory classes (domain.enums.OUTAGE_CLASSES): ۱. قابل کاهش با کلیدگذاری — random line faults cleared from an upstream device (breaker/recloser): sectionalizing + automation reduces the affected zone / restoration time. ۲. نیازمند تعمیر یا تعویض تجهیز — a named piece of equipment failed (transformer, insulator, conductor, …); the remedy is the repair program, not switching (گام ۲: «حذف خاموشی‌هایی که با سرویس/تعویض شبکه فرسوده جبران می‌شوند»). ۳. ناشی از هماهنگی حفاظتی — a downstream protective device (fuse cutout / _matches(text: str, patterns) [line 126]

First matching pattern (substring) or ''.

fault_type_group(cause_raw: str) [line 134]

سطح دوم شکل ۵ — نوع اشکال (Unmapped grouped). ''.

classify_event(rec) [line 142]

Return (outage_class, rule_id) for one OutageRecord. Pure function — no mutation; the service assigns the result onto the record. Rule order is fixed (see module docstring).

classify_all(records) [line 193]

Classify records in place; return rule/class counters. ``records[i].outage_class / .rule_id / .cause_group`` are filled. Unmapped-cause events (R-UNK-99) keep the «داده ناکافی» class and are reported by the service as Error issues (OUT-CAUSE-UNMAPPED) so an unknown future cause string can never silently pass (الزام بند ۱۰).

cause_known(rec) [line 213]

src/switchloc/reliability/coverage.py

تعداد رخدادهای تصادفی ÷ زمان مواجهه = واقعی). مفهوم رسمی: نرخ خرابی Exposure Time — نیاز ۸) Data-coverage / exposure-time model برحسب سال. زمان مواجهه باید بر اساس «واقعیت پوشش داده» محاسبه شود، نه تعداد سال‌های تقویمی و نه صرفا سال‌هایی که رخداد (Exposure Time) داشته‌اند: * «نبود رکورد» لزوما به معنی «نبود خاموشی» نیست ماه‌های داخل پنجره پوشش که هیچ رکوردی ندارند به‌عنوان «ماه فاقد داده» شناسایی و به‌صورت پیش‌فرض از زمان مواجهه حذف می‌شوند (مهندس می‌تواند صریحا پوشش بدهد). * دوره پوشش داده (شروع/پایان) قابل تعریف صریح است؛ اگر داده نشود از خود رکوردها استنتاج می‌شود (اولین/آخرین ماه دارای رکورد). * زمان مواجهه بر حسب «روز واقعی» جمع م

is_jalali_leap(year: int) [line 30]

کبیسه‌بودن سال شمسی با الگوی ۳۳ ساله (الگوریتم متداول بیرجندی).

jalali_month_days(year: int, month: int) [line 35]

طول واقعی یک ماه شمسی برحسب روز.

month_index(year: int, month: int) [line 46]

کلید مرتب‌پذیر یک ماه: (year*12 + month-1).

parse_year_month(text: str) [line 51]

«1404/07» یا «1404-7» یا «1404» (1404, 7)؛ نامعتبر «1404». None» تنها به‌معنای کل سال (ماه ۱ تا ۱۲) تفسیر نمی‌شود؛ پنجره پوشش باید صریح باشد، در غیر این صورت از داده استنتاج می‌شود.

enumerate_months(start: tuple[int, int], end: tuple[int, int]) [line 75]

همه ماه‌های (year, month) از start تا end شامل دو سر.

class CoverageResult [line 92]

نتیجه تحلیل پوشش داده — منشأ مخرج (قابل ردیابی و JSON-safe).

CoverageResult.has_coverage(self) [line 109]
CoverageResult.as_dict(self) [line 112]
infer_window(records) [line 125]

استنتاج پنجره پوشش از خود رکوردها (اولین/آخرین ماه دارای رکورد).

compute_coverage(records, start: tuple[int, int] | str | None=None, end: tuple[int, int] | str | None=None, force_cover=(), force_exclude=()) [line 140]

پوشش داده و زمان مواجهه واقعی را از رکوردها محاسبه می‌کند. * ``start``/``end``: دوره پوشش صریح («1404/07») — بدون آن‌ها پنجره از اولین/آخرین رکورد استنتاج می‌شود. * ``force_cover``: ماه‌های فاقد رکورد که مهندس صریحا دارای پوشش اعلام می‌کند (مثلا «پرونده آن ماه در بایگانی است»). * ``force_exclude``: ماه‌های دارای رکورد که باید حذف شوند (مثلا ثبت ناقص).

src/switchloc/reliability/indices.py

شاخص‌های پیوستگی (SAIDI / SAIFI / CAIDI / ASAI) از همان داده خاموشی. منشأ داده همان دفتر خاموشی است (ستون‌های «مدت قطع»، «انرژی توزیع‌نشده»، «قطع‌کننده»، «نام فیدر»). داده خاموشی ستون «تعداد مشترکین» ندارد (A-08)، پس سهم مشترکین متأثر از **انرژی توزیع‌نشده** استنتاج می‌شود: (P_f · T_i = E_i / f_i) (سهم بار قطع‌شده رخداد i از فیدر f) که در آن E همان «انرژی توزیع‌نشده» (واحد: MWh یا kWh — خودکار تشخیص داده می‌شود)، P بار فیدر (kW) و T مدت قطع (ساعت) است. سپس: SAIFI = f_i · T_i / سال مواجهه [قطع/مشترک/سال] SAIDI = / سال مواجهه [دقیقه/مشترک/سال] CAIDI = SAIDI / SAIFI [دقیقه] ASAI = 1 - SAIDI / 525600 سه محدودیت صریح (بدون آن‌ها عدد س

class FeederServiceIndices [line 47]

شاخص‌های یک فیدر برای پنجره مواجهه خودش.

FeederServiceIndices.has_data(self) [line 73]

رخداد واقعی برای این فیدر ثبت شده است؟ (۰ ≠ «بدون داده»)

FeederServiceIndices.as_dict(self) [line 77]
class ServiceIndices [line 84]

مجموع سامانه + ردیف هر فیدر.

ServiceIndices.has_result(self) [line 100]
ServiceIndices.as_dict(self) [line 103]
ServiceIndices.to_markdown(self) [line 110]
_is_whole_feeder(device: str) [line 135]
_is_partial(device: str) [line 140]
_fraction_of_event(rec, load_kw: float | None, unit: str) [line 145]

سهم بار قطع‌شده در یک رخداد + منشأ آن (قابل ردیابی).

infer_energy_unit(records, feeder_load_kw: dict | None) [line 168]

واحد «انرژی توزیع‌نشده» را از خود داده استنتاج می‌کند. واحد در منبع اعلام نشده است (A-08). با داشتن بار فیدر، تنها واحدی پذیرفته می‌شود که میانه سهم قطع در بازه (۰, ۱] بیفتد؛ اگر هیچ‌کدام نیفتاد، `kwh` (محافظه‌کارانه: سهم کوچک‌تر) انتخاب می‌شود تا SAIDI بزرگ‌نمایی نشود.

compute_service_indices(records, *, model_feeders=(), feeder_load_kw: dict | None=None, lam: list | None=None, coverage=None, energy_unit: str='auto', customers: dict | None=None) [line 203]

و SAIDI گرفته می‌شود تا اگر داده شود، زمان مواجهه هر فیدر از همان مبنای (r/ ردیف‌های) ``lam`` .برای هر فیدر مدل و کل سامانه SAIDI/SAIFI/CAIDI/ASAI روی یک مخرج باشند.

src/switchloc/reliability/lambdar.py

/r aggregation + the four official review levels of ۲ گام) ۵ شکل). Outputs per TAV122-02/00: * per-feeder (random events / covered years) and r (mean outage duration, zeros included; a zero-duration momentary event is a real reclose) — the values feeding ۶ و ۵ معیارهای جدول ۱۶ in فاز ۴; V-03 anchor Phase-1 binds the [h/yr] 60 / r × = U (جدول ۲۶). * سطح اول — by علت اشکال: events / mean duration / energy. * سطح دوم — by نوع اشکال (grouped cause): same metrics). * سطح سوم — by زمان رخداد: year×month event matrix. * سطح چهارم — by محدوده: feeder × operating-device counts. Only events mapped to modelled feeders (in_model, F1..F7) enter the /

class FeederLambdaR [line 27]

One row of the /r matrix (فیدرها — فاز ۳ /r ماتریس).

FeederLambdaR.u_hours(self) [line 51]

U = r × / 60 [ساعت/سال] — anchor V-03 (جدول ۲۶).

FeederLambdaR.as_dict(self) [line 55]
_mean(values) [line 61]
feeder_lambda_r(records, model_feeders=None, coverage: CoverageResult | None=None) [line 65]

/r for every modelled feeder. Feeders of ``model_feeders`` with *zero* recorded events still get a row (=0, r=0) so the matrix is complete for زمان مواجهه (نیاز ۸): * وقتی — مخرج ranking of ۴ فاز (its zero-inversion policy — V-01 — then applies explicitly, nothing is hidden). جدول ۱۶ — ماه‌های فاقد رکورد) exposure_years ÷ رخداد = :بر مبنای «روزهای واقعی دارای پوشش داده» است ،داده شود (compute_coverage نتیجه) ``coverage`` که نبودشان به معنی نبود خاموشی نیست — از مخرج حذف شده‌اند). * ب

level1_by_cause(records) [line 152]

سطح اول شکل ۵ — بر اساس علت اشکال: رخداد/مدت متوسط/انرژی.

level2_by_fault_group(records) [line 171]

سطح دوم شکل ۵ — بر اساس نوع اشکال (گروه‌های نگاشت رسمی).

level3_by_time(records) [line 197]

سطح سوم شکل ۵ — زمان رخداد: ماتریس سال×ماه + خلاصه سالانه.

level4_by_scope(records) [line 221]

سطح چهارم شکل ۵ — محدوده: فیدر × قطع‌کننده + توزیع طبقات فیدر.

classify_summary(records) [line 239]

Six-class distribution over all real events.

as_dict(records, lam: list[FeederLambdaR], coverage: CoverageResult | None=None) [line 244]

Everything the Phase-3 JSON report needs.

src/switchloc/reliability/load_model.py

:«Load Scaling» نیاز ۱۱ — مدل بار سه‌سطحی با منشأ ردیابی‌پذیر). سطح‌ها (پیشنهاد کاربر، جدول رسمی نیاز + روش چهارم) Three-level load model
سطح منبع بار کاربرد
بار واقعی ترانس یا کنتورهای اندازه‌گیری مطالعات معتبر precise
بار scaled ظرفیت نامی ترانس × ضریب فرضی نبود داده واقعی substitute بار پیک ترانس × ضریب بارگذاری مطالعات مقدماتی estimated
فیدر (P،Q) توزیع بین ترانس‌ها روش کاربر روش Load

class LoadModelSettings [line 71]

تنظیمات مدل بار — قابل تنظیم از GUI، ثبت‌شده در گزارش/پایگاه. ``feeder_pq``: ورودی روش Load Scaling (سطح scaled) — کلید فیدر {"p_kw": بار اکتیو فیدر، "q_kvar": .(در هر اجرا قابل ردیابی و تکرار باشد (منشأ عدد، بند ۱۹ cos و K بار راکتیو فیدر}. مقادیر به شکل خام ذخیره می‌شوند تا محاسبه

LoadModelSettings.__post_init__(self) [line 84]
LoadModelSettings.as_dict(self) [line 103]

LoadModelSettings.from_dict(cls, d: dict) [line 112]

بازسازی از as_dict/JSON — کلیدهای نمایشی مثل level_fa نادیده گرفته می‌شوند تا رفت‌وبرگشت تنظیمات همیشه کار کند.

class LoadModelResult [line 121]

خروجی مدل بار — مقدار + منشأ هر گره (ردیابی کامل).

feeder_key_of(name: str) [line 140]

شناسه فیدر (نام کامل نرمال‌شده) — همان شناسه ردیابی و فهرست فیدرها. قبلا توکن کوتاه («F2») برگردانده می‌شد و دو فیدر هم‌شماره «F2 - MESHKIN» و «F2 - HADIDMOBTAKERAN» کلید یکسانی می‌گرفتند؛ بار و طرح مقیاسشان روی هم می‌افتاد.

scaling_plan(net: CanonicalNetwork, feeder_pq: dict) [line 151]

طرح مقیاس Load Scaling به‌ازای هر فیدر — فرمول‌های کاربر. ورودی: {{feeder_pq = {feeder_key: {"p_kw": P, "q_kvar": Q} خروجی: (plan, warnings) با فهرست warnings: plan[feeder_key] = {"p_kw", "q_kvar", "s_load_kva", "cos_phi", "k_scale", "sum_s_rated_kva", "n_transformers", "n_no_capacity"} (برای فیدر بدون ورودی و فیدری که جمع ظرفیت نامی‌اش صفر است — صریح، هرگز سکوت (بند ۱۹ (code, message_fa).

build_node_loads(graph: NetworkGraph, net: CanonicalNetwork, settings: LoadModelSettings | None=None, issues: IssueLog | None=None) [line 216]

بار هر گره را در سطح انتخابی می‌سازد (جایگزین node_load_map). * precise : بارهای با ``p_kw`` واقعی (data_source اندازه‌گیری)؛ گره بدون بار واقعی = 0 و در «پوشش» شمرده می‌شود. * estimated : ظرفیت ترانس × ضریب بارگذاری (بار پیک تخمینی). * substitute : ظرفیت ترانس × ضریب جایگزین (رفتار قبلی kVA≈kW). * scaled : روش Load Scaling — بار (P،Q) کاربر برای هر فیدر به نسبت ظرفیت نامی بین ترانس‌های همان فیدر توزیع می‌شود (P_Ti = K·S_rated_i·cos، K = S_load/S_rated).

src/switchloc/reliability/loads.py

Load/priority model — بار / مدل ۶ فصل ،۵-۳ (گام ۱۲ جدول). Official table 12 priorities (verbatim, Phase-1 verified): ۱ روستایی · ۲ شهری · ۱.۵ صنعتی · ۴ بار خاص · ۱.۵ کشاورزی · ۸ جدول ،... مراکز درمانی، بیمارستان‌ها per «بند ۹ پرامپت مادر). «بار خاص / A-03 ابهام) configurable + every override is logged via settings_log with the official footnote: «اعداد پیشنهادی است و هر شرکت توزیع می‌تواند وزندهی متفاوتی داشته باشد» (A-05: same category). The GIS sample carries no load-category field — the model therefore ships (a) the audited table, (b) alias resolution for common spellings, (c) explicit assignment + sum per feeder, and (d) a recorded limitat

resolve_category(raw: str) [line 43]

Map a raw category string to a ۱۲ جدول category; '' if unknown.

assign_priorities(net: CanonicalNetwork, issues: IssueLog | None=None, category_overrides: dict[str, str] | None=None, priorities: dict[str, float] | None=None) [line 49]

Fill ``load_pt.priority_weight`` per ۱۲ جدول (or per override). Loads whose category is unknown are left untouched (``data_source='unassigned'``) and counted — never guessed. Returns a summary dict; issues (Information/Warning) describe what was assigned and what remains unassigned.

feeder_priority_sums(net: CanonicalNetwork) [line 112]

Per-feeder load/priority sums (input of criterion 4, ۱۶ جدول). transformer-only nodes without p_kw contribute their transformer capacity as the *proxy* load? — NO (۱۹ بند الزام: no invented data): only loads with a known p_kw count; the gap is visible in ``loads_without_kw``.

src/switchloc/reliability/outages.py

Outage-list ingestion (؛۵ شکل — ۳-۷-۲ گام / بند A-08 data limits). Reads the company outage workbook (دو ۱۴۰۵ لیست خاموشی‌های «پست خلخال» و «پست کمی آباد» شیت into canonical `OutageRecord`s. Data facts established in Phase 0/1 profiling and honoured here: * ~half of the spreadsheet rows are *entirely empty* (895 + 1,103); they are skipped and counted, never classified. * no «تعداد مشترکین متأثر» column exists (A-08) planned/unplanned is NOT guessed from free text beyond a narrow, audited «نوع خاموشی (با برنامه/بی‌برنامه» and no pattern list (OUT-POL-02). * the unit of «انرژی توزیع نشده» is not declared (A-08) kept as-is, reported as.

normalize_header(h) [line 62]

Fold Arabic/Persian letter variants, ZWNJ, NBSP, tatweel and spaces so «نام‌فیدر» == «نام فیدر» == «نام فیدر».

load_column_aliases(path: str | Path | None=None) [line 68]

Extra headerfield aliases from a JSON file (company-specific lists). Looked up in ``path`` or ``$SWITCHLOC_OUTAGE_COLUMNS`` or ``outage_columns.json`` in the current directory; format: ``{"نام فیدر": "feeder_raw", "تاریخ": "date_raw", ...}``. Unknown field names are ignored (with a log line), a broken file never aborts an import.

_column_lookup(extra: dict[str, str] | None=None) [line 100]

class OutageRecord [line 107]

One real outage event, cleaned and feeder-mapped.

OutageRecord.is_random(self) [line 134]

Only classifiable in-scope unplanned events count as random. Unknown/insufficient and out-of-scope records must never inflate .

OutageRecord.as_dict(self) [line 141]

clean_text(v) [line 148]

Collapse whitespace incl. U+00A0; '' for None (the کمی آباد feeder names contain U+00A0 — «کمي آباد1»).

parse_jalali_date(v) [line 156]

'1404/07/01' ('1404/07/01', 1404, 7). Unparseable ('', None, None).

parse_duration_minutes(v) [line 166]

Excel duration minutes. Accepts datetime.time, 'HH:MM:SS', 'HH:MM' or a plain number of minutes. None stays None.
parse_energy(v) [line 190]
default_feeder_mapper(model_feeders: frozenset | set | None=None) [line 202]
Return a (sheet, feeder_raw) (feeder_key, in_model) mapper. Two accepted shapes (both guarded by the modelled-feeder set): * a bare number («1», «2.0») ``F<n>`` — Khalkhal-style lists; * a leading index + free text («4بادآ كمي» — kamiabad-style) ``F<n>``: the conventional leading feeder index of the Tavanir outage lists. Persian/Arabic digits (۴، ٤) are normalized. Anything else stays unmapped with ``in_model=False`` — no silent invention (بند ۱۹); a substation code can never become a fee
fixed_feeder_mapper(feeder_key: str) [line 242]
وقتی کاربر فایل خاموشی «هر فیدر» را جداگانه وارد می‌کند، همه رکوردهای آن .(per-feeder ورود خاموشی) Mapper for a SINGLE-feeder outage workbook فایل به همان فیدر منسوب می‌شوند (صریح، توسط کاربر) — تشخیص عددی «شماره فیدر» دیگر لازم نیست؛ فایل حتی ممکن است ستون فیدر نداشته باشد.
read_outage_workbook(path: str | Path, feeder_mapper=None, source_tag: str='') [line 260]
Read the outage workbook into canonical records. Returns ``(records, reader_stats)``. Entirely-empty spreadsheet rows are skipped and counted (they are not events); rows that carry at least one non-empty cell become records — the classifier decides whether they are usable events or «داده ناکافی» (A-08). ``source_tag``: شناسه منبع برای ورود چندفایلی (خاموشی per-feeder) — در شناسه پایدار رکوردها و آمار خواننده ثبت می‌شود تا رکوردهای دو فایل مختلف هرگز تداخل شناسه نداشته باشند.
_sha256(path: Path) [line 360]

src/switchloc/reliability/sectionalizer.py

نام قدیمی ماژول — به :mod:`switchloc.reliability.switch_placement` منتقل شد. این فایل فقط برای سازگاری با کد/آزمون‌های قبلی نگه داشته می‌شود؛ همه نام‌های عمومی از ماژول جدید بازصادر می‌شوند. کد جدید باید مستقیم از ``switch_placement`` ایمپورت کند.

src/switchloc/reliability/switch_placement.py

روش جایابی بهینه کلید و کاهش EENS — موتور تحلیلی SwitchLoc. این ماژول پیش‌تر «بهینه‌سازی سکشنالایزر» نام داشت؛ اکنون هر سه نوع کلید توزیع را جایابی می‌کند: **ریکلوزر**، **سکشنالایزر** و **سکسیونر**. نوع کلید در ورودی انتخاب می‌شود و دسته خطاهایی که هر نوع پوشش می‌دهد متفاوت است را به («F2 MESHKIN — FAST SECTIONALIZER OPTIMIZATION» PowerFactory) این ماژول **روش** کد .(``SWITCH_TYPE_FAULT_SCOPE``) ساختار داده‌ای SwitchLoc منتقل می‌کند؛ هیچ وابستگی به Qt، PowerFactory یا Matplotlib ندارد و روی گراف/ردیابی/مدل بار خود SwitchLoc کار می‌کند (قابل آزمون با pytest و اجرا از CLI). مدل مهندسی (همان مبنای کد مرجع، مستند و قابل ردیابی — بند ۱۹) ------
class SwitchPlacementSettings [line 134]
تنظیمات موتور — همه اعداد منشأ دارند و در خروجی ثبت می‌شوند.
SwitchPlacementSettings.__post_init__(self) [line 170]
SwitchPlacementSettings.validate(self) [line 173]
SwitchPlacementSettings.as_dict(self) [line 199]
SwitchPlacementSettings.uses_transient_faults(self) [line 230]
SwitchPlacementSettings.effective_restoration_time_min(self) [line 234]
زمان بازیابی بار برای نوع کلید انتخاب‌شده. سکشنالایزر/ریکلوزر خودکار عمل می‌کنند (زمان تنظیمات)؛ سکسیونر دستی است و زمان بازیابی دستی استفاده می‌شود.
SwitchPlacementSettings.from_dict(cls, d: dict | None) [line 245]
class RadialTree [line 255]
درخت پوشای یک فیدر + جمع‌های پایین‌دست (یک‌بار محاسبه).
RadialTree.is_ancestor(self, a: str, b: str) [line 276]
.با بازه اویلر O(1) — باشد ``b`` (جد (یا خود ``a`` اگر True
RadialTree.is_strict_ancestor(self, a: str, b: str) [line 283]
RadialTree.path_to_root(self, node: str) [line 286]
RadialTree.descendants(self, node: str) [line 295]
build_radial_tree(graph: NetworkGraph, trace, node_load_kw: dict) [line 303]
درخت شعاعی یک فیدر را از ردیابی نرمال‌حالت آن می‌سازد. یال‌های اضافی (حلقه/موازی) حذف نمی‌شوند: شناسه و طول‌شان ثبت و طول به گره «جد مشترک» دو سر اضافه می‌شود تا طول قطعه‌ها با طول واقعی فیدر برابر بماند (بند ۱۹ — هیچ طولی گم نمی‌شود).
_candidate_label(feeder_key: str, edge, node_obj, depth: int) [line 419]
برچسب خوانا برای یک کاندید: فیدر · نوع خط · مختصات (در صورت وجود).
class Section [line 434]
یک قطعه شبکه بین مرزهای جداسازی.
Section.as_dict(self) [line 447]
_nearest_selected_ancestor(tree: RadialTree, node: str, selected: set) [line 462]
build_sections(tree: RadialTree, cut_nodes: Iterable[str]) [line 472]
قطعه‌بندی درخت با مجموعه گره‌های برش (کلید موجود + کاندید). قطعه ۰ قطعه ریشه است (همه گره‌هایی که زیر هیچ برشی نیستند)؛ هر گره برش یک قطعه می‌سازد که زیرقطعه‌های تودرتوی خودش را کم می‌کند — همان مدل قطعی کد مرجع، ولی با پشتیبانی از برش‌های تودرتو.
_assign_lambdas(sections: Sequence[Section], failure_rate: float, total_length_m: float) [line 514]
compute_eens_kwh(sections: Sequence[Segment], *, failure_rate: float, total_length_m: float, total_load_kw: float, repair_time_min: float, restoration_time_min: float, restoration_model: str='uniform', backup_kw: float | None=None) [line 522]
.(یعنی ظرفیت پشتیبان نامحدود (فرض کد مرجع ``backup_kw=None`` .سال) برای یک قطعه‌بندی داده‌شده (kWh/سالانه EENS

compute_transient_eens_kwh(tree: RadialTree, sections: Sequence[Section], *, transient_rate: float, total_load_kw: float, transient_time_min: float=1.0, reclose_time_min: float=0.25, reclose_success_probability: float=0.7, recloser_nodes: Iterable[str]=()) [line 571]

دائمی متناسب طول بین قطعه‌ها پخش گذرا همانند_ * :(سال) — فقط برای ریکلوزر. مدل (مستند و قابل ردیابی/kWh) **سالانه ناشی از **خطاهای گذرا EENS می‌شود. * خطای گذرا در نبود ریکلوزر، کل فیدر را ``transient_time_min`` دقیقه بی‌برق می‌کند. * ریکلوزر بالادست خطا، بار پایین‌دست خود را فقط به‌اندازه شکست وصل مجدد) بی‌برق می‌گذارد؛ بار بالادست همچنان کامل قطع می‌شود. ورودی) ``transient·(1-p) + reclose_time_min`` موفق) یا) ``reclose_time_min`` مجموعه گره‌هایی است که ریکلوزر روی آن ``recloser_nodes``

_section_is_descendant(child: Section, parent: Section, sections: Sequence[Section]) [line 619]

class SectionCandidate [line 637]

یک خط/گره قابل نصب کلید.

SectionCandidate.as_dict(self) [line 655]

class OptimizationStep [line 670]

نتیجه نصب k کلید (انتخاب حریصانه).

OptimizationStep.as_dict(self) [line 680]

class FeederStudy [line 693]

نتیجه کامل یک فیدر.

FeederStudy.best_eens_kwh(self) [line 725]

FeederStudy.improvement_percent(self) [line 729]

FeederStudy.as_dict(self) [line 735]

class SwitchPlacementStudy [line 771]

خروجی کامل موتور جایابی کلید برای همه فیدرهای مطالعه.

SwitchPlacementStudy.baseline_eens_kwh(self) [line 782]

SwitchPlacementStudy.best_eens_kwh(self) [line 786]

SwitchPlacementStudy.improvement_percent(self) [line 790]

SwitchPlacementStudy.total_selected(self) [line 797]

SwitchPlacementStudy.feeder(self, key: str) [line 800]

فیدر با شناسه کامل؛ کلید کوتاه («F3») هم اگر یکتا باشد می‌رسد.

SwitchPlacementStudy.summary_rows(self) [line 810]

ردیف‌های جدول خلاصه (هر فیدر) برای GUI/گزارش.

SwitchPlacementStudy.candidate_rows(self) [line 838]

همه کاندیدها (مرتب بر اساس بار پایین‌دست) برای جدول جزئیات.

SwitchPlacementStudy.as_dict(self) [line 859]

SwitchPlacementStudy.study_profile(self) [line 875]

نوع مطالعه انجام‌شده — گزارش دقیقا بر همین اساس ساخته می‌شود. ``study_scope``: single (تک‌فیدره) / multi (چندفیدره)؛ ``method``: all (کلی)؛ ``level``: سطح مدل بار (level-1 = actual / scaled / estimated).

SwitchPlacementStudy.rate_rows(self) [line 913]

نوع EENS) directional | uniform | manual).

SwitchPlacementStudy.formulations(self) [line 931]

مؤثر هر فیدر + منبع هر عدد (بدون عدد پنهان r و).

SwitchPlacementStudy.to_markdown(self) [line 973]

همه فرمول‌بندی موتور برای روش/نوع کلید همین مطالعه (متنی).

class SwitchPlacementEngine [line 1067]

گزارش Markdown — دقیقا منطبق با نوع مطالعه انجام‌شده.

SwitchPlacementEngine.__init__(self, settings: SwitchPlacementSettings | None=None, zone_policy=None) [line 1070]

جست‌وجوی حریصانه محل کلید (ریکلوزر/سکشنالایزر/سکسیونر) برای کمینه‌کردن EENS.

SwitchPlacementEngine.existing_cut_nodes(self, graph: NetworkGraph, net, traces: list) [line 1077]

گره‌هایی که در وضعیت نرمال مرز قطعه‌اند (کلیدهای موجود).

SwitchPlacementEngine.run(self, graph: NetworkGraph, traces: list, node_load_kw: dict, *, net=None, failure_rates: dict | None=None, transient_rates: dict | None=None, repair_times: dict | None=None, failure_rate_source: str='configured', backup_kw: dict | None=None, feeders: Iterable[str] | None=None, zone_of_node: dict | None=None, existing_cut_nodes: dict | None=None, branch_labels: dict | None=None) [line 1093]

اجرای موتور روی همه فیدرهای ردیابی‌شده (یا زیرمجموعه ``feeders``).

SwitchPlacementEngine._assumptions(self, failure_rate_source: str, backup_kw: dict | None) [line 1163]

SwitchPlacementEngine._run_feeder(self, graph: NetworkGraph, tr, key: str, node_load_kw: dict, rate: float, source: str, backup: float | None, existing_cuts: set, zone_of_node: dict, branch_labels: dict, net=None, transient_rate: float=0.0, repair_time: float | None=None) [line 1207]

eens_single_section_kwh(load_kw: float, failure_rate_per_year: float, repair_time_min: float) [line 1414]

پایه فیدر یک‌قطعه‌ای — لنگر تحلیلی کد مرجع EENS.

length_weighted_lambdas(lengths_m: Sequence[float], failure_rate_per_year: float) [line 1421]

هر قطعه متناسب با طول (قطعه‌ها) = فیدر).

theoretical_eens_kwh(load_kw: float, failure_rate_per_year: float, repair_time_min: float) [line 1430]

نظری کد مرجع EENS: (/60)·T_rep·P_total — برای کنترل محاسبه.

src/switchloc/reliability/zone_outages.py

Auditable zone-level U×P from explicitly confirmed event assignments.

class ZoneOutageMetric [line 8]
calculate_zone_up(records: Iterable, event_to_zone: dict[str, str], zone_load_kw: dict[str, float], exposure_years: float) [line 18]
Aggregate confirmed event durations by manually/geographically confirmed zone. Unassigned events are intentionally excluded (not spread across zones). This calculates observed assigned-zone interruption time, not the outage duration of every customer behind a feeder unless event impact is known.

src/switchloc/reporting/__init__.py

reporting — خروجی‌های فاز ۵: Word/PDF/Excel و GIS. The exporters consume ONE neutral content tree (`content.build_report`) built from a `StudyModel`, so Word, PDF and Excel always carry the same numbers (single source of truth — D-04 philosophy applied to reports). - model : StudyModel — network + topology + placement snapshot - content : neutral Persian report content (sections/tables/KPIs) - workbook : openpyxl RTL workbook (فاز ۵) - docx_report : python-docx RTL engineering report + map image - pdf_report : ReportLab PDF (Vazirmatn + arabic_reshaper + python-bidi) - gis_export : GeoJSON (network / switches+proposed / zones) + CSV schedule

src/switchloc/reporting/content.py

content.py — the neutral Persian report content (فاز ۵). `build_report(model)` walks the StudyModel once and produces a `ReportDoc`: ordered sections, each with intro paragraphs and captioned tables. `docx_report` and `pdf_report` only RENDER this tree — they never re-derive a number — so the Word and PDF outputs are identical by construction and any future channel (HTML, …) reuses the same tree. All Persian here is plain text; the DOCX/PDF writers are responsible for RTL direction and shaping.

class Table [line 32]
class Equation [line 43]
One display equation of the «محاسبات کامل» section. ``omml`` — the genuine Word-equation XML (docx renderer); ``text`` — the plain-text form (PDF renderer / copy-paste); both describe the SAME formula so the two outputs never diverge.
class Section [line 57]
class ReportDoc [line 67]
_fmt(v, nd=1) [line 76]
_money(v) [line 84]
build_report(m: StudyModel, map_note: str='') [line 90]
_fmt_cell(value, nd: int) [line 433]
feeder_index_rows(m: StudyModel) [line 443]
(سرستون‌ها، ردیف‌ها) برای جدول همه شاخص‌های ارزیابی فیدرها.
_feeder_indices_section(m: StudyModel) [line 458]
بخش «شاخص‌های ارزیابی فیدرها» — وقتی داده‌ای وجود دارد.
_total_load(m: StudyModel) [line 501]
kW. — (شاخص بار فیدرهای مدل‌شده (جدول ۱۴
_build_manual_section(m: StudyModel) [line 506]
بخش جایابی دستی — نتایج ارزیابی کلیدهای دستی مهندس (مورد ۶).
_worst_zone_label(m: StudyModel, f) [line 568]
بدترین زون به شماره زون (نه شناسه خام) — از درخت‌های strict فاز ۴.
_method_rows(m: StudyModel) [line 580]
_build_calculations_section(m: StudyModel) [line 616]
فرمول‌ها (equation بومی ورد) + مثال عددی واقعی همین اجرا.
_build_eens_report(m: StudyModel, map_note: str='') [line 725]
گزارش Word/PDF بر اساس مطالعه جایابی کلید ذخیره‌شده در پروژه.

src/switchloc/reporting/docx_report.py

docx_report.py — گزارش Word فارسی/RTL (فاز ۵، python-docx). Renders the neutral `ReportDoc` tree into a right-to-left engineering report: A4 title page, RTL headings/paragraphs (`w:bidi`), RTL tables (`w:bidiVisual` — first logical column shows on the right), complex- script font (Vazirmatn `w:cs`) and the network map with the proposed switches embedded as a figure. Word shapes Persian text natively — no reshaping here; the text written into the XML stays logical-order Unicode (searchable, copyable).

_p_bidi(p) [line 34]
_run_fa(run, size: float=11, bold: bool=False, color: RGBColor | None=None) [line 42]
Persian run: complex-script font + size + rtl.
_fa_para(doc, text: str, size: float=11, bold: bool=False, align=WD_ALIGN_PARAGRAPH.JUSTIFY, color=None, space_after: float=6) [line 74]
_tbl_bidi(table) [line 85]
append_equation(doc, omml_xml: str, caption: str='', note: str='') [line 91]

Inject a genuine Word equation (OMML) — «ورد استفاده کن» (درخواست کاربر equation از قابلیت). The paragraph holds the caption run followed by the `<m:oMathPara>` element itself, so the formula is a REAL Word equation: double-click in Word opens the equation editor; it is neither an image nor plain text.

_cell_fa(cell, text: str, bold: bool=False, size: float=9) [line 112]
export_docx(doc_content: ReportDoc, path: str, map_png: str | None=None, charts_png: str | None=None) [line 121]

Write the Word report and return the path.

src/switchloc/reporting/eens_report.py

گزارش نهایی نرم‌افزار باید **دقیقا** با نوع مطالعه‌ای که انجام شده بخواند. «EENS / بخش‌های گزارش نهایی برای «مطالعه جایابی کلید — eens_report.py (درخواست کاربر v1.15). این ماژول همان مطالعه ذخیره‌شده در پروژه (``project_kv['sectionalizer_study']``) را به بخش‌های آماده‌برای رندر تبدیل می‌کند: نوع مطالعه (تک‌فیدره / چندفیدره / کلی)، نوع EENS (دایرکشنال / یکنواخت / دستی)، سطح مدل بار، و r مؤثر هر فیدر با منبع هر عدد، نتیجه فیدرها، کلیدهای پیشنهادی، گام‌های بهینه‌سازی و **فرمول‌بندی کامل موتور**. هیچ بخشی که در این مطالعه وجود ندارد ساخته نمی‌شود (بدون آیتم اضافی) و هیچ عددی بازمحاسبه نمی‌شود؛ همه از خروجی موتور می‌آید.

_num(v, nd=1) [line 45]
eens_profile(study: dict | None) [line 54]

خلاصه «نوع مطالعه» از payload ذخیره‌شده موتور.

is_eens_study(study: dict | None) [line 100]

آیا این payload یک مطالعه EENS معتبر است؟

_rate_rows(prof: dict) [line 109]
_result_rows(prof: dict) [line 122]
_switch_rows(prof: dict) [line 142]
_step_rows(prof: dict) [line 162]
build_eens_sections(study: dict, map_note: str='') [line 177]

(پروفایل مطالعه، فهرست بخش‌های گزارش) — بدون بخش اضافی.

src/switchloc/reporting/equations.py

equations.py — هر فرمول به equation درخواست کاربر). «برای نوشتن فرمول‌ها از قابلیت) Word فرمول‌های محاسبات کامل در گزارش ورد استفاده کن» صورت OMML بومی Word ساخته می‌شود (مثل Insert Equation خود ورد: کسر واری genuine، رادیکال genuine، توان genuine) — نه متن ساده و نه تصویر. هر builder یک رشته XML معتبر `<m:oMathPara>` برمی‌گرداند؛ `docx_report.append_equation()` آن را داخل یک پاراگراف تزریق می‌کند. ساخت: رشته‌های XML ایستا (deterministic) — بدون وابستگی جدید؛ اجرای تست‌ها XML را با ElementTree پارس می‌کند تا معتبربودن تضمین شود.

_r(t: str) [line 24]

A plain math run.

_frac(num: str, den: str) [line 29]

Genuine Word fraction.

_sqrt(e: str) [line 34]

Genuine square root.

_sup(base: str, sup: str) [line 40]

Genuine superscript.

_sub(base: str, sub: str) [line 45]

Genuine subscript.

omml(inner: str) [line 50]

Full `<m:oMathPara>` element (one display equation).

eq_load_scaling_s_load() [line 58]

S_load = √(P² + Q²)

eq_load_scaling_cos_phi() [line 66]

= arctan(Q / P) ; cos = P / S_load

eq_load_scaling_p_rated() [line 73]

P_rated,i = S_rated,i × cos

eq_load_scaling_k() [line 79]

K = S_load / S_rated

eq_load_scaling_p_t() [line 85]

P_T,i = K × S_rated,i × cos

eq_lambda_uniform() [line 91]

_i = _feeder × L_i / L

eq_u_hours() [line 99]

U = × r / 60 [h/yr]

eq_ens() [line 104]

ENS = _i (_i / 60) · (r·P_shed,i + s·P_restored,i) [kWh/yr]

eq_ep_tl() [line 113]

EP = 100·max_i(P_shed,i)/P ; TL = 100·max_i(P_restored,i)/P

eq_rank_score() [line 123]

Score = _k w_k × x_k , w = (1,2,2,3,1,1) (فرمول ۱ / جدول ۱۶)

eq_ce_cn() [line 131]

CE/CN انتخاب طرح: بیشینه ; CE = 1000 × (E2 - E1)

eq_delta_ens_percent() [line 140]

ENS% = 100 × (E2 - E1) / E2

eq_lambda_segment() [line 205]

_i = _feeder × L_i / L

eq_eens_uniform() [line 211]

EENS = _i (_i/60)·[r·P_i + s·min(P_ok,i, C) + r·(P_ok,i - min(…))]

eq_eens_directional() [line 222]

EENS = _i (_i/60)·[r·P_i + s·min(P_down,i, C) + r·(P_down,i - min(…))]

eq_restoration_clamp() [line 233]

s = min(t_restore, r)

eq_eens_transient() [line 239]

EENS_tr = _i (_tr,i/60)·[(P-P_prot)·t_tr + P_prot·(t_rec+(1-p)·t_tr)]

eq_eens_gain() [line 250]

EENS% = 100·(EENS_base - EENS_best)/EENS_base

eq_outage_lambda_r() [line 258]

= N_random / سال‌های پوشش ; r = t_restore / N

_alleens() [line 266]

src/switchloc/reporting/figures.py

تصاویر نموداری گزارش (Word/PDF) — همه شاخص‌های ارزیابی فیدرها. برچسب فیدرها لاتین است (مثل «F1 - MESHKIN») و عنوان‌ها انگلیسی/کوتاه، چون Excel و در (QPainter) برعکس و جدا رسم می‌کند. همین نمودارها در داشبورد به‌صورت تعاملی و فارسی reshaper/bidi متن فارسی را بدون Matplotlib به‌صورت نمودار بومی Excel هم موجودند.

indices_figure(rows: list, path: str, title: str='Feeder indices — indicators of the study') [line 27]

شکل ۲×۳ از شاخص‌های فیدرها؛ اگر داده نبود ``None``.

src/switchloc/reporting/gis_export.py

gis_export.py — خروجی GIS فاز ۵ (GeoJSON + CSV، بدون وابستگی جدید). Four field/GIS-ready artefacts in one directory:
switchloc_network.geojson — feeder lines (canonical branch polylines, attributes: feeder, conductor, length, traced feeder, zone number)
switchloc_switches.geojson — points: existing switchgear + proposed Phase-4 switches + feeder-head sources
switchloc_zones.geojson — per-zone MultiLineString with load proxy and boundary switches (جدول ۱۵ geometry) switch_schedule.csv — the actionable crew sheet (Excel-friendly UTF-8-BOM): analysis-CRS metres + WGS84 GeoJSON follows RFC 7946: geometries are transformed to WGS84 lon/lat with

class _Wgs84 [line 37]

Cached per-model WGS84 transformer (None passthrough).

_Wgs84.__init__(self, m: StudyModel) [line 40]
_Wgs84.pt(self, xy) [line 56]
_Wgs84.line(self, coords) [line 62]
_feat(geom_type, coords, props: dict) [line 66]
export_gis(m: StudyModel, out_dir: str) [line 72]

Write the four artefacts; return their paths.

_edge_id_of_branch(m: StudyModel, branch_id: str) [line 225]

graph edge ids belonging to one canonical branch.

_dump(path: Path, payload: dict) [line 234]

src/switchloc/reporting/model.py

StudyModel — the read-only data bundle behind every Phase-5 export. Built from the three in-memory artefacts the pipeline already produces (`CanonicalNetwork`, the TopologyService result, the PlacementResult) — no exporter talks to SQLite or the engine directly, so a report can be regenerated offline from any completed project (or from a live run the UI just made — same class either way). Coordinates: the analysis CRS is whatever the Phase-1 import recorded (`net.crs_definitions[0].epsg`, UTM for the sample). Field deliverables are additionally offered in WGS84 (GeoJSON convention) through the lazy `to_wgs84()` helper — pyproj is already a co

class ProposedSwitch [line 27]

One actionable «کلید پیشنهادی» row (کاربرگ اجرایی).

ProposedSwitch.kind_fa(self) [line 43]

class StudyModel [line 51]
StudyModel.has_eens_study(self) [line 83]
آیا مطالعه EENS ذخیره‌شده‌ای برای گزارش وجود دارد؟
StudyModel.active_study(self) [line 89]
نوع مطالعه گزارش: ``eens`` یا ``placement`` (دقیقا یکی).
StudyModel.name(self) [line 99]
StudyModel.epsg(self) [line 103]
StudyModel.crs_label(self) [line 110]
StudyModel.node_coord(self, node_id: str) [line 119]
StudyModel.branch_polylines(self) [line 123]
[{branch_id, kind, feeder, conductor, length_m, lines:[[(x,y)…]]}] from the canonical branches (full polylines, not straight chords).
StudyModel.feeder_of_edge(self) [line 142]
edge_id feeder key of the traced winner (map renderer rule).
StudyModel.zone_edges(self, zone) [line 150]
StudyModel.zone_rows(self) [line 165]
Detailed, auditable zone register for reports and workbooks.
StudyModel.build_proposed(self) [line 190]
Flatten the recommendation into coordinate-annotated rows. Zone numbers come from the strict n-1 trees (the same trees the engine ranked); coordinates from the snapped graph nodes.
StudyModel.to_wgs84(self, x: float, y: float) [line 234]
Analysis-CRS metres (lon, lat) WGS84; None when unknown.
StudyModel.feeder_table(self) [line 254]
یک ردیف برای هر فیدر با همه شاخص‌های ارزیابی‌شده. منابع، همه از همان اجراهای ثبت‌شده پروژه (هیچ عددی ساخته نمی‌شود): * ردیابی: طول (km)، تعداد ترانس، تعداد مشترک/بار * ماتریس ، :r/ قابل کاهش، r، U * شاخص‌های پیوستگی: SAIDI، SAIFI، CAIDI، ASAI (None = بدون داده) * اقتصاد: ENS پایه و ENS پس از طرح‌های اصلی
StudyModel.kpis(self) [line 326]
StudyModel.issues_all(self) [line 367]
StudyModel.verdict(self) [line 372]
_scheme_feeders(placement, scheme_id: str) [line 377]

src/switchloc/reporting/pdf_report.py

pdf_report.py — گزارش PDF ۵ فاز) فارسی، ReportLab). Same `ReportDoc` tree as the Word report, rendered offline to PDF with true Persian shaping: arabic_reshaper (contextual forms) + python-bidi (visual reordering) + the bundled Vazirmatn TTF. Shaping is applied at the *paragraph* level; tables keep Western digits so numeric columns stay machine-readable. Dependencies are optional at runtime (`switchloc[report-pdf]`): when arabic_reshaper / python-bidi / reportlab / the font are missing the exporter raises `PdfUnavailableError` with a clear Persian message and the caller falls back to DOCX (Word prints to PDF natively). Fonts are looked up in
class PdfUnavailableError [line 27]
PDF extras missing (reportlab / arabic_reshaper / bidi / font).
_candidate_font_dirs() [line 31]
find_font(bold: bool=False) [line 42]
export_pdf(doc_content: ReportDoc, path: str, map_png: str | None=None, charts_png: str | None=None) [line 51]
Write the Persian PDF and return the path (raises PdfUnavailableError when the optional extras are not installed).
_is_wide_table(n_columns: int) [line 264]
جدول‌های پهن (۹ ستون و بیشتر) باید روی صفحه افقی بروند.
_col_widths(n: int, headers: list, fractions: list | None=None, usable: float=523.0) [line 269]
Usable width (523pt portrait / 770pt landscape) — explicit fractions when given, else distribute by header length with sane caps.

src/switchloc/reporting/service.py

ReportService — orchestration of the Phase-5 exports (فاز ۵). Two entry shapes, one code path: * **From a finished project** (`run(db_path=…)`): the placement pipeline is re-executed in-memory with ``persist=False`` (the engine is pure — same inputs identical numbers, guaranteed by the deterministic-id tests), then cross-checked against the latest persisted placement run of the project. The cross-check result is printed into the report itself — a report that disagrees with the database is a FAIL, never silent (۱۹ بند philosophy). * **From a live run** (`run(net=…, topo=…, placement=…)`): the Qt shell and the CLI placement command already ho
class ExportSettings [line 42]
class ExportResult [line 63]
ExportResult.summary_fa(self) [line 70]
class ReportService [line 86]
ReportService.__init__(self, settings: ExportSettings | None=None) [line 87]
ReportService.run(self, db_path: str | None=None, *, net=None, topo=None, placement=None, placement_run_id: str='') [line 91]

ReportService._load_outage_data(db_path: str | None, issues: list) [line 178]

ماتریس r/ و شاخص‌های پیوستگی از آخرین اجرای خاموشی پروژه.

ReportService._load_eens_study(db_path: str | None, issues: list) [line 205]

مطالعه جایابی کلید/EENS ذخیره‌شده در پروژه (یا dict خالی).

ReportService._rerun(self, db_path: str, issues: list) [line 228]

Fresh in-memory pipeline (persist=False) + persisted run id.

ReportService._crosscheck(db_path: str, placement) [line 252]

Latest persisted placement vs this fresh result.

ReportService._render_map(self, m: StudyModel, out_dir: str) [line 335]
_out(out_dir: str, name: str) [line 361]
_net_of(res, db_path: str) [line 368]

The CanonicalNetwork behind a PlacementRunResult (reloaded once).

src/switchloc/reporting/workbook.py

workbook.py — کتاب کار Excel ۵ فاز) مطالعه خروجی، openpyxl). One RTL workbook, one sheet per official artefact — the numbers are REBUILT FROM THE SAME StudyModel as the Word/PDF report, never copied: خلاصه — KPIs + run identity + cross-check verdict جدول ۱۳-۱۵ — connections, EP/TL, per-zone n-1 rows رتبه‌بندی — full audit matrix: raw / normalized / weighted / score ۱۷ جدول — طرح‌ها schemes ۱۸ جدول — CE/CN comparison کلیدها — actionable proposed-switch schedule (with coordinates) /r — the reliability matrix backing criteria 5/6 Sheets are rightToLeft so the workbook reads naturally in Excel; header styling is uniform; number

_sheet(wb: Workbook, title: str) [line 37]
_put_title(ws, row: int, text: str) [line 43]
_put_table(ws, row: int, headers: list, rows: list) [line 49]
_auto_width(ws, widths: list | None=None) [line 70]
write_feeder_indices_sheet(wb, m: StudyModel) [line 83]

برگه «شاخص‌های فیدرها» + نمودارهای بومی Excel (درخواست کاربر). ستون‌ها از همان `FEEDER_INDEX_COLUMNS` گزارش می‌آیند تا Word/PDF و Excel یک عدد نشان دهند. هیچ عددی ساخته نمی‌شود؛ مقدار نامعلوم خالی می‌ماند («—») تا با صفر اشتباه نشود.

export_workbook(m: StudyModel, path: str) [line 170]

Write the Phase-5 XLSX and return the path.

export_dashboard_feeder_indices(dashboard: dict, path: str) [line 329]

Export the displayed feeder metrics even when placement has not run.

src/switchloc/scenarios/__init__.py

scenarios — فاز ۲ (پایه سناریوهای بهره‌برداری و کلیدزنی). ARCHITECTURE.md §۵ lists `scenarios(پایه)` in Phase 2: the *model* that Phase 4 (طرح‌های کلیدگذاری جدول ۱۳-۱۸, EP/TL) and Phase 3 (خاموشی) will build upon. Scope kept deliberately small and fully deterministic: * `ScenarioState` — effective open/closed decision per switch, derived from the GIS service state + confirmed NO states + the auditable unknown-state policy (TOPO-POL-01); * `SwitchScenario` — an ordered, attributed list of switching actions (open/close) on top of the base state — every action carries user, reason and timestamp so each configuration is auditable (بند ۱۹); * `bas

src/switchloc/scenarios/base.py

Scenario model — فاز ۲ (وضعیت کلیدها، مانور و اثر آن بر برق‌داری). State semantics (documented, no hidden assumptions): * a switch is OPEN when: confirmed NO, or OUT_OF_SERVICE (TOPO-POL-01: physically out of service), or an explicit scenario action opened it; * a switch is CLOSED when: IN_SERVICE (and not NO) — or UNKNOWN while `unknown_switch_pass` holds (the same policy as tracing, so the base scenario is *consistent* with the normal-state map); * a scenario action may only reference a switch that exists; actions are applied in order and recorded verbatim.

class ScenarioAction [line 28]
ScenarioAction.as_dict(self) [line 35]
class ScenarioState [line 40]

Effective open/closed decision per switch id.

ScenarioState.__init__(self, open_switches: set[str]) [line 43]
ScenarioState.is_open(self, switch_id: str) [line 46]
ScenarioState.is_closed(self, switch_id: str) [line 49]
ScenarioState.as_dict(self) [line 52]
base_state(net: CanonicalNetwork, policy: TracingPolicy | None=None) [line 56]

Normal configuration straight from the canonical model (بند ۷-۷-۵).

class SwitchScenario [line 73]

One named switching configuration (base + ordered actions).

SwitchScenario.__init__(self, name: str, base: ScenarioState, actions: list[ScenarioAction] | None=None, note: str='') [line 76]
SwitchScenario.add_action(self, switch_id: str, action: str, reason_fa: str='', user: str='') [line 86]

SwitchScenario.effective_state(self) [line 97]

Base state with actions applied in order (deterministic).

SwitchScenario.as_dict(self) [line 109]
class ConnectivityResult [line 119]

Outcome of evaluating one scenario against the graph.

ConnectivityResult.summary(self) [line 129]
evaluate(graph: NetworkGraph, net: CanonicalNetwork, scenario: SwitchScenario, zone_of_edge: dict | None=None, issues: IssueLog | None=None) [line 142]

Energized/de-energized computation for one scenario (deterministic). Traversal rule mirrors `topology.tracing` with the scenario's effective states substituted for the GIS states, so the normal map and any what-if stay consistent by construction.

isolation_scenario(graph: NetworkGraph, net: CanonicalNetwork, zone_id: str, zone_of_edge: dict, boundary_of_switch: dict, name: str, user: str='') [line 206]

Fault-isolation switching for one zone (جدول ۱۵ base operation). Proposes opening every CLOSED boundary switch of the zone — the classic «باز کردن کلیدهای دو طرف زون معیوب». Returned as a scenario the engineer reviews; nothing is auto-applied to the GIS.

src/switchloc/topology/__init__.py

topology — موتور توپولوژی SwitchLoc (فاز ۲، ARCHITECTURE.md §۵). Modules: * graph : CanonicalNetwork traversable NetworkGraph (edges split at device vertices so a mid-line switch can actually interrupt the line — the GeoGrid splice trick, re-implemented; audit §۳-۴ / test TG-03); * tracing : normal-state feeder reconstruction from SOURCE nodes with passability rules (TOPO-POL-01/02) and GIS-label mismatch detection; * ties : A-09 — NO/Tie candidate detection (proposed rule R1/R2) + explicit user confirmation contract; * zones : زون‌بندی per TAV122-02/00 تعریف ۵-۲۱ — components bounded by switches (split-node component labelling), Zone0..n n

src/switchloc/topology/graph.py

NetworkGraph — CanonicalNetwork traversable graph (Phase 2). Why a graph pass is needed at all ---------- The Phase-1 canonical model stores each GIS polyline as ONE `Branch` between its two endpoint nodes, with the interior vertices kept only as `LineSegment` geometry. A point device (switch/fuse/transformer) that sits on a *mid-line vertex* therefore lives on a node that is not an endpoint of any branch — exactly the situation GeoGrid solved with its «splice» trick (equipment on a mid-vertex connects because the line is conceptually cut at every vertex; audit §۷, test GG-05). `NetworkGraph.from_network` performs tha

class GraphNode [line 53]
class GraphEdge [line 62]
GraphEdge.other(self, node: str) [line 74]
class _GridIndex [line 78]

Uniform-grid nearest-neighbour index (pure Python, O(1) lookup).

_GridIndex.__init__(self, cell: float) [line 81]
_GridIndex.add(self, node_id: str, pt: tuple) [line 86]
_GridIndex.nearest(self, pt: tuple, tol: float, kind_rank: dict | None=None) [line 91]

(node_id, distance) of the nearest point within `tol`. Deterministic tie-break: smaller distance first, then LOWER kind rank (JUNCTION beats DEVICE_HV beats SOURCE — line vertices must attach to junction nodes when several canonical nodes share the same coordinates), then smaller node id. Returns None when nothing lies within `tol`.

class _NodeMeta [line 119]
class NetworkGraph [line 126]

Adjacency multi-graph with device attachments (Phase 2 engine).

NetworkGraph.__init__(self) [line 129]
NetworkGraph.from_network(cls, net: CanonicalNetwork, issues: IssueLog | None=None, match_tol_m: float=DEFAULT_MATCH_TOL_M, source_attach_tol_m: float=DEFAULT_SOURCE_ATTACH_TOL_M) [line 147]
NetworkGraph._add_edge(self, br, dedup_path) [line 331]
NetworkGraph.neighbors(self, node: str) [line 351]
NetworkGraph.degree(self, node: str) [line 354]
NetworkGraph.edge(self, edge_id: str) [line 357]
NetworkGraph.is_source(self, node: str) [line 360]
NetworkGraph.has_switch(self, node: str) [line 363]

src/switchloc/topology/ties.py

NO / Tie detection — ابهام رسمی A-09 (فاز ۲). TAV122-02/00 knows two states for a sectioning key: NC (بسته در وضعیت عادی) and NO (باز در وضعیت عادی). The Khalkhal GIS export carries **no NO/NC field** — the Phase-0 audit registered this as ambiguity A-09 and decided: a *proposed rule* + explicit engineer confirmation, never a silent guess (main prompt §۵-۷: هر فرض باید ثبت و نمایش داده شود). Proposed rules (documented, deterministic): R1 — «هم‌گرایی دو فیدر»: two feeder traces reach the same node in the normal state an open point is missing somewhere at that node. With TOPO-POL-01 (unknown switches pass) whole feeders can flood into each ot

class TieDetectionPolicy [line 48]
class TieCandidate [line 54]

One proposed NO point awaiting the engineer's decision.

TieCandidate.as_dict(self) [line 67]
class TieDetectionService [line 77]

Rule proposal + explicit confirmation (A-09).

TieDetectionService.__init__(self, graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], tracing: TracingEngine | None=None, policy: TieDetectionPolicy | None=None) [line 80]
TieDetectionService.detect(self, issues: IssueLog) [line 94]
TieDetectionService.confirm(self, cand_id: str, user: str, note: str='', issues: IssueLog | None=None) [line 227]
TieDetectionService.reject(self, cand_id: str, user: str, note: str='', issues: IssueLog | None=None) [line 252]
TieDetectionService.to_tie_entities(self) [line 271]

TieConnection rows for persistence (status kept verbatim).

TieDetectionService.verified_after_confirmation(self) [line 282]

Re-trace with the confirmed NO states no overlaps left? Uses the same overlap rule as `TracingEngine.trace_all`: only passable nodes count (opposite sides of the newly-opened switch legitimately share its terminal node).

TieDetectionService._by_id(self, cid: str) [line 302]

src/switchloc/topology/tracing.py

+ تعریف ۵-۵/۱۵-۱۹ Method (docs/TAV122_TRACEABILITY.md).(بازسازی فیدرها در وضعیت عادی شبکه (فاز ۲ — Feeder tracing ARCHITECTURE.md §۵ فاز ۲): * seeds : SOURCE nodes attached to the graph (`NetworkGraph.source_of`). Fallback when none attach: BREAKER-family switch nodes grouped by their GIS feeder label (TOPO-POL-02b). * rule : normal-state breadth-first traversal. Traversal may ENTER any node and ENERGIZE it, but may PASS THROUGH a node only when every switch sitting on it is «passable» (TOPO-POL-01): - normally_state == "NO" open in normal state blocks; - current_state == OUT_OF_SERVICE physically out of service blocks; - current_stat

feeder_id(name: str) [line 54]

هویت کامل فیدر — نام نرمال‌شده کامل (بدون کوتاه‌سازی). «F2 - MESHKIN» و «F2 - HADIDMOBTAKERAN» دو فیدر جدا هستند ولی هر دو با «F1» با «F1» تبدیل می‌شدند و در فهرست فیدرها یکی می‌شدند. این تابع نام کامل را نگه می‌دارد؛ یکی‌دانستن نگارش کوتاه و آراسته (مثل «F2» به `func:`feeder_key` است. کار (`func:`same_feeder`: «F2 - MESHKIN.

same_feeder(a: str, b: str) [line 69]

آیا دو برچسب به یک فیدر اشاره می‌کنند؟ * نگارش‌های نرمال‌شده یکسان بله؛ * اگر هر دو فقط «توکن کوتاه» یکسانی دارند (F2 و F2) بله؛ * اگر یکی از دو نام **تنها** همان توکن کوتاه باشد («F1» در برابر «F1 - MESHKIN») بله (زنجیره فالبک GIS در لایه‌های مختلف کوتاه و آراسته را مخلوط می‌کند)؛ * «F2 - F2» در برابر «F2 - HADIDMOBTAKERAN» «F2 - MESHKIN» (دو فیدر** خیر).

resolve_feeder(name: str, candidates) [line 90]

نگاشت یک نام/کلید بیرونی به شناسه یکی از ``candidates``. ترتیب: تطبیق دقیق (پس از نرمال‌سازی) تطبیق `func:`same_feeder` وقتی **یکتا** باشد. اگر چند کاندید تطبیق دهند (مثل «F2» وقتی هم «F2 - MESHKIN» و هم «F2 - HADIDMOBTAKERAN» وجود دارد) یا هیچ‌کدام، همان نام نرمال‌شده برگردانده می‌شود — بدون حدس و بدون ادغام دو فیدر متفاوت.

match_feeders(name: str, candidates) [line 111]

همه کاندیدهایی که با یک برچسب یکی‌اند (برای انتخاب‌های ذخیره‌شده). مثل `func:`resolve_feeder` ولی **فهرست** برمی‌گرداند: اگر برچسب کوتاه («F2») به دو فیدر بخورد، هر دو را می‌دهد (نمایش هر دو بهتر از نمایش هیچ است)؛ اگر هیچ تطبیقی نبود، فهرست خالی. برای فیلترهای نمایشی (لایه فیدرهای نقشه، رنگ فیدر، دامنه گزارش) که مقادیر ذخیره‌شده نسخه‌های قبل با کلید کوتاه نوشته شده‌اند.

feeder_key(name: str) [line 132]

Canonical short feeder identity: leading token, RTL-normalized. «F1 - KHALKHAL», «f1», «F1» «F1» — the GIS fallback chain mixes short and decorated names for the same feeder, so mismatch detection must compare identities, not decorations. توجه: این «توکن کوتاه» برای مقایسه/نمایش کوتاه است، نه شناسه یکتا؛ شناسه فیدر `func:`feeder_id` است.

class TracingPolicy [line 152]

Explicit, auditable traversal policies (nothing hidden).

class FeederTrace [line 164]
FeederTrace.as_dict(self) [line 179]
class TracingEngine [line 194]

Normal-state feeder reconstruction (deterministic BFS).

TracingEngine.__init__(self, graph: NetworkGraph, net: CanonicalNetwork, policy: TracingPolicy | None=None) [line 197]
TracingEngine.switch_passable(self, sw: SwitchDev) [line 205]
TracingEngine.node_passable(self, node: str) [line 214]
TracingEngine.seeds(self) [line 222]

قطعی و بدون تکرار. یک گره می‌تواند سرچند فیدر باشد (شینه پست)؛ هر فیدر یک بذر می‌گیرد. قبلا نقشه «گره — [(feeder_name, feeder_id, junction_node)] یک نام» فیدرهای بعدی را روی قبلی بازنویسی می‌کرد و آن فیدرها از فهرست/توپولوژی کامل حذف می‌شدند.

TracingEngine.fallback_seeds(self) [line 247]

TOPO-POL-02b: no usable SOURCE nodes seed at breaker nodes.

TracingEngine.trace_from(self, feeder_name: str, key: str, start: str) [line 263]
TracingEngine.trace_all(self, issues: IssueLog, quiet: bool=False) [line 295]
Trace every seed; `quiet=True` returns traces without Issues (used by the strict-core diagnostic so it cannot pollute QC).

src/switchloc/topology/zones.py

زون: آن قسمت شبکه» :(verified in Phase 1 — تعریف ۵-۲۱ ,Official definition (TAV122-02/00. (زون‌بندی شبکه فشار متوسط (فاز ۲ — Zone building فشار متوسط به‌همراه تمام تجهیزات و بارهای آن که محدود به یک یا چند کلید می‌شود.» Algorithm — per-feeder split-node component labelling (deterministic): 1. a node carrying a *boundary device* is declared a «cut node»; 2. for EVERY feeder trace, the feeder's own traced edges are decomposed into connected components where traversal through cut nodes is forbidden — each component becomes one zone OF THAT FEEDER, numbered Zone0 (the component holding the source) upward by breadth-first walk across switch bounda
class ZonePolicy [line 55]
class SwitchZone [line 64]
SwitchZone.as_dict(self) [line 77]
class BoundaryRecord [line 90]
class ZoneResult [line 99]
ZoneResult.zones_of_feeder(self, feeder: str) [line 106]
ZoneResult.zone_by_id(self, zone_id: str) [line 112]
ZoneResult.as_dict(self) [line 118]
class ZoneBuilder [line 131]
Per-feeder split-node labelling (deterministic, issue-annotated).
ZoneBuilder.__init__(self, graph: NetworkGraph, net: CanonicalNetwork, traces: list[FeederTrace], policy: ZonePolicy | None=None) [line 134]
ZoneBuilder.is_boundary_device(self, sw: SwitchDev) [line 144]
ZoneBuilder.node_is_cut(self, node: str) [line 158]
ZoneBuilder._components(self, edges: set[str]) [line 173]
ZoneBuilder._materialize(graph: NetworkGraph, comp_edges: list[str], feeder: str, zone_no: int, kind: str, source_node: str | None) [line 197]
ZoneBuilder.build(self, issues: IssueLog) [line 224]

src/switchloc/ui/__init__.py

قابل آزمون در خط :(Agg/PNG، خالص Matplotlib) Qt** نمایشگر نقشه شبکه **بدون وابستگی — `map_renderer` * .(D-04 نمایش (فاز ۲، تصمیم — ui لوله مهندسی و قابل استفاده در گزارش‌ها. برچسب‌های نقشه با کدهای فنی (SW/BR ،Z0.. ،F1.../....) هستند؛ چرا؟ زیرا متن فارسی در Matplotlib بدون و در MD/JSON و فونت عربی‌پشتیبان درست رندر نمی‌شود و بسته نصب آفلاین را سنگین می‌کند. متن فارسی کامل در گزارش‌های arabic_reshaper/bidi پوسته PySide6 (رندر RTL بومی Qt) آمده است. * `/qt` — پوسته PySide6 (ویزارد نگاشت + نقشه + توپولوژی)؛ وابستگی اختیاری (`requirements-qt.txt`)؛ منطق مهندسی ندارد و فقط از سرویس‌های application/ استفاده می‌کند (§۱ ARCHITECTURE.md — قاعده ۱).

src/switchloc/ui/dashboard_data.py

Dashboard data assembly — Qt-free (the management view of a study). `build_dashboard()` merges, per feeder, the normal-state trace (nodes / edges / km / transformers), the outage /r matrix (Phase 3), the n-1 EP/TL results, the formula-(1) ranking (table 16), the selected scheme per feeder and the table-18 CE/CN comparison — one row per feeder, ready for the Qt dashboard tab or any other presenter. Discipline: the engineering logic lives here and is testable without Qt; the Qt tab (`ui/qt/dashboard_tab.py`) only renders the dict it receives (same split as map_canvas.py).
gis_transformer_map(net: CanonicalNetwork | None) [line 20]
ترانس‌ها به تفکیک فیدر بر اساس «برچسب T_FeederNa) «GIS). این «ترکیب رسمی فیدر» است که شرکت توزیع می‌شناسد (جدول کاربر) و ممکن است با تعداد ردیابی‌شده (مسیر تغذیه واقعی) متفاوت باشد — اختلاف این دو، ایراد داده GIS است (مثلا ترانس‌های F6 با برچسب F5) و باید صریح گزارش شود، نه پنهان (بند ۱۹). خروجی: {"feeder_key": تعداد, "s_rated_kva": جمع ظرفیت نامی, "blank_label": تعداد بدون برچسب, "no_capacity": تعداد بدون ظرفیت}
zone_load_map(topo, net, load_model=None, placement=None) [line 46]
Exact per-zone connected load (kW) on the same model as N-1. After a placement run the persisted N-1 zone allocation is authoritative. Before placement, reproduce its transformer allocation from the current load-model settings and assign each transformer to one deterministic owning zone (lowest zone_no at a shared boundary).
_length_label(length_m: float) [line 106]
Engineering-friendly zone length: metres below 1 km, km otherwise.
zone_rows(topo, net=None, placement=None, load_model=None) [line 112]
Build explicit, project-scoped zone records for the dashboard. When placement exists, its strict n-1 zone tree and load allocation are authoritative. Otherwise use the current topology zones and the same configured load model used by placement.
_outage_lambda_rows(outage) [line 162]
OutageResult **or** a persisted-outage dict /r rows as dicts.
outage_coverage(outage) [line 178]
پوشش داده (نیاز ۸) از OutageResult یا حافظه پایدار — JSON-safe.
capacity_rows_map(capacity_rows) [line 195]

GUI/store capacity rows feeder_key dict (نیاز ۹).

plan_status_map(placement) [line 217]

placement feeder_key plan-status dict (نیاز ۱۰، بدون اجبار).

reconciliation_rows(rows: list[dict], gis_tr: dict | None) [line 238]

جدول آشتی تعداد ترانس هر فیدر: برچسب GIS در برابر مسیر تغذیه. هر اختلاف = یا داده ناقص GIS (برچسب اشتباه/خالی، مثال واقعی: ترانس‌های F6 با برچسب (در خلخال) یا قطعه‌ای که با وضعیت کلیدها از منبع فیدر جدا افتاده (پشت کلید باز). هیچ‌کدام حدس زده نمی‌شود؛ فقط شمرده و گزارش می‌شوند (بند ۱۹ F5).

_index_by(rows: list[dict], key: str='feeder_key') [line 270]

_same(name, other) [line 274]

همان‌دانستن نگارش کوتاه/آراسته فیدر («F3» با «F3 - KOSAR»).

_in_scope(name, selected) [line 280]

آیا فیدر (با هر نگارشی) در دامنه انتخابی است؟ ``None`` = همه.

_pick(index: dict, key: str) [line 287]

ردیف متناظر با شناسه فیدر؛ در نبود تطبیق دقیق، تطبیق یکتای «کوتاه/آراسته» و وگرنه دیکشنری خالی.

summarize_outage(outage, feeder_keys=None) [line 296]

Outage block of the dashboard (records, classes, years, /r). When a study scope is active, live records are filtered before counts and class summaries are built. Persisted compact payloads are filtered by their per-feeder lambda rows; aggregate-only counts are not falsely presented as scope-specific.

_indices_map(outage) [line 371]

شاخص‌های پیوستگی هر فیدر (SAIDI/SAIFI/…) از OutageResult یا payload.

feeder_rows(traces, lam_rows: list[dict], snap: dict, cap_map: dict | None=None, plan_map: dict | None=None, gis_tr: dict | None=None, indices_map: dict | None=None) [line 391]

One merged row per feeder — the heart of the dashboard. ``gis_tr``: ترانس‌ها بر اساس برچسب GIS (ترانس) — ستون «ترانس» و جمع «(GIS) ترکیب رسمی فیدر) ظرفیت نامی؛ عدد ردیابی‌شده در ستون کناری می‌ماند تا اختلاف داده صریح دیده شود (ریشه‌یابی اختلاف تعداد ترانس خلخال).

build_dashboard(net: CanonicalNetwork | None=None, topo=None, outage=None, placement=None, db_path: str='', lambda_source: str='', capacity_rows=None, scope: str | list | tuple | set | None=None) [line 479]

Assemble the complete dashboard payload (JSON-safe dict).

src/switchloc/ui/inputs_model.py

جدول همه داده‌های ورودی که … placement جدول تأیید نهایی ورودی‌ها (درخواست کاربر: «قبل از انجام فرایند — Placement-inputs confirmation model در منطق محاسبات استفاده می‌شود جهت تائید نهایی به کاربر نمایش داده شود … در صورت تغییر هر کدام محاسبات با مقادیر جدید انجام گردد»). The pure layer builds ONE flat editable table (گروه / پارامتر / فیدر / مقدار) from the exact objects the pipeline consumes — the /r matrix, the load model, the feeder capacities, the cost model, the acceptance criteria, the coverage window and the ranking weights — and applies the user's edits back onto those SAME objects, so the run after «تأیید» is guaranteed to use the co

missing_load_scaling_inputs(feeder_keys, feeder_pq) [line 25]

Feeders lacking a non-zero total P or Q input for Load Scaling.

scoped_feeder_keys(keys, scope: str | list | tuple | set | None=None) [line 47]

Return feeders requested by a study scope. ``scope`` may be one feeder (legacy single-feeder API) or an explicit iterable of feeders (multi-feeder study). Selection order is normalized deterministically so calculations and persisted reports stay reproducible.

parse_number(text) [line 64]

Cell text float (Persian/Arabic digits, commas, % tolerated); None when not a number.

class InputRow [line 81]

One editable row of the confirmation table.

InputRow.as_list(self) [line 91]

collect_placement_rows(*, lam_matrix=None, capacity_rows=None, load_model=None, cost_model=None, criteria=None, coverage=None, weights=None, engine_info=None, feeder_keys=None) [line 98]

All inputs the placement/manual logic will consume InputRows. ``lam_matrix``: FeederLambdaR-like objects or dicts (feeder_key, lambda_per_year, r_minutes). ``capacity_rows``: نیاز ۹ dicts. The remaining dicts mirror the engineering-settings structure.

_num_str(v) [line 223]

apply_row_edits(rows: list[InputRow]) [line 236]

Edited rows write-back bundle for the caller. Returns ``{"lambda_overrides": {fk: (, r) | {"lam": …, "r": …} dict}, "feeder_pq": {fk: {p_kw, q_kvar}}, "load_model": {...}, "capacity_updates": {fk: {nominal_kw, peak_kw, emergency_kw}}, "cost_model": {...}, "criteria": {...}, "coverage": {...}, "weights": [...]}`` — only keys the user actually touched (or that parse cleanly) appear; unparseable edits are silently ignored (the original value stays) and reported in ``ignored``.

_months(text: str) [line 328]

apply_lambda_overrides(lam_matrix, overrides: dict) [line 332]

/r matrix (objects or dicts) with per-feeder edits applied. ``overrides``: feeder {"lambda_per_year": x, "r_minutes": y} (partial dicts allowed). Used by BOTH the ranked placement service and the manual evaluation, so the confirmed table rules over both.

src/switchloc/ui/map_renderer.py

Network map renderer — Matplotlib headless (D-04, Phase 2). Draws the NetworkGraph: edges coloured per feeder, boundary switches marked (filled = closed/NC, hollow/open diamond = NO or out-of-service), sources as squares, transformers as small dots, tie candidates starred. Persian text is intentionally NOT drawn (see ui/__init__ — RTL shaping limitation of Matplotlib); labels use technical codes only. All Persian reporting lives in the MD/JSON reports and the PySide6 shell.

_feeder_selected(feeder: str | None, selection) [line 32]

آیا این فیدر در انتخاب فعلی است؟ (نگارش کوتاه/آراسته یکی است) انتخاب ممکن است از نسخه‌های قبل با کلید کوتاه («F1») ذخیره شده باشد؛ مقایسه دقیق آن را رد می‌کرد و نقشه هیچ فیدری نشان نمی‌داد.

feeder_palette(traces: list[FeederTrace], style: MapStyle | None=None) [line 46]

feeder colour — automatic palette overlaid with the user's MapStyle colours (تنظیمات نمایشی — درخواست کاربر). Kept as a module function for backward compatibility with existing callers/tests.

edge_color_of(graph: NetworkGraph, palette: dict[str, str], unknown_color: str=UNKNOWN_COLOR) [line 58]

edge_id colour (traced feeders win over raw GIS hints). First claim wins — traces arrive sorted by feeder key, so the alphabetically-first feeder keeps the edge when several feeders' normal-state traces overlap (the A-09 flood-through), matching the global zone_of_edge rule in `topology.zones`.

switch_display_code(sw) [line 89]

Resolve the actual GIS device subtype, preferring explicit raw code.

class NetworkMapRenderer [line 109]

Feeder/zone map of the whole network or a single feeder.

NetworkMapRenderer.__init__(self, graph: NetworkGraph, traces: list[FeederTrace] | None=None, zones: ZoneResult | None=None, switches: dict | None=None, tie_switch_ids: set[str] | None=None, style: MapStyle | None=None) [line 112]

NetworkMapRenderer.render(self, path: str, title: str='', feeder_filter=None, show_devices: bool=True, figsize=(14, 12), dpi: int=150, zone_labels: bool | None=None, proposed_nodes=None, rotate: bool | None=None, manual_nodes=None, highlight_point=None, zone_load_kw: dict | None=None, selected_zone: str | None=None, hidden_switch_ids=None, zone_up_mwh: dict | None=None, zone_outage_hours: dict | None=None, outage_points=None, sectionalizer_nodes=None, hidden_outage_ids=None) [line 136]

Render to a PNG file (headless Agg) and return the path.

NetworkMapRenderer.network_is_portrait(graph: NetworkGraph) [line 170]

استفاده از «تمام فضای» قاب نقشه بود، نه landscape وقتی محدوده شبکه بلندتر از پهناست (فقط اطلاعاتی). حذف چرخش ۹۰ درجه (بازخورد کاربر): منظور از True چرخش تصویر — قاب با aspect خودکار (auto) پر می‌شود و هندسه جغرافیایی دست‌نخورده می‌ماند.

NetworkMapRenderer.render_into(self, fig, title: str='', feeder_filter=None, show_devices: bool=True, zone_labels: bool | None=None, proposed_nodes=None, rotate: bool | None=None, manual_nodes=None, style: MapStyle | None=None, highlight_point=None, zone_load_kw: dict | None=None, selected_zone: str | None=None, hidden_switch_ids=None, zone_up_mwh: dict | None=None, zone_outage_hours: dict | None=None, manual_tie_pairs=None, zone_label_positions=None, outage_points=None, sectionalizer_nodes=None, hidden_outage_ids=None) [line 185]

Draw onto an existing Matplotlib figure (Qt canvas shares this). ``proposed_nodes``: node ids of Phase-4 proposed switch locations (drawn as green stars so the placement output is visible on the very same rendering path — one rendering path, D-04). ``manual_nodes``: node ids of جایابی دستی — درخواست کاربر: «وقتی نقطه‌ای روی مپ پیک می‌شود و ارزیابی می‌شود روی نقشه نیز نمایش داده MANUALLY placed / evaluated switches شود») — blue plus markers. ``style``: MapStyle override for THIS render (falls ba

NetworkMapRenderer._data_per_point(ax, fontsize: float=5.0) [line 548]

اندازه یک «نقطه تایپوگرافی» بر حسب واحد داده محور. نقشه با ``aspect='equal'`` رسم می‌شود؛ در این حالت مقیاس هر دو محور یکی است و مقدار درست، بزرگ‌ترین نسبت «واحد داده به نقطه» (یعنی محور محدودکننده) است — برآورد محافظه‌کارانه که برچسب‌ها را کمی بزرگ‌تر از واقعیت می‌بیند و از هم‌پوشانی جلوگیری می‌کند.

NetworkMapRenderer._label_offset_pt(k: int, step_pt: float) [line 570]

فاصله عمودی k-ام (نقطه) برای برچسب‌های هم‌مکان: 0, +2s, -s, +s…

NetworkMapRenderer._draw_outage_labels(self, ax, st) [line 588]

برچسب خاموشی‌ها را **چسبیده به نقطه خودشان** می‌چیند. دو شکست قبلی و راه‌حل واحد: * **روی‌هم‌افتادن** — نسخه اول همه برچسب‌ها را در یک ستون عمودی می‌گذاشت؛ با چند نقطه نزدیک، متن‌ها هم‌پوشان می‌شدند. * **پراکندگی غیرواقعی** (تصویر ارسالی کاربر) — نسخه دوم نردبان عمودی بی‌کران (۲۴ پله) داشت و برچسب را صدها متر از نقطه دور می‌کرد. راه‌حل: برای هر برچسب فقط **۸ جهت چسبیده به نقطه** در ``outage_label_max_levels`` حلقه کوچک (پیش‌فرض ۳) آزمایش می‌شود؛ اولین جای بدون هم‌پوشانی انتخاب می‌شود، وگر

NetworkMapRenderer._c(self, coord) [line 686]

همه‌ی المان‌های ترسیم‌شده از اینجا رد می‌شوند. چرخش ۹۰° حذف شده است؛ مختصات پروژه عینا استفاده می‌شود (قاب با Coordinate accessor aspect=auto کل فضا را پر می‌کند).

NetworkMapRenderer._edge_in_feeder(self, eid: str, feeder_key: str) [line 693]

NetworkMapRenderer._draw_devices(self, ax, feeder_filter, hidden_switch_ids: set[str] | None=None) [line 700]

NetworkMapRenderer._node_in_feeder(self, node: str, feeder_key: str) [line 810]

NetworkMapRenderer._legend_handles(self, palette: dict[str, str], feeder_filter, proposed: bool=False, manual: bool=False) [line 815]

لجند کامل: رنگ فیدرها + «همه‌ی نمادهای داخل نقشه» (بازخورد کاربر — مربع، دایره، لوزی، نقطه، ستاره). فقط دسته‌هایی که واقعا در همین رندر ترسیم شده‌اند می‌آیند تا لجند دقیق بماند.

src/switchloc/ui/map_style.py

Map style model — نمایش قابل‌تنظیم نقشه (درخواست کاربر). Everything the map draws (feeder colours/thickness, switch / source / open-point / transformer / proposed / manual markers) comes from ONE `MapStyle` object, so: * the on-screen map and the report PNG stay identical (D-04); * the engineer can tune colours & marker sizes in the GUI («رنگ و ضخامت فیدرها، سایز/مارکر/نماد کلیدها، سورس open point…»); * the DEFAULT feeder colour of every feeder in the style dialog is the very colour the renderer would assign automatically («هم اعمال کن map style رنگ اولیه که برای هر فیدر تخصیص می‌دهی را در» — per-feeder distinct palette), never one shared de

_clean_heat_levels(value) [line 41]

سطوح توزیع رنگی را اعتبارسنجی می‌کند (حد بالا + رنگ معتبر).

auto_palette(traces=None, feeders=None) [line 67]

feeder_key → the automatic deterministic colour (feeder_palette). ``traces`` may be FeederTrace objects (with ``feeder_key``) or plain strings; ``feeders`` overrides the extracted key set outright.

class MapStyle [line 81]

Every tunable visual parameter of the network map.

MapStyle.default_for(cls, traces=None, feeders=None) [line 188]

Style whose feeder colours ARE the automatic per-feeder palette (user request — no shared default code in the dialog).

MapStyle.palette_of(self, traces=None, feeders=None) [line 195]

Effective feeder palette: automatic palette overlaid with the user-chosen colours (a feeder added later still gets a colour). رنگ‌های ذخیره‌شده در نسخه‌های قبل با کلید کوتاه («F1») نوشته شده‌اند؛ آن‌ها روی شناسه کامل فعلی («F1 - MESHKIN») نگاشت می‌شوند تا رنگ انتخابی کاربر گم نشود، و انتخاب مبهم با دو فیدر) روی هر دو فیدر اعمال می‌شود «F2»).

MapStyle.color_of(self, feeder_key: str) [line 218]

Colour of one feeder (style override first, then palette).

MapStyle.as_dict(self) [line 240]

JSON-safe dict (project store / style book).

MapStyle.from_dict(cls, d: dict | None) [line 251]

Rebuild from ``as_dict`` — unknown keys ignored, bad values fall back to defaults (a future field can never break the load).

merge_style_book(book: dict | None, traces=None, feeders=None) [line 292]

(styles book from the project) → (active style, active name). The book is ``{name: style_dict}``; the ACTIVE entry is whichever ``_active`` key the caller kept — this helper only guarantees a usable default when the book is empty or corrupt.

src/switchloc/ui/osm_tiles.py

OpenStreetMap raster background (نیاز ۵ — فعال/غیرفعال، زیر شبکه). Behaviour: * the caller gives the CURRENT view bbox in the project CRS (UTM metres, e.g. EPSG:32639) — this module converts the bbox to WGS84 lon/lat with pyproj, picks a Web-Mercator tile zoom that keeps the tile count small (≤ 24 tiles), downloads the tiles and reprojects the mosaic corners back into the project CRS. * the returned ``(image, extent, note)`` is drawn by the map canvas with ``imshow(..., zorder=0)`` UNDER the network layer. * OFFLINE-safe: every failure raises ``OsmUnavailableError`` and the UI degrades to the plain network map with an explicit message — a mis

class OsmUnavailableError [line 31]

پس‌زمینه OSM در دسترس نیست (آفلاین/فیلترینگ/خطای شبکه) — صریح.

class OsmBackground [line 36]

OSM mosaic ready for ``imshow`` in the project CRS.

lonlat_to_tile(lon: float, lat: float, zoom: int) [line 47]

WGS84 (lon, lat) → Web-Mercator tile indices (slippy map).

choose_zoom(lon_min: float, lon_max: float, lat_min: float, lat_max: float, max_tiles: int=MAX_TILES) [line 57]

Highest zoom whose bbox tile count stays within ``max_tiles``.

_fetch(url: str, timeout: float=8.0) [line 70]

Download helper — isolated for tests (mock target).

fetch_bbox_background(x0: float, x1: float, y0: float, y1: float, crs: str | int | None=None, max_tiles: int=MAX_TILES, fetch_fn=None) [line 78]

OSM mosaic of the bbox (project CRS metres) → OsmBackground. Raises ``OsmUnavailableError`` on ANY failure (network, decoding, pyproj) — the caller shows a message and continues without tiles.

src/switchloc/ui/qt/__init__.py

PySide6 shell (اختیاری — `requirements-qt.txt`). This package imports PySide6 lazily: importing `switchloc.ui` alone must never fail on a headless/offline machine (the engineering pipeline and tests do not need Qt). Use `qt_available()` to probe.

qt_available() [line 11]

src/switchloc/ui/qt/charts.py

نمودارهای ستونی سبک برای داشبورد (بدون وابستگی تازه). چرا QPainter و نه Matplotlib؟ برچسب‌ها فارسی‌اند و Qt شکل‌دهی (shaping) و چیدمان راست‌به‌چپ فارسی را درست انجام می‌دهد؛ در Matplotlib متن فارسی بدون reshaper/bidi به‌صورت حروف جدا و برعکس رسم می‌شود. این ویجت فقط می‌خواند و هیچ‌چیز را در مدل تغییر نمی‌دهد.

_fmt_value(v: float) [line 18]

نمایش عدد: هزار جداکننده، بدون صفرهای اضافی.

class BarChartWidget [line 34]

یک نمودار ستونی افقی: عنوان، واحد و یک میله برای هر فیدر. ``items``: فهرست ``(label, value, color)``؛ مقدار ``None`` یعنی «بدون داده» و با «—» نشان داده می‌شود (صفر نمایش داده نمی‌شود تا «نداشتن داده» با «صفر بودن» اشتباه نشود).

BarChartWidget.__init__(self, title: str='', unit: str='', parent=None) [line 42]
BarChartWidget.set_data(self, items, title: str | None=None, unit: str | None=None) [line 53]
BarChartWidget.paintEvent(self, event) [line 68]
chart_items(rows: list, key: str) [line 165]

داده یک شاخص از ردیف‌های فیدر داشبورد آیتم‌های نمودار.

src/switchloc/ui/qt/chrome.py

Application chrome: menus, toolbar, side navigation, status bar, recent projects, drag-and-drop open, theme/language switching. The chrome contains NO engineering logic: it only wires callables handed in by ``main_window.launch`` (``handlers``) and decorates the existing widgets. Qt is imported lazily so the pure helpers (``workflow_html``, ``normalize_recent``, ``push_recent``) are unit-testable headless.

normalize_recent(raw) [line 30]

QSettings may hand back ``None``, a str (single item) or a list.

push_recent(current: list[str], path: str) [line 44]

Most-recent-first, de-duplicated, capped at ``MAX_RECENT``.

workflow_html(state_name: str, tokens: dict) [line 50]

Stepper text for the workflow strip (rich text, colour-coded).

class Chrome [line 73]

Built once by :func:`install_chrome`; main_window keeps the handle.

Chrome.__init__(self) [line 76]
Chrome.settings() [line 95]
Chrome.recent(self) [line 99]
Chrome.add_recent(self, path: str) [line 106]
Chrome.clear_recent(self) [line 114]
Chrome._rebuild_recent_menu(self) [line 121]
Chrome._icon(self, name: str, kind: str='normal') [line 142]

kind: normal | nav (on the dark sidebar) | primary (on the accent-coloured button) — each has its own contrast colour.

Chrome.decorate(self, target, name: str, kind: str='normal') [line 150]

Give a button/action an icon and remember it for theme switches.

Chrome.style_button(self, btn, icon_name: str | None=None, primary: bool=False) [line 158]
Chrome._refresh_icons(self) [line 167]
Chrome.status(self, text: str, ms: int=5000) [line 176]
Chrome.set_project_info(self, name: str='', feeders: int | None=None, crs: str='') [line 182]
Chrome.set_workflow(self, state_name: str) [line 195]
Chrome.set_theme(self, mode: str, persist: bool=True) [line 201]
Chrome.toggle_theme(self) [line 215]
install_chrome(win, app, tabs, workflow_label, handlers: dict, assets_dir: Path | None, version: str='') [line 222]

Build menus / toolbar / sidebar / status bar around ``tabs``. ``handlers`` keys: open_project(path|None), import_gis, import_outage, per_feeder, topology, placement, settings, exports, approve, map_style, help, about. Returns the :class:`Chrome` handle whose ``central`` widget must become the window's central widget.

src/switchloc/ui/qt/dashboard_tab.py

Dashboard tab — the management view (Phase 3/4 results at a glance). Layout (top bottom): 1. KPI cards — feeders / zones / switches / / worst feeder / proposed switches / selected schemes / issues 2. charts — per-feeder , ranking score, EP% (one Figure, matplotlib only — the data+chart code is Qt-free and Agg-testable; only the wrapper touches Qt) 3. feeder matrix — one merged row per feeder (trace + /r + EP/TL + rank + selected scheme) 4. table 18 — CE/CN comparison of the switching schemes 5. notes — source, outage summary, issues (Persian text from the results, rendered by Qt — RTL is fine in widgets; only Matplotlib cannot shape

draw_charts(fig, dash: dict) [line 31]

/ score / EP per feeder as three bar panels on ``fig``.

_build_chart_section() [line 69]

بخش نمودارها (ویجت، {key: نمودار})؛ ایمپورت Qt تنبل است.

_fmt(v, spec: str=':g') [line 99]
make_dashboard_tab(parent=None) [line 112]

Return the dashboard widget exposing ``show_dashboard(dash)``.

src/switchloc/ui/qt/i18n.py

Tiny catalog-based translation for the application chrome. Scope (honest): the navigation chrome — menus, toolbar, sidebar, status bar, main buttons, tab titles, workflow stepper and the new dialogs — is available in English and Persian. Long engineering panels keep their existing bilingual wording. The language is stored in ``QSettings`` and applied at the next start (``apply_layout_direction`` flips RTL for fa). Qt-free: ``t()`` is a plain dict lookup, unit-testable.

set_language(lang: str) [line 91]
get_language() [line 95]
is_rtl() [line 99]
t(text: str) [line 103]

Translate a chrome string (identity for English / unknown keys).

src/switchloc/ui/qt/icons.py

Built-in vector icon set (24×24 line icons, drawn from SVG paths). No image files to ship: each icon is an inline SVG string rendered with ``QSvgRenderer`` at the requested size/colour (HiDPI-aware). ``icon()`` never raises — if ``QtSvg`` is unavailable it returns an empty ``QIcon`` and the UI simply shows text labels.

available() [line 46]
svg_text(name: str, color: str='#37474F', stroke: float=1.9) [line 50]

Complete SVG document for an icon (Qt-free — testable).

icon(name: str, color: str='#37474F', size: int=24, disabled_color: str='#9AA5AB') [line 62]

``QIcon`` for ``name`` (empty icon on any failure — never raises).

clear_cache() [line 106]

Drop cached pixmaps (call after a theme switch).

src/switchloc/ui/qt/inputs_confirm_dialog.py

Placement-inputs confirmation dialog — ۵ مورد (درخواست کاربر، تأیید نهایی ورودی‌ها). Shows ONE sortable/copyable/editable table of EVERY input the placement/manual-evaluation logic consumes (/r بار هر فیدر، Load Scaling، و پنجره پوشش داده، معیارهای پذیرش، هزینه‌ها، ظرفیت‌ها). Only «تأیید و اجرا» (وزن‌های جدول ۱۶) lets the run proceed; the edited values are written back through the Qt-free `apply_row_edits` so the calculation really uses the confirmed numbers. Discipline: the data logic lives in ui/inputs_model.py; this module is the thin Qt glue (same split as settings_dialog).

make_inputs_confirm_dialog(parent, rows: list[InputRow], title: str='تأیید نهایی داده\u200cهای ورودی محاسبات') [line 20]

Return the dialog; caller exec()s it and on accept reads ``dlg.result_edits()`` (the apply_row_edits bundle) — or None when nothing was touched (fast path).

src/switchloc/ui/qt/main_window.py

MainWindow — PySide6 shell (Phases 1-5 + نیازهای ۱-۱۲, English UI). Five tabs, zero engineering logic (ARCHITECTURE.md rule 1) — each tab drives one application service and shows its result: 1. Data Import — pick the GIS folder or open a project, mapping wizard, QC verdict + a persistent result-summary panel, and the outage-list import: ONE workbook for all feeders **or** a per-feeder outage file entered next to each feeder (نیاز/گردش کار مطالعات), with an explicit study-scope box (single/multi-feeder). 2. Topology — run TopologyService on the open project; results panel with feeders / zones / tie-switch candidates (A-09), the engineer's conf

_asset_dir() [line 63]

Bundled assets: inside a PyInstaller onefile exe -> sys._MEIPASS, in development -> <project>/build_assets.

_write_log(text: str) [line 72]

Append text to the central rotating log (never raises).

_fmt_exc(exc: BaseException) [line 80]
_workspace_dir() [line 84]

Writable base folder for ``projects/`` and ``reports/``. ``SWITCHLOC_HOME`` > current directory > ``~/Documents/SwitchLoc`` (a packaged exe may sit in a read-only location such as Program Files).

_projects_dir() [line 107]
_reports_dir() [line 111]
_close_installer_splash() [line 115]

Close the PyInstaller splash screen (no-op in development runs).

fmt_import_result(res) [line 130]

ImportResult -> multi-line English summary for the Import tab.

fmt_topology_result(res) [line 153]

TopologyResult -> multi-line English summary for the Topology tab.

fmt_placement_result(res) [line 183]

PlacementResult -> multi-line English summary for the Placement tab.

fmt_outage_result(res) [line 253]

OutageResult -> multi-line English summary for the Import tab.

_feeder_load_kw(state) [line 331]

بار هر فیدر (kW) برای شاخص‌های SAIDI/SAIFI. منبع: P واردشده در «مدل بار» (همان P_load کاربر). اگر برای فیدری P وارد نشده باشد، هیچ عددی ساخته نمی‌شود: شاخص آن فیدر فقط از قطع‌کننده (بریکر/ریکلوزر) و به‌صورت «کف» محاسبه می‌شود و علتش در گزارش می‌آید.

launch(db_path: str | None=None) [line 351]

Start the desktop shell; returns a Qt exit code.

src/switchloc/ui/qt/map_canvas.py

Map canvas — interactive Qt map tab around the headless renderer. The Qt canvas embeds the very same `NetworkMapRenderer` used for report PNGs, so the on-screen map and the printed report can never diverge (one rendering path — D-04 discipline). Interactive zoom pack (plain Matplotlib events, no extra dependency): * mouse-wheel zoom, anchored at the cursor position * grab-and-drag panning with the left mouse button * double-click = zoom in at the cursor * «ذره‌بین» region zoom — drag a rectangle, the view fills it (نیاز ۴) * toolbar buttons: Zoom in / Zoom out / Fit view / Region zoom * zoom indicator on a 0–100 % scale (user feedback: «اسکیل

zoom_limits(lo: float, hi: float, anchor: float, scale: float) [line 60]

New ``(lo, hi)`` after zooming by ``scale`` around ``anchor``. ``scale < 1`` zooms in (the span tightens), ``scale > 1`` zooms out; the point ``anchor`` keeps its relative position inside the interval (cursor-anchored zoom).

clamp_span(lo: float, hi: float, anchor: float, home_lo: float, home_hi: float) [line 71]

Snap a zoomed span back inside the allowed window (same anchor). Used by the engine when a requested step overshoots the cap: instead of refusing the whole step the span lands exactly on the limit, so the user can always reach the full 10 000x depth with wheel notches.

span_ok(lo: float, hi: float, home_lo: float, home_hi: float) [line 89]

True when ``(lo, hi)`` is inside the allowed zoom window. The window is [full extent / ZOOM_IN ... full extent * ZOOM_OUT] plus an absolute floor — zooming never degenerates to a zero span. A relative 1e-9 tolerance absorbs the floating-point reconstruction error of ``clamp_span`` (``lo2 + span - lo2`` loses ~1e-10 on UTM-scale coordinates, which otherwise refuses the LAST wheel step exactly at the cap).

_close(a: float, b: float) [line 109]

class MapCanvasMixin [line 113]

Qt-free zoom/pan engine for a single-axes Matplotlib canvas. Expects from the host canvas: ``self.fig``, ``draw_idle()``, ``mpl_connect()`` and — optionally — ``setCursor()`` (guarded).

MapCanvasMixin._init_state(self) [line 120]

MapCanvasMixin.set_pick_mode(self, on: bool) [line 137]

جایابی دستی: کلیک بعدی روی نقشه، مختصات آن را در CRS پروژه برمی‌گرداند.

MapCanvasMixin.connect_events(self) [line 142]

MapCanvasMixin.show_network(self, renderer: NetworkMapRenderer, title: str='', feeder_key: str | None=None, zone_labels: bool=False, proposed_nodes=None, rotate: bool | None=None, osm_image=None, osm_extent=None, osm_note: str='', manual_nodes=None, highlight_point=None, zone_load_kw=None, selected_zone=None, hidden_switch_ids=None, zone_up_mwh=None, zone_outage_hours=None, manual_tie_pairs=None, outage_points=None, sectionalizer_nodes=None, hidden_outage_ids=None) [line 150]

MapCanvasMixin._style_map_frame(ax) [line 268]

Border of the displayable map area (user feedback: a frame must mark the visible map extent). Ticks stay hidden — only the frame rectangle is drawn.

MapCanvasMixin._scale_line_widths(self, pct: float | None) [line 290]

Scale conductor widths with the zoom level (legibility). At 100 % the renderer widths are unchanged; at 100 000 % the width is up to 3x so single spans stay visible instead of collapsing into sub-pixel hairlines.

MapCanvasMixin.main_ax(self) [line 310]

MapCanvasMixin._zoom_axes(self, ax, x: float, y: float, scale: float) [line 315]

Apply a cursor-anchored zoom; False when the step is refused. A step that overshoots the cap is snapped exactly onto the cap (returns True once); when already at the cap the step is refused so callers/loops see a real end of the zoom range.

MapCanvasMixin._zoom_center(self, scale: float) [line 344]

MapCanvasMixin.fit_view(self) [line 351]

Restore the full-extent view (home limits).

MapCanvasMixin.set_region_mode(self, on: bool) [line 361]

Toggle the rubber-band region-zoom mode (magnifier tool).

MapCanvasMixin._discard_region(self) [line 368]

MapCanvasMixin.region_rect_data(self) [line 378]

((x0, x1), (y0, y1)) of the current rubber band (test hook).

MapCanvasMixin.zoom_to_region(self, ax, x0: float, y0: float, x1: float, y1: float) [line 389]
Fill the view with the given data-space rectangle (نیاز ۴). Degenerate rectangles (zero span after the display-pixel filter) are refused; the span caps of `span_ok` still apply, so a tiny marquee never degenerates the axes.
MapCanvasMixin._on_region_press(self, ev) [line 425]
Start the rubber band; True when the event was consumed.
MapCanvasMixin._on_region_motion(self, ev) [line 440]
MapCanvasMixin._on_region_release(self, ev) [line 451]
Finish the rubber band and zoom; True when consumed.
MapCanvasMixin._px_per_data_x(ax) [line 467]
MapCanvasMixin._px_per_data_y(ax) [line 475]
MapCanvasMixin._magnifier_cursor(self, on: bool) [line 482]
MapCanvasMixin._notify(self) [line 490]
MapCanvasMixin.zoom_pct(self) [line 496]
Current zoom level in % (100 = full extent).
MapCanvasMixin.zoom_level_pct(self) [line 509]
Normalized zoom level 0-100 % on a log scale (user feedback: the zoom scale must display up to 100 %). 0 % = full view, 100 % = the deepest zoom (ZOOM_IN); every wheel notch moves the needle uniformly.
MapCanvasMixin._on_scroll(self, ev) [line 529]
MapCanvasMixin._on_pick_event(self, ev) [line 535]
MapCanvasMixin._zone_label_hit(ax, ev) [line 541]
MapCanvasMixin._on_press(self, ev) [line 555]
MapCanvasMixin._on_motion(self, ev) [line 588]
MapCanvasMixin._on_release(self, ev) [line 614]
MapCanvasMixin._grab(self, on: bool) [line 635]
make_map_canvas(parent=None, osm_fetch=None) [line 644]
Return the interactive map tab widget (map + switch visibility panel). Switch visibility controls are per physical GIS switch ID. Hiding a switch affects rendering only; the network graph, zone model and engineering calculations are never mutated.

src/switchloc/ui/qt/map_style_dialog.py

Map style dialog — درخواست کاربر (تنظیمات نمایشی نقشه). Features (per the user's two requests): 1. the feeder colour table opens pre-filled with the EXACT colours the renderer assigns automatically (per-feeder palette — no shared default code); every colour/thickness/marker is then editable; 2. MULTIPLE NAMED STYLES: the dialog keeps a book ``{name: MapStyle}`` — «ثبت تغییرات» overwrites the active name, «ذخیره با نام جدید…» creates one, «حذف استایل» removes one — and the caller persists the book inside the project (project_kv ``map_styles``) so closing/reopening the project restores the exact settings. Discipline: the pure helpers (`feeder
feeder_rows(style: MapStyle, feeders: list) [line 24]
(feeder, colour) rows for the colour table — every modelled feeder gets a row with the colour the renderer would use (style override first, then the automatic per-feeder palette).
collect_style(editor: dict) [line 33]
Editor state (dict of fieldvalue) MapStyle (validated).
fit_dialog_to_screen(dlg, *, margin: int=48, min_w: int=560, min_h: int=420) [line 134]
پنجره را داخل صفحه نمایش جا می‌دهد و **وسط صفحه** می‌گذارد. درخواست کاربر: «قسمت پایین پنجره (دکمه OK) خارج از دید است، پنجره را نسبت به صفحه سنتر کن». بنابراین اندازه به ناحیه قابل‌استفاده صفحه محدود می‌شود (تا نوار عنوان و دکمه‌ها همیشه دیده شوند) و سپس مرکز همان ناحیه قرار می‌گیرد — نه مرکز پنجره والد.
make_map_style_dialog(parent, feeders: list, current: MapStyle, book: dict | None=None, active_name: str='') [line 158]
Return the dialog; caller does exec() then reads ``dlg.result_style() / dlg.result_book() / dlg.result_active()``.

src/switchloc/ui/qt/mapping_wizard.py

Mapping wizard — Qt front-end of the layer/field mapping service (D-04). Wraps `gis.mapping.LayerMappingService` (the Phase-1 service — unchanged): suggest() suggestion table with confidence/reason user review confirm() with recorded confirmer (no automatic mapping is ever final — rule §5-3). The engineering logic lives in the service; this dialog only binds Qt widgets to it. (The former placeholder tab builders were replaced by the rich tab panels in `main_window.launch()`.)
_reason_en(suggestion) [line 33]
class MappingWizardPage [line 41]
Review-and-confirm dialog over MappingSuggestions (requires Qt).
MappingWizardPage.__init__(self, suggestions, mapping: LayerMappingService, layers: list[LayerInfo], parent=None) [line 44]
MappingWizardPage._item(text: str) [line 77]
MappingWizardPage.exec(self) [line 81]
MappingWizardPage.apply(self) [line 87]
Write the user's (possibly edited) decisions into the service.

src/switchloc/ui/qt/outage_history_dialog.py

Read-only, model-backed inspection of saved outage events (no topology needed).

show_outage_history(parent, records) [line 5]
show_saved_workbooks(parent, db_path) [line 80]

src/switchloc/ui/qt/outage_ranking_dialog.py

دیالوگ «رتبه‌بندی مکان‌های خاموشی» (درخواست کاربر). فهرست مکان‌ها را بر اساس **تعداد تکرار خاموشی** مرتب می‌کند: رتبه، مکان، فیدر، تعداد، سهم درصدی، توزیع وضعیت بازبینی، زون غالب و مختصات مرکز. خروجی قابل ذخیره به‌صورت CSV است تا در گزارش رسمی استفاده شود. منطق گروه‌بندی در application/outage_review.py::outage_place_ranking`` است Qt-free) و آزمون‌شده)؛ این فایل فقط نمایش است.``

show_outage_ranking_dialog(parent, points, *, title: str='', on_focus=None) [line 14]

نمایش جدول رتبه‌بندی مکان‌های خاموشی؛ برگرداندن دیالوگ اجراشده.

src/switchloc/ui/qt/outage_review_dialog.py

دیالوگ بازبینی محل/زون خاموشی‌ها (جدول اکسل‌مانند). سه رفتار خواسته‌شده در همین دیالوگ پیاده شده است: ۱. **زون خودکار** — هر ردیف بر اساس فیدر رخداد و مختصات «مکان پیشنهادی» زون می‌گیرد (`application.outage_review.pick_zone`)؛ مبنای انتخاب در ستون «مبنای زون» ثبت می‌شود (بند ۱۹). با تغییر «مکان پیشنهادی» یا تغییر دستی زون، مبنا هم به‌روز می‌شود. ۲. **مکان پیشنهادی اول فهرست** — بهترین تطبیق بانک آفلاین اولین گزینه و انتخاب پیش‌فرض است (`place_options`). ۳. **وضعیت پیش‌فرض «تأیید مهندسی»** برای همه ردیف‌هایی که زون دارند؛ ردیف بدون زون «در انتظار بررسی» می‌ماند و صریح علامت می‌خورد، چون تأیید بدون زون یعنی انتساب حدسی. منطق تصمیم‌گیری (plans)

make_outage_review_dialog(parent, *, records, plans, geometries=None, max_zone_distance_m: float=DEFAULT_MAX_ZONE_DISTANCE_M, bank_line: str='', info_extra: str='', transform=None) [line 26]

دیالوگ بازبینی؛ پس از ``exec()`` مقدار ``dlg.decisions()`` خوانده می‌شود.

src/switchloc/ui/qt/preflight_dialog.py

پنجره ری‌چک کامل EENS قبل از اجرای جایابی بهینه» :(v1.15) درخواست کاربر .EENS پنجره «ری‌چک ورودی‌ها» پیش از اجرای جایابی — preflight_dialog.py ورودی‌ها برای بررسی نهایی باز شود؛ در حالت فعلی با داده بار پیش‌فرض کار می‌کند.» پنجره فقط **نشان می‌دهد**؛ هیچ عددی را تغییر نمی‌دهد. اگر خطایی وجود داشته باشد دکمه اجرا غیرفعال می‌شود تا اجرای بی‌معنا (مثلا بار صفر یا دامنه خالی) رخ ندهد. هشدارها اجرا را نمی‌بندند ولی صریح می‌گویند کدام ورودی پیش‌فرض/تخمینی است.

make_preflight_dialog(parent, report: PreflightReport, *, title: str='ری\u200cچک ورودی\u200cکلید بهینه پیش از اجرای جایابی ها (EENS)') [line 18]

دیالوگ تأیید ورودی‌ها را می‌سازد و برمی‌گرداند (``exec()`` لازم است).

src/switchloc/ui/qt/sectionalizer_tab.py

نام قدیمی تب — به :mod:`switchloc.ui.qt.switch_placement_tab` منتقل شد.

src/switchloc/ui/qt/settings_dialog.py

Engineering settings dialog (تنظیمات مهندسی در نیاز ۱۲ — GUI). One dialog, five pages — every page edits a plain dict; the caller applies the dict to the engine settings (the dialog itself holds no engineering logic, same split as dashboard_tab): * Load model (نیاز ۱۱) — level + loading/substitute factors * Feeder capacity (نیاز ۹) — nominal / peak / emergency per feeder * Costs — NC / NO / automation / rial-per-kWh / switching minutes (company policy, logged) * Acceptance criteria (نیاز ۱۰) — ENS thresholds, caps, budget * Data coverage (نیاز ۸) — explicit coverage window + forced / excluded months (the «missing months ≠ no outage» control

default_capacity_rows(feeder_keys) [line 31]

New rows use an explicitly marked, editable engineering default.

_lookup_by_feeder(mapping: dict, key: str) [line 39]

مقدار ذخیره‌شده فیدر با هر نگارشی («F3»/«F3 - KOSAR»).

parse_capacity_rows(table_rows) [line 50]

Raw GUI rows clean dicts (None for unknown, no silent zero).

make_settings_dialog(parent, feeder_keys, current: dict, feeder_s_rated: dict | None=None) [line 79]

Return the dialog instance; exec() + apply() by the caller. ``feeder_s_rated``: feeder_key S_rated [kVA] (برای پیش‌نمایش زنده — GIS از ترانس‌های) محاسبات Load Scaling (K و cos) روی همان صفحه.

make_exports_dialog(parent, formats=('xlsx', 'docx', 'pdf', 'gis')) [line 388]

Output-format selection dialog (انتخاب فرمت خروجی نیاز ۱۲ —).

src/switchloc/ui/qt/splash.py

Splash screen with a real progress bar (نوار لودینگ نیاز ۲ —). A `QSplashScreen` drawn from the bundled splash image hosts a `QProgressBar`; every start-up step (`step(msg, weight)`) advances the bar and keeps the event loop alive. The splash stays visible until the main window is actually shown (`finish(win)`) — the bar never lies: its weights sum to 100 % across the real init steps. Qt lives only inside the factory (`make_splash`), the weights table is Qt-free and testable (same discipline as map_canvas.py / dashboard_tab).

cumulative_progress(done_steps: int) [line 26]

Bar value (0-100) after ``done_steps`` completed steps.

make_splash(app) [line 32]

Return ``(splash, step_fn)``; ``(None, no-op)`` without assets.

src/switchloc/ui/qt/switch_placement_tab.py

تب «روش جایابی بهینه کلید و کاهش EENS». بازطراحی v1.13 (درخواست کاربر): «این پنجره به دو بخش باکس‌های کنار هم تبدیل شود، جدول نتایج برگردد، و موارد قفل‌شده کم‌رنگ شوند». ۱) نوع کلید و روش خلاصه نتیجه (KPI: پایه نهایی ٪) ۲) نرخ خرابی و زمان‌ها ۳) خطای گذرا (ریکلوزر) جدول نتایج فیدرها ۴) محدودیت‌های جست‌وجو [اجرای جایابی] [نقشه] جدول کلیدهای پیشنهادی (نتیجه نهایی) وضعیت

make_sectionalizer_tab(parent=None) [line 35]

ساخت تب «روش جایابی بهینه کلید» (Qt) و برگرداندن کنترلر آن.

src/switchloc/ui/qt/theme.py

Professional light/dark theme (Fusion style + one stylesheet). ``build_qss(tokens)`` is pure string work (Qt-free, unit-testable); ``apply_theme(app, mode)`` installs the Fusion style, a matching ``QPalette`` and the stylesheet. Colours live in ``LIGHT`` / ``DARK`` — nothing else in the app hard-codes the look, so a theme switch is one call.

build_qss(c: dict) [line 40]

Application stylesheet for the token set ``c``.

palette_for(c: dict) [line 153]

A ``QPalette`` matching the tokens (so native-drawn bits agree).

load_fonts(assets_dir: Path) [line 172]

Register bundled Vazirmatn fonts; returns the loaded family names.

apply_theme(app, mode: str='light', assets_dir: Path | None=None, base_font_pt: int=10) [line 188]

Install Fusion + palette + stylesheet + fonts; returns the tokens.

src/switchloc/ui/qt/widgets.py

Shared Qt widgets/helpers for the SwitchLoc shell (درخواست کاربر). * explanation blocks with a distinct blue background («قسمت هایی که به عنوان توضیحات هستند با یک بک گراند مثلا آبی رنگ مشخص کن») * required-field marking with a red asterisk («فیلدهای الزامی ستاره‌دار») * every table sortable by its column headers («قابلیت سورتینگ بر اساس هر هدر ستون») with numeric-aware ordering (12 > 9, not "12" < "9") * copy (Ctrl+C) / paste (Ctrl+V) on ALL input and output tables («قابلیت کپی کردن و پیست کردن را برای تمامی جداول فعال کن») — clipboard format is TSV so Excel/Word paste round-trips. Discipline: the pure logic (`cmp_cells`, `rows_to_tsv`, `tsv

num_or_text(s) [line 28]

Parse a cell as a number (comma thousand separators tolerated).

_natural_key(s: str) [line 39]

Natural-sort key: digit runs compare as numbers (F2 < F10).

cmp_cells(a: str, b: str) [line 46]

Numeric-aware cell comparison for table sorting (-1/0/1). Two numbers compare as numbers; number < text; text compares case-insensitively with NATURAL digit runs — so «9» sorts before «12» and «F2» before «F10».

required_html(text: str) [line 64]

Label text with a red asterisk (required field, user request).

rows_to_tsv(rows: list) [line 69]

Table selection TSV text (clipboard-friendly, Excel round-trip).

tsv_to_rows(text: str) [line 77]

TSV text rows of strings (paste source; handles CRLF).

make_explain_label(text: str, parent=None) [line 92]

Compact per-section Help control; long explanations open on demand. Kept under the historical function name so every existing section automatically gains the same space-saving help interaction.

make_required_label(text: str, parent=None) [line 128]

Form label with the red required asterisk. NOTE (bug fix — switchloc_run_log.txt): ``setTextFormat(1)`` crashes on strict PySide6 builds («called with wrong argument types … setTextFormat(int)») — the argument MUST be the ``Qt.TextFormat`` enum, never a bare int.

make_table_item(text: str, editable: bool=False) [line 143]

Cell item with numeric-aware sorting (user request: هر هدر ستون).

_cell_text(table, i: int, j: int) [line 158]

Cell text — cell widgets first (their shown text), else the item.

_selected_rows(table) [line 175]

Selection row-wise TSV block. Excel-like behaviour: the copy covers the *bounding rectangle* of the selection (cells inside the rectangle that

are not part of the selection are emitted as empty cells). Cell-widget columns (e.g. combo boxes) contribute their shown text.
_paste_into_cell(table, i: int, j: int, val: str) [line 206]
Paste one cell: combo closest match (text/data), editable item set text. Returns True when the cell accepted the value.
_paste_rows(table, rows: list) [line 246]
Paste TSV rows starting at the current cell (bounds-clipped). Returns (pasted, skipped) cell counts so the caller can report read-only/unmatched cells honestly.
enable_copy_paste(table, editable: bool=False, on_paste=None) [line 271]
Ctrl+C copies the selection as TSV; Ctrl+V pastes into editable cells from the current cell — applied to ALL tables (user request). Cell-widget columns (combo boxes etc.) copy/paste their shown text. When ``on_paste(pasted, skipped)`` is given it is called after a paste.
_select_line(table, is_column: bool, idx: int, modifiers) [line 339]
Header click select the whole column/row (Excel-like). * plain click: select only that line * Ctrl+click: toggle that line in the selection * Shift+click: range from the last anchor (stored on the table)
make_excel_like(table, *, paste: bool=True, header_select: bool=True, on_paste=None) [line 395]
Make a review/result table behave like an Excel grid (user request): * extended range selection (click, Shift=range, Ctrl=toggle) * clicking a ROW or COLUMN header selects the entire line (Shift extends from the anchor, Ctrl toggles) * Ctrl+C / Ctrl+Ins copy the selection as TSV (Excel round-trip; combo-box columns contribute their shown text) * Ctrl+V / Shift+Ins paste TSV from the current cell (combo cells match by text/data; read-only cells are skipped and reported via ``on_paste(pasted, skip
fill_table(table, headers: list, rows: list, editable_cols: set | None=None, stretch_last: bool=True) [line 449]
Fill a table with sortable, copyable, numeric-aware cells. ``rows``: list of string lists; ``editable_cols``: column indexes the user may edit. Sorting is toggled off during the fill (Qt rule) and re-enabled afterwards — every header click sorts (user request).

src/switchloc/ui/qt/worker.py

Background execution for heavy services (topology, placement, outages, import, exports) — the GUI thread never blocks. Design (all Qt objects are created lazily so this module imports without PySide6, and the Qt-free parts are unit-testable): * ``run_task(parent, label, work, on_success, on_error=…)`` runs ``work()`` in a ``QThread``; ``on_success(result)`` / ``on_error(exc)`` are ALWAYS invoked on the GUI thread (delivered through a controller ``QObject`` that lives in the GUI thread — never through a bare lambda, whose thread affinity differs between Qt bindings). * While a task runs a busy ``QProgressDialog`` blocks input to the parent win
class _GuiRequest [line 42]
A callable to run on the GUI thread plus its outcome slot.
_GuiRequest.__init__(self, fn: Callable[[], Any]) [line 45]
is_busy() [line 52]
call_in_gui(fn: Callable[[], Any]) [line 56]
Run ``fn()`` on the GUI thread and return its result. Called from the worker thread; when there is no worker (sync mode or plain GUI thread) it simply calls ``fn()``.
_sync_mode(sync: bool | None) [line 73]
_default_error(parent, title: str, exc: BaseException) [line 79]
error_box(parent, title: str) [line 86]
``on_error`` callback: a critical message box with the log path.
run_task(parent, label: str, work: Callable[[], Any], on_success: Callable[[Any], None], on_error: Callable[[BaseException], None] | None=None, title: str='SwitchLoc', sync: bool | None=None) [line 91]
Run ``work`` off the GUI thread; return False if rejected (busy).
install_excepthook(parent_getter: Callable[[], Any] | None=None) [line 218]
Route uncaught exceptions (incl. those raised in Qt slots) to the log and a message box — a crash is never silent and never lost.

src/switchloc/validation/__init__.py

validation — Issue model (three severities) + QC engine/report.

src/switchloc/validation/issues.py

model of validation issues (main-prompt §19, GeoGrid-audit §10 decision). An issue is *the* unit of engineering traceability: every exclusion, assumption or data gap in SwitchLoc must surface as one of these.
class ValidationIssue [line 18]
One engineering/data discrepancy (main prompt §5-7). Fields mirror the main prompt exactly: source file, layer, record id, related field, current value, suggested fix, study impact, status.
ValidationIssue.as_dict(self) [line 37]
class IssueLog [line 41]
Ordered, filterable issue collector with stable ids (SL-0001...).
IssueLog.__init__(self) [line 44]
IssueLog.add(self, issue: ValidationIssue) [line 47]
IssueLog.extend(self, issues) [line 52]

IssueLog.count(self, severity: Optional[str]=None) [line 56]
IssueLog.by_code(self, code: str) [line 61]
IssueLog.items(self) [line 64]
IssueLog.has_errors(self) [line 67]

src/switchloc/validation/qc.py

QC engine + report (main prompt §۱۹ / ARCHITECTURE.md §۳). The reader/assembler already emit the atomic issues; the QC engine *aggregates and judges*: per-severity/code/layer counts, the acceptance verdict (0 unresolved Error issues) and a Persian markdown report. Nothing here mutates the underlying issues.

class QcReport [line 18]
QcReport.as_dict(self) [line 30]
QcReport.to_markdown(self) [line 43]
run_qc(network_summary: dict, issues: IssueLog, project: str, layer_feature_counts: dict | None=None) [line 76]